

```

#!/usr/bin/env perl
# mysqltuner.pl - Version 1.7.24
# High Performance MySQL Tuning Script
# Copyright (C) 2006-2020 Major Hayden - major@mhtx.net
#
# For the latest updates, please visit http://mysqltuner.pl/
# Git repository available at https://github.com/major/MySQLTuner-perl
#
# This program is free software: you can redistribute it and/or modify
# it under the terms of the GNU General Public License as published by
# the Free Software Foundation, either version 3 of the License, or
# (at your option) any later version.
#
# This program is distributed in the hope that it will be useful,
# but WITHOUT ANY WARRANTY; without even the implied warranty of
# MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
# GNU General Public License for more details.
#
# You should have received a copy of the GNU General Public License
# along with this program. If not, see <https://www.gnu.org/licenses/>.
#
# This project would not be possible without help from:
#   Matthew Montgomery      Paul Kehrer      Dave Burgess
#   Jonathan Hinds          Mike Jackson    Nils Breunese
#   Shawn Ashlee            Luuk Vosslander Ville Skytta
#   Trent Hornibrook       Jason Gill      Mark Imbriaco
#   Greg Eden              Aubin Galinotti Giovanni Bechis
#   Bill Bradford           Ryan Novosielski Michael Scheidell
#   Blair Christensen      Hans du Plooy   Victor Trac
#   Everett Barnes          Tom Krouper     Gary Barrueto
#   Simon Greenaway         Adam Stein      Isart Montane
#   Baptiste M.             Cole Turner     Major Hayden
#   Joe Ashcraft            JeSan-Marie Renouard Christian Loos
#   Julien Francoz          Daniel Black
#
# Inspired by Matthew Montgomery's tuning-primer.sh script:
# http://www.day32.com/MySQL/
#
package main;

use 5.005;
use strict;
use warnings;

use diagnostics;
use File::Spec;
use Getopt::Long;
use Pod::Usage;
use File::Basename;

```

```

use Cwd 'abs_path';

use Data::Dumper;
$Data::Dumper::Pair = " : ";

# for which()
#use Env;

# Set up a few variables for use in the script
my $tunerver = "1.7.24";
my ( @adjvars, @generalrec );

# Set defaults
my %opt = (
    "silent"           => 0,
    "nobad"            => 0,
    "nogood"           => 0,
    "noinfo"           => 0,
    "debug"            => 0,
    "nocolor"          => ( !-t STDOUT ),
    "color"            => 0,
    "forcemem"         => 0,
    "forceswap"        => 0,
    "host"             => 0,
    "socket"           => 0,
    "port"             => 0,
    "user"             => 0,
    "pass"             => 0,
    "password"         => 0,
    "ssl-ca"           => 0,
    "skipsize"         => 0,
    "checkversion"     => 0,
    "updateversion"    => 0,
    "buffers"          => 0,
    "passwordfile"     => 0,
    "bannedports"      => '',
    "maxportallowed"   => 0,
    "outputfile"       => 0,
    "noprocess"        => 0,
    "dbstat"           => 0,
    "nodbstat"         => 0,
    "server-log"       => '',
    "tbstat"           => 0,
    "notbstat"         => 0,
    "colstat"          => 0,
    "nocolstat"        => 0,
    "idxstat"          => 0,
    "noidxstat"        => 0,
    "sysstat"          => 0,

```

```

        "nosysstat"      => 0,
        "pfstat"        => 0,
        "nopfstat"      => 0,
        "skippassword"  => 0,
        "noask"         => 0,
        "template"      => 0,
        "json"          => 0,
        "prettyjson"    => 0,
        "reportfile"    => 0,
        "verbose"       => 0,
        "defaults-file" => '',
    );

    # Gather the options from the command line
    GetOptions(
        \%opt,
        'nogood',          'noinfo',
        'debug',           'nocolor',
        'forcemem=i',      'forceswap=i',
        'host=s',          'socket=s',
        'port=i',          'user=s',
        'pass=s',          'skipsize',
        'checkversion',    'mysqladmin=s',
        'mysqlcmd=s',      'help',
        'buffers',         'skippassword',
        'passwordfile=s',  'outputfile=s',
        'silent',          'noask',
        'json',            'prettyjson',
        'template=s',      'reportfile=s',
        'cvefile=s',       'bannedports=s',
        'updateversion',   'maxportallowed=s',
        'verbose',         'password=s',
        'passenv=s',       'userenv=s',
        'defaults-file=s', 'ssl-ca=s',
        'color',           'noprocess',
        'dbstat',          'nodbstat',
        'tbstat',          'notbstat',
        'colstat',         'nocolstat',
        'sysstat',         'nosysstat',
        'pfstat',          'nopfstat',
        'idxstat',         'noidxstat',
        'server-log=s',
    )
    or pod2usage(
        -exitval => 1,
        -verbose  => 99,
        -sections => [
            "NAME",
            "IMPORTANT USAGE GUIDELINES",

```

```

        "CONNECTION AND AUTHENTICATION",
        "PERFORMANCE AND REPORTING OPTIONS",
        "OUTPUT OPTIONS"
    ]
);

if ( defined $opt{'help'} && $opt{'help'} == 1 ) {
    pod2usage(
        -exitval => 0,
        -verbose => 99,
        -sections => [
            "NAME",
            "IMPORTANT USAGE GUIDELINES",
            "CONNECTION AND AUTHENTICATION",
            "PERFORMANCE AND REPORTING OPTIONS",
            "OUTPUT OPTIONS"
        ]
    );
}

my $devnull = File::Spec->devnull();
my $basic_password_files =
    ( $opt{passwordfile} eq "0" )
    ? abs_path( dirname(__FILE__) ) . "/basic_passwords.txt"
    : abs_path( $opt{passwordfile} );

# Username from envvar
if ( exists $opt{userenv} && exists $ENV{ $opt{userenv} } ) {
    $opt{user} = $ENV{ $opt{userenv} };
}

# Related to password option
if ( exists $opt{passenv} && exists $ENV{ $opt{passenv} } ) {
    $opt{pass} = $ENV{ $opt{passenv} };
}
$opt{pass} = $opt{password} if ( $opt{pass} eq 0 and $opt{password} ne 0 );

# for RPM distributions
$basic_password_files = "/usr/share/mysqltuner/basic_passwords.txt"
unless -f "$basic_password_files";

# check if we need to enable verbose mode
if ( $opt{verbose} ) {
    $opt{checkversion} = 1;    #Check for updates to MySQLTuner
    $opt{dbstat}        = 1;    #Print database information
    $opt{tbstat}        = 1;    #Print database information
    $opt{idxstat}       = 1;    #Print index information
    $opt{sysstat}       = 1;    #Print index information
    $opt{buffers}       = 1;    #Print global and per-thread buffer values
}

```

```

    $opt{pfstat}      = 1;      #Print performance schema info.
    $opt{cvefile} = 'vulnerabilities.csv';      #CVE File for vulnerability checks
}
$opt{nocolor} = 1 if defined( $opt{outputfile} );
$opt{tbstat}   = 0 if ( $opt{notbstat} == 1 );      # Don't Print table
information
$opt{colstat}  = 0 if ( $opt{nocolstat} == 1 );      # Don't Print column
information
$opt{dbstat}   = 0 if ( $opt{nodbstat} == 1 );      # Don't Print database
information
$opt{noprocess} = 0
    if ( $opt{noprocess} == 1 );      # Don't Print process information
$opt{sysstat}  = 0 if ( $opt{nosysstat} == 1 );      # Don't Print sysstat
information
$opt{pfstat}   = 0
    if ( $opt{nopfstat} == 1 );      # Don't Print performance schema information
$opt{idxstat}  = 0 if ( $opt{noidxstat} == 1 );      # Don't Print index
information

# for RPM distributions
$opt{cvefile} = "/usr/share/mysqltuner/vulnerabilities.csv"
    unless ( defined $opt{cvefile} and -f "$opt{cvefile}" );
$opt{cvefile} = ' ' unless -f "$opt{cvefile}";
$opt{cvefile} = './vulnerabilities.csv' if -f './vulnerabilities.csv';

$opt{'bannedports'} = ' ' unless defined( $opt{'bannedports'} );
my @banned_ports = split ' ', $opt{'bannedports'};

#
my $outputfile = undef;
$outputfile = abs_path( $opt{outputfile} ) unless $opt{outputfile} eq "0";

my $fh = undef;
open( $fh, '>', $outputfile )
    or die("Fail opening $outputfile")
    if defined($outputfile);
$opt{nocolor} = 1 if defined($outputfile);
$opt{nocolor} = 1 unless ( -t STDOUT );

$opt{nocolor} = 0 if ( $opt{color} == 1 );

# Setting up the colors for the print styles
my $me = `whoami`;
$me =~ s/\n//g;

# Setting up the colors for the print styles
my $good = ( $opt{nocolor} == 0 ) ? "[\e[0;32mOK\e[0m]" : "[OK]";
my $bad  = ( $opt{nocolor} == 0 ) ? "[\e[0;31m!!\e[0m]" : "[!!]";
my $info = ( $opt{nocolor} == 0 ) ? "[\e[0;34m--\e[0m]" : "[--]";

```

```

my $deb = ( $opt{nocolor} == 0 ) ? "\e[0;31mDG\e[0m" : "[DG]";
my $cmd = ( $opt{nocolor} == 0 ) ? "\e[1;32m[CMD] ($me)" : "[CMD] ($me)";
my $end = ( $opt{nocolor} == 0 ) ? "\e[0m" : "";

# Maximum lines of log output to read from end
my $maxlines = 30000;

# Checks for supported or EOL'ed MySQL versions
my ( $mysqlvermajor, $mysqlverminor, $mysqlvermicro );

# Database
my @dblist;

# Super structure containing all information
my %result;
$result{'MySQLTuner'}{'version'} = $tunerverversion;
$result{'MySQLTuner'}{'options'} = \%opt;

# Functions that handle the print styles
sub prettyprint {
    print $_[0] . "\n" unless ( $opt{'silent'} or $opt{'json'} );
    print $fh $_[0] . "\n" if defined($fh);
}
sub goodprint { prettyprint $good. " " . $_[0] unless ( $opt{nogood} == 1 ); }
sub infoprint { prettyprint $info. " " . $_[0] unless ( $opt{noinfo} == 1 ); }
sub badprint { prettyprint $bad. " " . $_[0] unless ( $opt{nobad} == 1 ); }
sub debugprint { prettyprint $deb. " " . $_[0] unless ( $opt{debug} == 0 ); }

sub redwrap {
    return ( $opt{nocolor} == 0 ) ? "\e[0;31m" . $_[0] . "\e[0m" : $_[0];
}

sub greenwrap {
    return ( $opt{nocolor} == 0 ) ? "\e[0;32m" . $_[0] . "\e[0m" : $_[0];
}

sub cmdprint { prettyprint $cmd. " " . $_[0] . $end; }

sub infoprinttml {
    for my $ln (@_) { $ln =~ s/\n//g; infoprint "\t$ln"; }
}

sub infoprintcmd {
    cmdprint "@_";
    infoprinttml grep { $_ ne "'" and $_ !~ /\s*/ } `@_ 2>&1`;
}

sub subheaderprint {
    my $tln = 100;
    my $sln = 8;

```

```

my $ln = length("@_") + 2;

prettyprint " ";
prettyprint "-" x $sln . " @_" . "-" x ( $tln - $ln - $sln );
}

sub infoprinthcmd {
    subheaderprint "$_[0]";
    infoprintcmd "$_[1]";
}

# Calculates the number of physical cores considering HyperThreading
sub cpu_cores {
    my $cntCPU =
`awk -F: '/^core id/ && !P[\\$2] { CORES++; P[\\$2]=1 }; /^physical id/ &&
!N[\\$2] { CPUs++; N[\\$2]=1 }; END { print CPUs*CORES }' /proc/cpuinfo`;
    return ( $cntCPU == 0 ? `nproc` : $cntCPU );
}

# Calculates the parameter passed in bytes, then rounds it to one decimal
place
sub hr_bytes {
    my $num = shift;
    return "0B" unless defined($num);
    return "0B" if $num eq "NULL";

    if ( $num >= ( 1024**3 ) ) {          #GB
        return sprintf( "%.1f", ( $num / ( 1024**3 ) ) ) . "G";
    }
    elsif ( $num >= ( 1024**2 ) ) {       #MB
        return sprintf( "%.1f", ( $num / ( 1024**2 ) ) ) . "M";
    }
    elsif ( $num >= 1024 ) {               #KB
        return sprintf( "%.1f", ( $num / 1024 ) ) . "K";
    }
    else {
        return $num . "B";
    }
}

sub hr_raw {
    my $num = shift;
    return "0" unless defined($num);
    return "0" if $num eq "NULL";
    if ( $num =~ /\^(\\d+)G$/ ) {
        return $1 * 1024 * 1024 * 1024;
    }
    if ( $num =~ /\^(\\d+)M$/ ) {
        return $1 * 1024 * 1024;
    }
}

```

```

    }
    if ( $num =~ /\^(\\d+)K$/ ) {
        return $1 * 1024;
    }
    if ( $num =~ /\^(\\d+)$/ ) {
        return $1;
    }
    return $num;
}

```

Calculates the parameter passed in bytes, then rounds it to the nearest integer

```

sub hr_bytes_rnd {
    my $num = shift;
    return "0B" unless defined($num);
    return "0B" if $num eq "NULL";

    if ( $num >= ( 1024**3 ) ) {      #GB
        return int( ( $num / ( 1024**3 ) ) ) . "G";
    }
    elsif ( $num >= ( 1024**2 ) ) {    #MB
        return int( ( $num / ( 1024**2 ) ) ) . "M";
    }
    elsif ( $num >= 1024 ) {           #KB
        return int( ( $num / 1024 ) ) . "K";
    }
    else {
        return $num . "B";
    }
}

```

Calculates the parameter passed to the nearest power of 1000, then rounds it to the nearest integer

```

sub hr_num {
    my $num = shift;
    if ( $num >= ( 1000**3 ) ) {      # Billions
        return int( ( $num / ( 1000**3 ) ) ) . "B";
    }
    elsif ( $num >= ( 1000**2 ) ) {    # Millions
        return int( ( $num / ( 1000**2 ) ) ) . "M";
    }
    elsif ( $num >= 1000 ) {           # Thousands
        return int( ( $num / 1000 ) ) . "K";
    }
    else {
        return $num;
    }
}

```

```

# Calculate Percentage
sub percentage {
    my $value = shift;
    my $total = shift;
    $total = 0 unless defined $total;
    $total = 0 if $total eq "NULL";
    return 100, 00 if $total == 0;
    return sprintf( "%.2f", ( $value * 100 / $total ) );
}

# Calculates uptime to display in a more attractive form
sub pretty_uptime {
    my $uptime = shift;
    my $seconds = $uptime % 60;
    my $minutes = int( ( $uptime % 3600 ) / 60 );
    my $hours = int( ( $uptime % 86400 ) / (3600) );
    my $days = int( $uptime / (86400) );
    my $uptimestring;
    if ( $days > 0 ) {
        $uptimestring = "${days}d ${hours}h ${minutes}m ${seconds}s";
    }
    elsif ( $hours > 0 ) {
        $uptimestring = "${hours}h ${minutes}m ${seconds}s";
    }
    elsif ( $minutes > 0 ) {
        $uptimestring = "${minutes}m ${seconds}s";
    }
    else {
        $uptimestring = "${seconds}s";
    }
    return $uptimestring;
}

# Retrieves the memory installed on this machine
my ( $physical_memory, $swap_memory, $duflags, $xargsflags );

sub memerror {
    badprint
    "Unable to determine total memory/swap; use '--forcemem' and '--forceswap";
    exit 1;
}

sub os_setup {
    my $os = `uname`;
    $duflags = ( $os =~ /Linux/ ) ? '-b' : '';
    $xargsflags = ( $os =~ /Darwin|SunOS/ ) ? '' : '-r';
    if ( $opt{'forcemem'} > 0 ) {
        $physical_memory = $opt{'forcemem'} * 1048576;
        infoprint "Assuming $opt{'forcemem'} MB of physical memory";
    }
}

```

```

if ( $opt{'forceswap'} > 0 ) {
    $swap_memory = $opt{'forceswap'} * 1048576;
    infoprint "Assuming $opt{'forceswap'} MB of swap space";
}
else {
    $swap_memory = 0;
    badprint "Assuming 0 MB of swap space (use --forceswap to specify)";
}
}
else {
    if ( $os =~ /Linux|CYGWIN/ ) {
        $physical_memory =
            `grep -i memtotal: /proc/meminfo | awk '{print \$2}'`
            or memerror;
        $physical_memory *= 1024;

        $swap_memory =
            `grep -i swaptotal: /proc/meminfo | awk '{print \$2}'`
            or memerror;
        $swap_memory *= 1024;
    }
    elsif ( $os =~ /Darwin/ ) {
        $physical_memory = `sysctl -n hw.memsize` or memerror;
        $swap_memory =
            `sysctl -n vm.swapusage | awk '{print \$3}' | sed 's/\.[.]*\$//`
            or memerror;
    }
    elsif ( $os =~ /NetBSD|OpenBSD|FreeBSD/ ) {
        $physical_memory = `sysctl -n hw.physmem` or memerror;
        if ( $physical_memory < 0 ) {
            $physical_memory = `sysctl -n hw.physmem64` or memerror;
        }
        $swap_memory =
            `swapctl -l | grep '^/' | awk '{ s+= \$2 } END { print s }'`
            or memerror;
    }
    elsif ( $os =~ /BSD/ ) {
        $physical_memory = `sysctl -n hw.realmem` or memerror;
        $swap_memory =
            `swapinfo | grep '^/' | awk '{ s+= \$2 } END { print s }'`;
    }
    elsif ( $os =~ /SunOS/ ) {
        $physical_memory =
            `/usr/sbin/prtconf | grep Memory | cut -f 3 -d ' '`
            or memerror;
        chomp($physical_memory);
        $physical_memory = $physical_memory * 1024 * 1024;
    }
    elsif ( $os =~ /AIX/ ) {

```

```

        $physical_memory =
            `lsattr -El sys0 | grep realmem | awk '{print \$2}'`
            or memerror;
        chomp($physical_memory);
        $physical_memory = $physical_memory * 1024;
        $swap_memory = `lsps -as | awk -F"(MB| +)" '{print \$2}'`
            or memerror;
        chomp($swap_memory);
        $swap_memory = $swap_memory * 1024 * 1024;
    }
    elsif ( $os =~ /windows/i ) {
        $physical_memory =
            `wmic ComputerSystem get TotalPhysicalMemory | perl -ne "chomp; print if
/[0-9]+/;"`
            or memerror;
        $swap_memory =
            `wmic OS get FreeVirtualMemory | perl -ne "chomp; print if /[0-9]+/;"`
            or memerror;
    }
}

debugprint "Physical Memory: $physical_memory";
debugprint "Swap Memory: $swap_memory";
chomp($physical_memory);
chomp($swap_memory);
chomp($os);
$result{'OS'}{'OS Type'} = $os;
$result{'OS'}{'Physical Memory'}{'bytes'} = $physical_memory;
$result{'OS'}{'Physical Memory'}{'pretty'} = hr_bytes($physical_memory);
$result{'OS'}{'Swap Memory'}{'bytes'} = $swap_memory;
$result{'OS'}{'Swap Memory'}{'pretty'} = hr_bytes($swap_memory);
$result{'OS'}{'Other Processes'}{'bytes'} = get_other_process_memory();
$result{'OS'}{'Other Processes'}{'pretty'} =
    hr_bytes( get_other_process_memory() );
}

sub get_http_cli {
    my $httpcli = which( "curl", $ENV{'PATH'} );
    chomp($httpcli);
    if ($httpcli) {
        return $httpcli;
    }

    $httpcli = which( "wget", $ENV{'PATH'} );
    chomp($httpcli);
    if ($httpcli) {
        return $httpcli;
    }
    return "";
}

```

```

# Checks for updates to MySQLTuner
sub validate_tuner_version {
    if ( $opt{'checkversion'} eq 0 ) {
        print "\n" unless ( $opt{'silent'} or $opt{'json'} );
        infoprint "Skipped version check for MySQLTuner script";
        return;
    }

    my $update;
    my $url =
"https://raw.githubusercontent.com/major/MySQLTuner-perl/master/mysqltuner.pl";
    my $httpcli = get_http_cli();
    if ( $httpcli =~ /curl$/ ) {
        debugprint "$httpcli is available.";

        debugprint
"$httpcli -m 3 -silent '$url' 2>/dev/null | grep 'my \${tunerver}' | cut
-d\\\" -f2";
        $update =
`$httpcli -m 3 -silent '$url' 2>/dev/null | grep 'my \${tunerver}' | cut
-d\\\" -f2`;
        chomp($update);
        debugprint "VERSION: $update";

        compare_tuner_version($update);
        return;
    }

    if ( $httpcli =~ /wget$/ ) {
        debugprint "$httpcli is available.";

        debugprint
"$httpcli -e timestamping=off -t 1 -T 3 -O - '$url' 2>$devnull | grep 'my
\${tunerver}' | cut -d\\\" -f2";
        $update =
`$httpcli -e timestamping=off -t 1 -T 3 -O - '$url' 2>$devnull | grep 'my
\${tunerver}' | cut -d\\\" -f2`;
        chomp($update);
        compare_tuner_version($update);
        return;
    }
    debugprint "curl and wget are not available.";
    infoprint "Unable to check for the latest MySQLTuner version";
    infoprint
"Using --pass and --password option is insecure during MySQLTuner
execution>Password disclosure"
    if ( defined( $opt{'pass'} ) );
}

```

```

# Checks for updates to MySQLTuner
sub update_tuner_version {
    if ( $opt{'updateversion'} eq 0 ) {
        badprint "Skipped version update for MySQLTuner script";
        print "\n" unless ( $opt{'silent'} or $opt{'json'} );
        return;
    }

    my $update;
    my $url = "https://raw.githubusercontent.com/major/MySQLTuner-perl/master/";
    my @scripts =
        ( "mysqltuner.pl", "basic_passwords.txt", "vulnerabilities.csv" );
    my $totalScripts = scalar(@scripts);
    my $receivedScripts = 0;
    my $httpcli = get_http_cli();

    foreach my $script (@scripts) {

        if ( $httpcli =~ /curl$/ ) {
            debugprint "$httpcli is available.";

            debugprint
                "$httpcli --connect-timeout 3 '$url$script' 2>$devnull > $script";
            $update =
                ` $httpcli --connect-timeout 3 '$url$script' 2>$devnull > $script `;
            chomp($update);
            debugprint "$script updated: $update";

            if ( -s $script eq 0 ) {
                badprint "Couldn't update $script";
            }
            else {
                ++$receivedScripts;
                debugprint "$script updated: $update";
            }
        }
        elsif ( $httpcli =~ /wget$/ ) {

            debugprint "$httpcli is available.";

            debugprint
                "$httpcli -qe timestamping=off -t 1 -T 3 -O $script '$url$script'";
            $update =
                ` $httpcli -qe timestamping=off -t 1 -T 3 -O $script '$url$script' `;
            chomp($update);

            if ( -s $script eq 0 ) {
                badprint "Couldn't update $script";
            }
        }
    }
}

```

```

    }
    else {
        ++$receivedScripts;
        debugprint "$script updated: $update";
    }
}
else {
    debugprint "curl and wget are not available.";
    infoprint "Unable to check for the latest MySQLTuner version";
}

}

if ( $receivedScripts eq $totalScripts ) {
    goodprint "Successfully updated MySQLTuner script";
}
else {
    badprint "Couldn't update MySQLTuner script";
}
infoprint "Stopping program: MySQLTuner has be updated.";
exit 0;
}

sub compare_tuner_version {
    my $remoteversion = shift;
    debugprint "Remote data: $remoteversion";

    #exit 0;
    if ( $remoteversion ne $tunerverversion ) {
        badprint
            "There is a new version of MySQLTuner available($remoteversion)";
        update_tuner_version();
        return;
    }
    goodprint "You have the latest version of MySQLTuner($tunerverversion)";
    return;
}

# Checks to see if a MySQL login is possible
my ( $mysqllogin, $doremove, $remotestring, $mysqlcmd, $mysqladmincmd );

my $osname = $^O;
if ( $osname eq 'MSWin32' ) {
    eval { require Win32; } or last;
    $osname = Win32::GetOSName();
    infoprint "* Windows OS($osname) is not fully supported.\n";

    #exit 1;
}

```

```

sub mysql_setup {
    $doremove      = 0;
    $remotestring = '';
    if ( $opt{mysqladmin} ) {
        $mysqladmincmd = $opt{mysqladmin};
    }
    else {
        $mysqladmincmd = which( "mysqladmin", $ENV{'PATH'} );
        if ( !-e $mysqladmincmd ) {
            $mysqladmincmd = which( "mariadb-admin", $ENV{'PATH'} );
        }
    }
    chomp($mysqladmincmd);
    if ( !-e $mysqladmincmd && $opt{mysqladmin} ) {
        badprint "Unable to find the mysqladmin command you specified: "
            . $mysqladmincmd . " ";
        exit 1;
    }
    elsif ( !-e $mysqladmincmd ) {
        badprint
            "Couldn't find mysqladmin/mariadb-admin in your \${PATH}. Is MySQL installed?";
        exit 1;
    }
    if ( $opt{mysqlcmd} ) {
        $mysqlcmd = $opt{mysqlcmd};
    }
    else {
        $mysqlcmd = which( "mysql", $ENV{'PATH'} );
        if ( !-e $mysqlcmd ) {
            $mysqlcmd = which( "mariadb", $ENV{'PATH'} );
        }
    }
    chomp($mysqlcmd);
    if ( !-e $mysqlcmd && $opt{mysqlcmd} ) {
        badprint "Unable to find the mysql command you specified: "
            . $mysqlcmd . " ";
        exit 1;
    }
    elsif ( !-e $mysqlcmd ) {
        badprint
            "Couldn't find mysql/mariadb in your \${PATH}. Is MySQL installed?";
        exit 1;
    }
    $mysqlcmd =~ s/\n$//g;
    my $mysqlclidefaults = ` $mysqlcmd --print-defaults `;
    debugprint "MySQL Client: $mysqlclidefaults";
    if ( $mysqlclidefaults =~ /auto-vertical-output/ ) {
        badprint

```

```

        "Avoid auto-vertical-output in configuration file(s) for MySQL like";
    exit 1;
}

debugprint "MySQL Client: $mysqlcmd";

$opt{port} = ( $opt{port} eq 0 ) ? 3306 : $opt{port};

# Are we being asked to connect via a socket?
if ( $opt{socket} ne 0 ) {
    $remotestring = " -S $opt{socket} -P $opt{port}";
}

# Are we being asked to connect to a remote server?
if ( $opt{host} ne 0 ) {
    chomp( $opt{host} );

# If we're doing a remote connection, but forcemem wasn't specified, we need
to exit
    if ( $opt{'forcemem'} eq 0
        && ( $opt{host} ne "127.0.0.1" )
        && ( $opt{host} ne "localhost" ) )
    {
        badprint "The --forcemem option is required for remote connections";
        exit 1;
    }
    infoprint "Performing tests on $opt{host}:$opt{port}";
    $remotestring = " -h $opt{host} -P $opt{port}";
    if ( ( $opt{host} ne "127.0.0.1" ) && ( $opt{host} ne "localhost" ) ) {
        $doremote = 1;
    }
}
else {
    $opt{host} = '127.0.0.1';
}

if ( $opt{'ssl-ca'} ne 0 ) {
    if ( -e -r -f $opt{'ssl-ca'} ) {
        $remotestring .= " --ssl-ca=$opt{'ssl-ca'}";
        infoprint
            "Will connect using ssl public key passed on the command line";
        return 1;
    }
    else {
        badprint
            "Attempted to use passed ssl public key, but it was not found or could not be
            read";
        exit 1;
    }
}

```

```

}

# Did we already get a username without password on the command line?
if ( $opt{user} ne 0 and $opt{pass} eq 0 ) {
    $mysqllogin = "-u $opt{user} " . $remotestring;
    my $loginstatus = `mysqladmincmd ping $mysqllogin 2>&1`;
    if ( $loginstatus =~ /mysqld is alive/ ) {
        goodprint "Logged in using credentials passed on the command line";
        return 1;
    }
    else {
        badprint
            "Attempted to use login credentials, but they were invalid";
        exit 1;
    }
}

# Did we already get a username and password passed on the command line?
if ( $opt{user} ne 0 and $opt{pass} ne 0 ) {
    $mysqllogin = "-u $opt{user} -p'$opt{pass}'" . $remotestring;
    my $loginstatus = `mysqladmincmd ping $mysqllogin 2>&1`;
    if ( $loginstatus =~ /mysqld is alive/ ) {
        goodprint "Logged in using credentials passed on the command line";
        return 1;
    }
    else {
        badprint
            "Attempted to use login credentials, but they were invalid";
        exit 1;
    }
}

my $svcprop = which( "svcprop", $ENV{'PATH'} );
if ( substr( $svcprop, 0, 1 ) =~ "/" ) {

    # We are on solaris
    ( my $mysql_login =
`svcprop -p quickbackup/username svc:/network/mysql-quickbackup:default`
    ) =~ s/\s+$/;
    ( my $mysql_pass =
`svcprop -p quickbackup/password svc:/network/mysql-quickbackup:default`
    ) =~ s/\s+$/;
    if ( substr( $mysql_login, 0, 7 ) ne "svcprop" ) {

        # mysql-quickbackup is installed
        $mysqllogin = "-u $mysql_login -p$mysql_pass";
        my $loginstatus = `mysqladmin $mysqllogin ping 2>&1`;
        if ( $loginstatus =~ /mysqld is alive/ ) {
            goodprint "Logged in using credentials from mysql-quickbackup.";
            return 1;
        }
    }
}

```

```

    }
    else {
        badprint
        "Attempted to use login credentials from mysql-quickbackup, but they failed.";
        exit 1;
    }
}

}

elseif ( -r "/etc/psa/.psa.shadow" and $doremove == 0 ) {

    # It's a Plesk box, use the available credentials
    $mysqllogin = "-u admin -p`cat /etc/psa/.psa.shadow`;";
    my $loginstatus = `mysqladmincmd ping $mysqllogin 2>&1`;
    unless ( $loginstatus =~ /mysqld is alive/ ) {

        # Plesk 10+
        $mysqllogin =
            "-u admin -p`/usr/local/psa/bin/admin --show-password`;";
        $loginstatus = `mysqladmincmd ping $mysqllogin 2>&1`;
        unless ( $loginstatus =~ /mysqld is alive/ ) {
            badprint
            "Attempted to use login credentials from Plesk and Plesk 10+, but they
            failed.";
            exit 1;
        }
    }
}

elseif ( -r "/usr/local/directadmin/conf/mysql.conf" and $doremove == 0 ) {

    # It's a DirectAdmin box, use the available credentials
    my $mysqluser =
        `cat /usr/local/directadmin/conf/mysql.conf | egrep '^user=.*'`;
    my $mysqlpass =
        `cat /usr/local/directadmin/conf/mysql.conf | egrep '^passwd=.*'`;

    $mysqluser =~ s/user=//;
    $mysqluser =~ s/[\r\n]//;
    $mysqlpass =~ s/passwd=//;
    $mysqlpass =~ s/[\r\n]//;

    $mysqllogin = "-u $mysqluser -p$mysqlpass";

    my $loginstatus = `mysqladmin ping $mysqllogin 2>&1`;
    unless ( $loginstatus =~ /mysqld is alive/ ) {
        badprint
        "Attempted to use login credentials from DirectAdmin, but they failed.";
        exit 1;
    }
}

}

```

```

elseif ( -r "/etc/mysql/debian.cnf"
    and $doreremote == 0
    and $opt{'defaults-file'} eq '' )
{
    # We have a Debian maintenance account, use it
    $mysqllogin = "--defaults-file=/etc/mysql/debian.cnf";
    my $loginstatus = `$mysqladmincmd $mysqllogin ping 2>&1`;
    if ( $loginstatus =~ /mysqld is alive/ ) {
        goodprint
            "Logged in using credentials from Debian maintenance account.";
        return 1;
    }
    else {
        badprint
            "Attempted to use login credentials from Debian maintenance account, but they
            failed.";
        exit 1;
    }
}

elseif ( $opt{'defaults-file'} ne '' and -r "$opt{'defaults-file'}" ) {

    # defaults-file
    debugprint "defaults file detected: $opt{'defaults-file'}";
    my $mysqlclidefaults = `$mysqlcmd --print-defaults`;
    debugprint "MySQL Client Default File: $opt{'defaults-file'}";

    $mysqllogin = "--defaults-file=" . $opt{'defaults-file'};
    my $loginstatus = `$mysqladmincmd $mysqllogin ping 2>&1`;
    if ( $loginstatus =~ /mysqld is alive/ ) {
        goodprint "Logged in using credentials from defaults file account.";
        return 1;
    }
}

else {
    # It's not Plesk or Debian, we should try a login
    debugprint "$mysqladmincmd $remotestring ping 2>&1";
    my $loginstatus = `$mysqladmincmd $remotestring ping 2>&1`;
    if ( $loginstatus =~ /mysqld is alive/ ) {

        # Login went just fine
        $mysqllogin = " $remotestring ";

        # Did this go well because of a .my.cnf file or is there no password
        set?

        my $userpath = `printenv HOME`;
        if ( length($userpath) > 0 ) {
            chomp($userpath);
        }
    }
}

```

```

unless ( -e "${userpath}/.my.cnf" or -e "${userpath}/.mylogin.cnf" )
{
    badprint
    "Successfully authenticated with no password - SECURITY RISK!";
}
return 1;
}
else {
    if ( $opt{'noask'} == 1 ) {
        badprint
        "Attempted to use login credentials, but they were invalid";
        exit 1;
    }
    my ( $name, $password );

    # If --user is defined no need to ask for username
    if ( $opt{user} ne 0 ) {
        $name = $opt{user};
    }
    else {
        print STDERR "Please enter your MySQL administrative login: ";
        $name = <STDIN>;
    }

    # If --pass is defined no need to ask for password
    if ( $opt{pass} ne 0 ) {
        $password = $opt{pass};
    }
    else {
        print STDERR
        "Please enter your MySQL administrative password: ";
        system("stty -echo >$devnull 2>&1");
        $password = <STDIN>;
        system("stty echo >$devnull 2>&1");
    }
    chomp($password);
    chomp($name);
    $mysqllogin = "-u $name";

    if ( length($password) > 0 ) {
        $mysqllogin .= " -p'$password'";
    }
    $mysqllogin .= $remotestring;
    my $loginstatus = `mysqladmincmd ping $mysqllogin 2>&1`;
    if ( $loginstatus =~ /mysqld is alive/ ) {
        print STDERR "";
        if ( !length($password) ) {

```

```

# Did this go well because of a .my.cnf file or is there no password
set?

    my $userpath = `printenv HOME`;
    chomp($userpath);
    unless ( -e "$userpath/.my.cnf" ) {
        badprint
"Successfully authenticated with no password - SECURITY RISK!";
    }
}
return 1;
}
else {
    badprint
        "Attempted to use login credentials, but they were invalid.";
    exit 1;
}
exit 1;
}
}

# MySQL Request Array
sub select_array {
    my $req = shift;
    debugprint "PERFORM: $req ";
    my @result = `mysqlcmd mysqllogin -Bse "\\w$req" 2>>/dev/null`;
    if ( $? != 0 ) {
        badprint "failed to execute: $req";
        badprint "FAIL Execute SQL / return code: $?";
        debugprint "CMD      : mysqlcmd";
        debugprint "OPTIONS: mysqllogin";
        debugprint `mysqlcmd mysqllogin -Bse "$req" 2>&1`;

        #exit $?;
    }
    debugprint "select_array: return code : $?";
    chomp(@result);
    return @result;
}

sub human_size {
    my ( $size, $n ) = ( shift, 0 );
    ++$n and $size /= 1024 until $size < 1024;
    return sprintf "%.2f %s", $size, (qw[ bytes KB MB GB ])[ $n ];
}

# MySQL Request one
sub select_one {

```

```

my $req = shift;
debugprint "PERFORM: $req ";
my $result = `$mysqlcmd $mysqllogin -Bse "\\w$req" 2>>/dev/null`;
if ( $? != 0 ) {
    badprint "failed to execute: $req";
    badprint "FAIL Execute SQL / return code: $?";
    debugprint "CMD      : $mysqlcmd";
    debugprint "OPTIONS: $mysqllogin";
    debugprint `$mysqlcmd $mysqllogin -Bse "$req" 2>&1`;

    #exit $?;
}
debugprint "select_array: return code : $?";
chomp($result);
return $result;
}

# MySQL Request one
sub select_one_g {
    my $pattern = shift;

    my $req = shift;
    debugprint "PERFORM: $req ";
    my @result = `$mysqlcmd $mysqllogin -re "\\w$req\\G" 2>>/dev/null`;
    if ( $? != 0 ) {
        badprint "failed to execute: $req";
        badprint "FAIL Execute SQL / return code: $?";
        debugprint "CMD      : $mysqlcmd";
        debugprint "OPTIONS: $mysqllogin";
        debugprint `$mysqlcmd $mysqllogin -Bse "$req" 2>&1`;

        #exit $?;
    }
    debugprint "select_array: return code : $?";
    chomp(@result);
    return ( grep { /$pattern/ } @result )[0];
}

sub select_str_g {
    my $pattern = shift;

    my $req = shift;
    my $str = select_one_g $pattern, $req;
    return () unless defined $str;
    my @val = split /\:/, $str;
    shift @val;
    return trim(@val);
}

```

```

sub get_tuning_info {
    my @infoconn = select_array "\\s";
    my ( $tkey, $tval );
    @infoconn =
        grep { !/Threads:/ and !/Connection id:/ and !/pager:/ and !/Using/ }
        @infoconn;
    foreach my $line (@infoconn) {
        if ( $line =~ /\s*(.):\s*(.)/ ) {
            debugprint "$1 => $2";
            $tkey = $1;
            $tval = $2;
            chomp($tkey);
            chomp($tval);
            $result{'MySQL Client'}{$tkey} = $tval;
        }
    }
    $result{'MySQL Client'}{'Client Path'} = $mysqlcmd;
    $result{'MySQL Client'}{'Admin Path'} = $mysqladmincmd;
    $result{'MySQL Client'}{'Authentication Info'} = $mysqllogin;
}

# Populates all of the variable and status hashes
my ( %mystat, %myvar, $dummyselect, %myrepl, %myslaves );

sub arr2hash {
    my $href = shift;
    my $harr = shift;
    my $sep = shift;
    $sep = '\s' unless defined($sep);
    foreach my $line (@$harr) {
        next if ( $line =~ m/^\*\*\*\*\*\*\*/ );
        $line =~ /([a-zA-Z_]*)\s*$sep\s*(.)/;
        $$href{$1} = $2;
        debugprint "V: $1 = $2";
    }
}

sub get_all_vars {
    # We need to initiate at least one query so that our data is useable
    $dummyselect = select_one "SELECT VERSION()";
    if ( not defined($dummyselect) or $dummyselect eq "" ) {
        badprint
        "You probably did not get enough privileges for running MySQLTuner ...";
        exit(256);
    }
    $dummyselect =~ s/(.*)\s*$/1/;
    debugprint "VERSION: " . $dummyselect . " ";
}

```

```

$result{'MySQL Client'}{'Version'} = $dummyselect;

my @mysqlvarlist = select_array("SHOW VARIABLES");
push( @mysqlvarlist, select_array("SHOW GLOBAL VARIABLES") );
arr2hash( \%myvar, \@mysqlvarlist );
$result{'Variables'} = \%myvar;

my @mysqlstatlist = select_array("SHOW STATUS");
push( @mysqlstatlist, select_array("SHOW GLOBAL STATUS") );
arr2hash( \%mystat, \@mysqlstatlist );
$result{'Status'} = \%mystat;
unless ( defined( $myvar{'innodb_support_xa'} ) ) {
    $myvar{'innodb_support_xa'} = 'ON';
}
$mystat{'Uptime'} = 1
    unless defined( $mystat{'Uptime'} )
    and $mystat{'Uptime'} > 0;
$myvar{'have_galera'} = "NO";
if ( defined( $myvar{'wsrep_provider_options'} )
    && $myvar{'wsrep_provider_options'} ne ""
    && $myvar{'wsrep_on'} ne "OFF" )
{
    $myvar{'have_galera'} = "YES";
    debugprint "Galera options: " . $myvar{'wsrep_provider_options'};
}

# Workaround for MySQL bug #59393 wrt. ignore-builtin-innodb
if ( ( $myvar{'ignore_builtin_innodb'} || "" ) eq "ON" ) {
    $myvar{'have_innodb'} = "NO";
}

# Support GTID MODE FOR MARIADB
# Issue MariaDB GTID mode #513
$myvar{'gtid_mode'} = 'ON'
    if ( defined( $myvar{'gtid_current_pos'} )
        and $myvar{'gtid_current_pos'} ne '' );

$myvar{'have_threadpool'} = "NO";
if ( defined( $myvar{'thread_pool_size'} )
    and $myvar{'thread_pool_size'} > 0 )
{
    $myvar{'have_threadpool'} = "YES";
}

# have_* for engines is deprecated and will be removed in MySQL 5.6;
# check SHOW ENGINES and set corresponding old style variables.
# Also works around MySQL bug #59393 wrt. skip-innodb
my @mysqlenginelist = select_array "SHOW ENGINES";
foreach my $line (@mysqlenginelist) {

```

```

    if ( $line =~ /^( [a-zA-Z_]+ )\s+( \S+ ) / ) {
        my $engine = lc($1);

        if ( $engine eq "federated" || $engine eq "blackhole" ) {
            $engine .= "_engine";
        }
        elsif ( $engine eq "berkeleydb" ) {
            $engine = "bdb";
        }
        my $val = ( $2 eq "DEFAULT" ) ? "YES" : $2;
        $myvar{"have_$engine"} = $val;
        $result{'Storage Engines'}{$engine} = $2;
    }
}

debugprint Dumper(@mysqlenginelist);
my @mysqlslave = select_array("SHOW SLAVE STATUS\\G");
arr2hash( \%myrepl, \@mysqlslave, ':' );
$result{'Replication'}{'Status'} = \%myrepl;
my @mysqlslaves = select_array "SHOW SLAVE HOSTS";
my @lineitems = ();
foreach my $line (@mysqlslaves) {
    debugprint "L: $line ";
    @lineitems = split /\s+/, $line;
    $mysqlslaves{ $lineitems[0] } = $line;
    $result{'Replication'}{'Slaves'}{ $lineitems[0] } = $lineitems[4];
}
}

sub remove_cr {
    return map {
        my $line = $_;
        $line =~ s/\n$//g;
        $line =~ s/^\s+$//g;
        $line;
    } @_;
}

sub remove_empty {
    grep { $_ ne '' } @_;
}

sub grep_file_contents {
    my $file = shift;
    my $patt;
}

sub get_file_contents {
    my $file = shift;
    open( my $fh, "<", $file ) or die "Can't open $file for read: $!";
}

```

```

my @lines = <$fh>;
close $fh or die "Cannot close $file: $!";
@lines = remove_cr @lines;
return @lines;
}

sub get_basic_passwords {
    return get_file_contents(shift);
}

sub get_log_file_real_path {
    my $file      = shift;
    my $hostname = shift;
    my $datadir   = shift;
    if ( -f "$file" ) {
        return $file;
    }
    elsif ( -f "$hostname.log" ) {
        return "$hostname.log";
    }
    elsif ( -f "$hostname.err" ) {
        return "$hostname.err";
    }
    elsif ( -f "$datadir$hostname.err" ) {
        return "$datadir$hostname.err";
    }
    elsif ( -f "$datadir$hostname.log" ) {
        return "$datadir$hostname.log";
    }
    elsif ( -f "$datadir" . "mysql_error.log" ) {
        return "$datadir" . "mysql_error.log";
    }
    elsif ( -f "/var/log/mysql.log" ) {
        return "/var/log/mysql.log";
    }
    elsif ( -f "/var/log/mysqld.log" ) {
        return "/var/log/mysqld.log";
    }
    elsif ( -f "/var/log/mysql/$hostname.err" ) {
        return "/var/log/mysql/$hostname.err";
    }
    elsif ( -f "/var/log/mysql/$hostname.log" ) {
        return "/var/log/mysql/$hostname.log";
    }
    elsif ( -f "/var/log/mysql/" . "mysql_error.log" ) {
        return "/var/log/mysql/" . "mysql_error.log";
    }
    else {
        return $file;
    }
}

```

```

    }
}

sub log_file_recommendations {
    my $fh;
    $myvar{'log_error'} = $opt{'server-log'}
    || get_log_file_real_path( $myvar{'log_error'}, $myvar{'hostname'},
        $myvar{'datadir'} );

    subheaderprint "Log file Recommendations";
    if ( "$myvar{'log_error'}" eq "stderr" ) {
        badprint "log_error is set to $myvar{'log_error'} MT can't read stderr";
        return;
    }
    elsif ( $myvar{'log_error'} =~ /^(docker|podman|kubect1):(.*)/ ) {
        open( $fh, '-|', "$1 logs --tail=$maxlines '$2'" )
            // die "Can't start $1 $!";
        goodprint "Log from cloud ` $myvar{'log_error'} exists";
    }
    elsif ( $myvar{'log_error'} =~ /^systemd:(.*)/ ) {
        open( $fh, '-|', "journalctl -n $maxlines -b -u '$1'" )
            // die "Can't start journalctl $!";
        goodprint "Log journal ` $myvar{'log_error'} exists";
    }
    elsif ( -f "$myvar{'log_error'}" ) {
        goodprint "Log file $myvar{'log_error'} exists";
        my $size = ( stat $myvar{'log_error'} )[7];
        infoprint "Log file: "
            . $myvar{'log_error'} . "("
            . hr_bytes_rnd($size) . ")";

        if ( $size > 0 ) {
            goodprint "Log file $myvar{'log_error'} is not empty";
            if ( $size < 32 * 1024 * 1024 ) {
                goodprint "Log file $myvar{'log_error'} is smaller than 32 Mb";
            }
            else {
                badprint "Log file $myvar{'log_error'} is bigger than 32 Mb";
                push @generalrec,
                    $myvar{'log_error'}
                    . " is > 32Mb, you should analyze why or implement a rotation
log strategy such as logrotate!";
            }
        }
        else {
            infoprint
"Log file $myvar{'log_error'} is empty. Assuming log-rotation. Use
--server-log={file} for explicit file";
            return;
        }
    }
}

```

```

    }
    if ( !open( $fh, '<', $myvar{'log_error'} ) ) {
        badprint "Log file $myvar{'log_error'} isn't readable.";
        return;
    }
    goodprint "Log file $myvar{'log_error'} is readable.";

    if ( $maxlines * 80 < $size ) {
        seek( $fh, -$maxlines * 80, 2 );
        <$fh>;      # discard line fragment
    }
}
else {
    badprint "Log file $myvar{'log_error'} doesn't exist";
    return;
}

my $numLi      = 0;
my $nbWarnLog  = 0;
my $nbErrLog   = 0;
my @lastShutdowns;
my @lastStarts;

while ( my $logLi = <$fh> ) {
    chomp $logLi;
    $numLi++;
    debugprint "$numLi: $logLi"
        if $logLi =~ /warning|error/i and $logLi !~ /Logging to/;
    $nbErrLog++ if $logLi =~ /error/i and $logLi !~ /Logging to/;
    $nbWarnLog++ if $logLi =~ /warning/i;
    push @lastShutdowns, $logLi
        if $logLi =~ /Shutdown complete/ and $logLi !~ /Innodb/i;
    push @lastStarts, $logLi if $logLi =~ /ready for connections/;
}
close $fh;

if ( $nbWarnLog > 0 ) {
    badprint "$myvar{'log_error'} contains $nbWarnLog warning(s).";
    push @generalrec,
        "Control warning line(s) into $myvar{'log_error'} file";
}
else {
    goodprint "$myvar{'log_error'} doesn't contain any warning.";
}

if ( $nbErrLog > 0 ) {
    badprint "$myvar{'log_error'} contains $nbErrLog error(s).";
    push @generalrec, "Control error line(s) into $myvar{'log_error'} file";
}
else {

```

```

        goodprint "$myvar{'log_error'} doesn't contain any error.";
    }

    infoprint scalar @lastStarts . " start(s) detected in $myvar{'log_error'}";
    my $nStart = 0;
    my $nEnd   = 10;
    if ( scalar @lastStarts < $nEnd ) {
        $nEnd = scalar @lastStarts;
    }
    for my $startd ( reverse @lastStarts[ -$nEnd .. -1 ] ) {
        $nStart++;
        infoprint "$nStart) $startd";
    }
    infoprint scalar @lastShutdowns
        . " shutdown(s) detected in $myvar{'log_error'}";
    $nStart = 0;
    $nEnd   = 10;
    if ( scalar @lastShutdowns < $nEnd ) {
        $nEnd = scalar @lastShutdowns;
    }
    for my $shutd ( reverse @lastShutdowns[ -$nEnd .. -1 ] ) {
        $nStart++;
        infoprint "$nStart) $shutd";
    }

    #exit 0;
}

sub cve_recommendations {
    subheaderprint "CVE Security Recommendations";
    unless ( defined( $opt{cvefile} ) && -f "$opt{cvefile}" ) {
        infoprint "Skipped due to --cvefile option undefined";
        return;
    }

    # $mysqlvermajor=10;
    # $mysqlverminor=1;
    # $mysqlvermicro=17;
    # prettyprint "Look for related CVE for $myvar{'version'} or lower in
    $opt{cvefile}";
    my $cvefound = 0;
    open( my $fh, "<", $opt{cvefile} )
        or die "Can't open $opt{cvefile} for read: $!";
    while ( my $cveline = <$fh> ) {
        my @cve = split( ';', $cveline );
        debugprint
"Comparing $mysqlvermajor\.$mysqlverminor\.$mysqlvermicro with
$cve[1]\.$cve[2]\.$cve[3] : "
        . ( mysql_version_le( $cve[1], $cve[2], $cve[3] ) ? '<=' : '>' );
    }

```

```

# Avoid not major/minor version corresponding CVEs
next
    unless ( int( $cve[1] ) == $mysqlvermajor
        && int( $cve[2] ) == $mysqlverminor );
    if ( int( $cve[3] ) >= $mysqlvermicro ) {
        badprint "$cve[4](<= $cve[1]\.$cve[2]\.$cve[3]) : $cve[6]";
        $result{'CVE'}{'List'}{$cvefound} =
            "$cve[4](<= $cve[1]\.$cve[2]\.$cve[3]) : $cve[6]";
        $cvefound++;
    }
}
close $fh or die "Cannot close $opt{cvefile}: $!";
$result{'CVE'}{'nb'} = $cvefound;

my $cve_warning_notes = "";
if ( $cvefound == 0 ) {
    goodprint "NO SECURITY CVE FOUND FOR YOUR VERSION";
    return;
}
if ( $mysqlvermajor eq 5 and $mysqlverminor eq 5 ) {
    infoprint
        "False positive CVE(s) for MySQL and MariaDB 5.5.x can be found.";
    infoprint "Check careful each CVE for those particular versions";
}
badprint $cvefound . " CVE(s) found for your MySQL release.";
push( @generalrec,
    $cvefound
    . " CVE(s) found for your MySQL release. Consider upgrading your
version !"
);
}

sub get_opened_ports {
    my @opened_ports = `netstat -ltn`;
    @opened_ports = map {
        my $v = $_;
        $v =~ s/.*:(\d+)\s.*$/\1/;
        $v =~ s/\D//g;
        $v;
    } @opened_ports;
    @opened_ports = sort { $a <=> $b } grep { !/^$/ } @opened_ports;
    debugprint Dumper \@opened_ports;
    $result{'Network'}{'TCP Opened'} = \@opened_ports;
    return @opened_ports;
}

sub is_open_port {
    my $port = shift;

```

```

    if ( grep { /^$port$/ } get_opened_ports ) {
        return 1;
    }
    return 0;
}

sub get_process_memory {
    my $pid = shift;
    my @mem = `ps -p $pid -o rss`;
    return 0 if scalar @mem != 2;
    return $mem[1] * 1024;
}

sub get_other_process_memory {
    return 0 if ( $opt{tbsstat} == 0 );
    my @procs = `ps eaxo pid,command`;
    @procs = map {
        my $v = $_;
        $v =~ s/.*PID.*//;
        $v =~ s/.*mysqld.*//;
        $v =~ s/.*\[.*\].*//;
        $v =~ s/^\s+$/g;
        $v =~ s/.*PID.*CMD.*//;
        $v =~ s/.*systemd.*//;
        $v =~ s/\s*?(\d+)\s*.*$/1/g;
        $v;
    } @procs;
    @procs = remove_cr @procs;
    @procs = remove_empty @procs;
    my $totalMemOther = 0;
    map { $totalMemOther += get_process_memory($_); } @procs;
    return $totalMemOther;
}

sub get_os_release {
    if ( -f "/etc/lsb-release" ) {
        my @info_release = get_file_contents "/etc/lsb-release";
        my $os_release = $info_release[3];
        $os_release =~ s/.*="//;
        $os_release =~ s/"$/;
        return $os_release;
    }

    if ( -f "/etc/system-release" ) {
        my @info_release = get_file_contents "/etc/system-release";
        return $info_release[0];
    }

    if ( -f "/etc/os-release" ) {

```

```

    my @info_release = get_file_contents "/etc/os-release";
    my $os_release = $info_release[0];
    $os_release =~ s/.*="//;
    $os_release =~ s/"$//;
    return $os_release;
}

if ( -f "/etc/issue" ) {
    my @info_release = get_file_contents "/etc/issue";
    my $os_release = $info_release[0];
    $os_release =~ s/\s+\n.*//;
    return $os_release;
}
return "Unknown OS release";
}

sub get_fs_info {
    my @sinfo = `df -P | grep '%'`;
    my @iinfo = `df -Pi | grep '%'`;
    shift @iinfo;
    @sinfo = map {
        my $v = $_;
        $v =~ s/.*\s(\d+)\s+(\.+)/$1\t$2/g;
        $v;
    } @sinfo;
    foreach my $info (@sinfo) {
        next if $info =~ m{(\d+)\t/(run|dev|sys|proc|snap) ($|/)};
        if ( $info =~ /(\d+)\t(.*)/ ) {
            if ( $1 > 85 ) {
                badprint "mount point $2 is using $1 % total space";
                push( @generalrec, "Add some space to $2 mountpoint." );
            }
            else {
                infoprint "mount point $2 is using $1 % of total space";
            }
            $result{'Filesystem'}{'Space Pct'}{$2} = $1;
        }
    }
}

@iinfo = map {
    my $v = $_;
    $v =~ s/.*\s(\d+)\s+(\.+)/$1\t$2/g;
    $v;
} @iinfo;
foreach my $info (@iinfo) {
    next if $info =~ m{(\d+)\t/(run|dev|sys|proc) ($|/)};
    if ( $info =~ /(\d+)\t(.*)/ ) {
        if ( $1 > 85 ) {
            badprint "mount point $2 is using $1 % of max allowed inodes";

```

```

                push( @generalrec,
"Cleanup files from $2 mountpoint or reformat you filesystem."
                );
            }
            else {
                infoprint "mount point $2 is using $1 % of max allowed inodes";
            }
            $result{'Filesystem'}{'Inode Pct'}{$2} = $1;
        }
    }
}

sub merge_hash {
    my $h1      = shift;
    my $h2      = shift;
    my %result = {};
    foreach my $substanceref ( $h1, $h2 ) {
        while ( my ( $k, $v ) = each %$substanceref ) {
            next if ( exists $result{$k} );
            $result{$k} = $v;
        }
    }
    return \%result;
}

sub is_virtual_machine {
    my $isVm = `grep -Ec '^flags.*\hypervisor\' /proc/cpuinfo`;
    return ( $isVm == 0 ? 0 : 1 );
}

sub infocmd {
    my $cmd = "@_";
    debugprint "CMD: $cmd";
    my @result = `$cmd`;
    @result = remove_cr @result;
    for my $l (@result) {
        infoprint "$l";
    }
}

sub infocmd_tab {
    my $cmd = "@_";
    debugprint "CMD: $cmd";
    my @result = `$cmd`;
    @result = remove_cr @result;
    for my $l (@result) {
        infoprint "\t$l";
    }
}

```

```

sub infocmd_one {
    my $cmd      = "@_";
    my @result = ` $cmd 2>&1 `;
    @result = remove_cr @result;
    return join ' ', @result;
}

sub get_kernel_info {
    my @params = (
        'fs.aio-max-nr',          'fs.aio-nr',
        'fs.file-max',           'sunrpc.tcp_fin_timeout',
        'sunrpc.tcp_max_slot_table_entries', 'sunrpc.tcp_slot_table_entries',
        'vm.swappiness'
    );
    infoprint "Information about kernel tuning:";
    foreach my $param (@params) {
        infocmd_tab("sysctl $param 2>/dev/null");
        $result{'OS'}{'Config'}{$param} = `sysctl -n $param 2>/dev/null`;
    }
    if ( `sysctl -n vm.swappiness` > 10 ) {
        badprint
            "Swappiness is > 10, please consider having a value lower than 10";
        push @generalrec, "setup swappiness lower or equals to 10";
        push @adjvars,
            'vm.swappiness <= 10 (echo 10 > /proc/sys/vm/swappiness)';
    }
    else {
        infoprint "Swappiness is < 10.";
    }

    # only if /proc/sys/sunrpc exists
    my $tcp_slot_entries =
        `sysctl -n sunrpc.tcp_slot_table_entries 2>/dev/null`;
    if ( -f "/proc/sys/sunrpc"
        and ( $tcp_slot_entries eq '' or $tcp_slot_entries < 100 ) )
    {
        badprint
            "Initial TCP slot entries is < 1M, please consider having a value greater than 100";
        push @generalrec, "setup Initial TCP slot entries greater than 100";
        push @adjvars,
            'sunrpc.tcp_slot_table_entries > 100 (echo 128 > /proc/sys/sunrpc/tcp_slot_table_entries)';
    }
    else {
        infoprint "TCP slot entries is > 100.";
    }
}

```

```

    if ( `sysctl -n fs.aio-max-nr` < 1000000 ) {
        badprint
        "Max running total of the number of events is < 1M, please consider having a
        value greater than 1M";
        push @generalrec, "setup Max running number events greater than 1M";
        push @adjvars,
            'fs.aio-max-nr > 1M (echo 1048576 > /proc/sys/fs/aio-max-nr)';
    }
    else {
        infoprint "Max Number of AIO events is > 1M.";
    }
}

sub get_system_info {
    $result{'OS'}{'Release'} = get_os_release();
    infoprint get_os_release;
    if (is_virtual_machine) {
        infoprint "Machine type          : Virtual machine";
        $result{'OS'}{'Virtual Machine'} = 'YES';
    }
    else {
        infoprint "Machine type          : Physical machine";
        $result{'OS'}{'Virtual Machine'} = 'NO';
    }

    $result{'Network'}{'Connected'} = 'NO';
    `ping -c 1 ipecho.net &>/dev/null`;
    my $isConnected = $?;
    if ( $? == 0 ) {
        infoprint "Internet              : Connected";
        $result{'Network'}{'Connected'} = 'YES';
    }
    else {
        badprint "Internet              : Disconnected";
    }

    $result{'OS'}{'NbCore'} = cpu_cores;
    infoprint "Number of Core CPU : " . cpu_cores;
    $result{'OS'}{'Type'} = `uname -o`;
    infoprint "Operating System Type : " . infocmd_one "uname -o";
    $result{'OS'}{'Kernel'} = `uname -r`;
    infoprint "Kernel Release          : " . infocmd_one "uname -r";
    $result{'OS'}{'Hostname'} = `hostname`;
    $result{'Network'}{'Internal Ip'} = `hostname -I`;
    infoprint "Hostname                : " . infocmd_one "hostname";
    infoprint "Network Cards           : ";
    infocmd_tab "ifconfig| grep -Al mtu";
    infoprint "Internal IP              : " . infocmd_one "hostname -I";
    $result{'Network'}{'Internal Ip'} = `ifconfig| grep -Al mtu`;

```

```

my $httpcli = get_http_cli();
infoprint "HTTP client found: $httpcli" if defined $httpcli;

my $ext_ip = "";
if ( $httpcli =~ /curl$/ ) {
    $ext_ip = infocmd_one "$httpcli -m 3 ipecho.net/plain";
}
elseif ( $httpcli =~ /wget$/ ) {

    $ext_ip = infocmd_one "$httpcli -t 1 -T 3 -q -O - ipecho.net/plain";
}
infoprint "External IP          : " . $ext_ip;
$result{'Network'}{'External Ip'} = $ext_ip;
badprint
    "External IP          : Can't check because of Internet connectivity"
unless defined($httpcli);
infoprint "Name Servers          : "
    . infocmd_one "grep 'nameserver' /etc/resolv.conf \| awk '{print \$2}'";
infoprint "Logged In users        : ";
infocmd_tab "who";
$result{'OS'}{'Logged users'} = `who`;
infoprint "Ram Usages in Mb        : ";
infocmd_tab "free -m | grep -v +";
$result{'OS'}{'Free Memory RAM'} = `free -m | grep -v +`;
infoprint "Load Average            : ";
infocmd_tab "top -n 1 -b | grep 'load average:'";
$result{'OS'}{'Load Average'} = `top -n 1 -b | grep 'load average:'`;

infoprint "System Uptime            : ";
infocmd_tab "uptime";
$result{'OS'}{'Uptime'} = `uptime`;
}

sub system_recommendations {
    return if ( $opt{sysstat} == 0 );
    subheaderprint "System Linux Recommendations";
    my $os = `uname`;
    unless ( $os =~ /Linux/i ) {
        infoprint "Skipped due to non Linux server";
        return;
    }
    prettyprint "Look for related Linux system recommendations";

    #prettyprint '- 'x78;
    get_system_info();
    my $omem = get_other_process_memory;
    infoprint "User process except mysqld used "
        . hr_bytes_rnd($omem) . " RAM.";
    if ( ( 0.15 * $physical_memory ) < $omem ) {

```

```

        badprint
"Other user process except mysqld used more than 15% of total physical memory "
        . percentage( $omem, $physical_memory ) . "% ("
        . hr_bytes_rnd($omem) . " / "
        . hr_bytes_rnd($physical_memory) . ")";
        push( @generalrec,
"Consider stopping or dedicate server for additional process other than
mysqld."
        );
        push( @adjvars,
"DON'T APPLY SETTINGS BECAUSE THERE ARE TOO MANY PROCESSES RUNNING ON THIS
SERVER. OOM KILL CAN OCCUR!"
        );
    }
    else {
        infoprint
"Other user process except mysqld used less than 15% of total physical memory "
        . percentage( $omem, $physical_memory ) . "% ("
        . hr_bytes_rnd($omem) . " / "
        . hr_bytes_rnd($physical_memory) . ")";
    }

    if ( $opt{'maxportallowed'} > 0 ) {
        my @opened_ports = get_opened_ports;
        infoprint "There is "
        . scalar @opened_ports
        . " listening port(s) on this server.";
        if ( scalar(@opened_ports) > $opt{'maxportallowed'} ) {
            badprint "There is too many listening ports: "
            . scalar(@opened_ports)
            . " opened > "
            . $opt{'maxportallowed'}
            . "allowed.";
            push( @generalrec,
"Consider dedicating a server for your database installation with less services
running on !"
            );
        }
        else {
            goodprint "There is less than "
            . $opt{'maxportallowed'}
            . " opened ports on this server.";
        }
    }

    foreach my $banport (@banned_ports) {
        if ( is_open_port($banport) ) {
            badprint "Banned port: $banport is opened..";
            push( @generalrec,

```

```

"Port $banport is opened. Consider stopping program handling this port."
    );
}
else {
    goodprint "$banport is not opened.";
}
}

get_fs_info;
get_kernel_info;
}

sub security_recommendations {
    subheaderprint "Security Recommendations";

    if ( mysql_version_eq(8) ) {
        infoprint "Skipped due to unsupported feature for MySQL 8";
        return;
    }

    #exit 0;
    if ( $opt{skippassword} eq 1 ) {
        infoprint "Skipped due to --skippassword option";
        return;
    }

    my $PASS_COLUMN_NAME = 'password';

    # New table schema available since mysql-5.7 and mariadb-10.2
    # But need to be checked
    if ( $myvar{'version'} =~ /5\.7|10\.[2-5]\.\.*MariaDB*/ ) {
        my $password_column_exists =
`$mysqlcmd $mysqllogin -Bse "SELECT 1 FROM information_schema.columns WHERE
TABLE_SCHEMA = 'mysql' AND TABLE_NAME = 'user' AND COLUMN_NAME = 'password'"
2>>/dev/null`;
        my $authstring_column_exists =
`$mysqlcmd $mysqllogin -Bse "SELECT 1 FROM information_schema.columns WHERE
TABLE_SCHEMA = 'mysql' AND TABLE_NAME = 'user' AND COLUMN_NAME =
'authentication_string'" 2>>/dev/null`;
        if ( $password_column_exists && $authstring_column_exists ) {
            $PASS_COLUMN_NAME =
"IF(plugin='mysql_native_password', authentication_string, password)";
        }
        elsif ( $authstring_column_exists ) {
            $PASS_COLUMN_NAME = 'authentication_string';
        }
        elsif ( !$password_column_exists ) {
            infoprint "Skipped due to none of known auth columns exists";
            return;
        }
    }
}

```

```

    }
}
debugprint "Password column = $PASS_COLUMN_NAME";

# Looking for Anonymous users
my @mysqlstatlist = select_array
"SELECT CONCAT(QUOTE(user), '\\@', QUOTE(host)) FROM mysql.user WHERE TRIM(USER)
= '' OR USER IS NULL";
debugprint Dumper \@mysqlstatlist;

#exit 0;
if (@mysqlstatlist) {
    push( @generalrec,
        "Remove Anonymous User accounts - there are "
        . scalar(@mysqlstatlist)
        . " anonymous accounts." );
    foreach my $line ( sort @mysqlstatlist ) {
        chomp($line);
        badprint "User "
            . $line
            . " is an anonymous account. Remove with DROP USER "
            . $line . ";;";
    }
}
else {
    goodprint "There are no anonymous accounts for any database users";
}
if ( mysql_version_le( 5, 1 ) ) {
    badprint "No more password checks for MySQL version <=5.1";
    badprint "MySQL version <=5.1 are deprecated and end of support.";
    return;
}

# Looking for Empty Password
if ( mysql_version_ge( 10, 4 ) ) {
    @mysqlstatlist = select_array
q{SELECT CONCAT(QUOTE(user), '@', QUOTE(host)) FROM mysql.global_priv WHERE
user != ''
AND JSON_CONTAINS(Priv, '"mysql_native_password"', '$.plugin') AND
JSON_CONTAINS(Priv, '""', '$.authentication_string')
AND NOT JSON_CONTAINS(Priv, 'true', '$.account_locked')};
}
else {
    @mysqlstatlist = select_array
"SELECT CONCAT(QUOTE(user), '\\@', QUOTE(host)) FROM mysql.user WHERE
($PASS_COLUMN_NAME = '' OR $PASS_COLUMN_NAME IS NULL)
AND user != ''
/*!50501 AND plugin NOT IN ('auth_socket', 'unix_socket', 'win_socket',
'auth_pam_compat') */";
}

```

```

/*!80000 AND account_locked = 'N' AND password_expired = 'N' */";
}
if (@mysqlstatlist) {
    foreach my $line ( sort @mysqlstatlist ) {
        chomp($line);
        badprint "User '" . $line . "' has no password set.";
        push( @generalrec,
"Set up a Secure Password for $line user: SET PASSWORD FOR $line =
PASSWORD('secure_password');"
        );
    }
}
else {
    goodprint "All database users have passwords assigned";
}

if ( mysql_version_ge( 5, 7 ) ) {
    my $valPlugin = select_one(
"select count(*) from information_schema.plugins where
PLUGIN_NAME='validate_password' AND PLUGIN_STATUS='ACTIVE'"
    );
    if ( $valPlugin >= 1 ) {
        infoprint
"Bug #80860 MySQL 5.7: Avoid testing password when validate_password is
activated";
        return;
    }
}

# Looking for User with user/ uppercase /capitalise user as password
@mysqlstatlist = select_array
"SELECT CONCAT(QUOTE(user), '\\@\\', QUOTE(host)) FROM mysql.user WHERE user != ''
AND (CAST($PASS_COLUMN_NAME as Binary) = PASSWORD(user) OR
CAST($PASS_COLUMN_NAME as Binary) = PASSWORD(UPPER(user)) OR
CAST($PASS_COLUMN_NAME as Binary) = PASSWORD(CONCAT(UPPER(LEFT(User, 1)),
SUBSTRING(User, 2, LENGTH(User)))));";
if (@mysqlstatlist) {
    foreach my $line ( sort @mysqlstatlist ) {
        chomp($line);
        badprint "User " . $line . " has user name as password.";
        push( @generalrec,
"Set up a Secure Password for $line user: SET PASSWORD FOR $line =
PASSWORD('secure_password');"
        );
    }
}

@mysqlstatlist = select_array
"SELECT CONCAT(QUOTE(user), '\\@\\', host) FROM mysql.user WHERE HOST='%';";

```

```

if (@mysqlstatlist) {
    foreach my $line ( sort @mysqlstatlist ) {
        chomp($line);
        my $luser = ( split /\@/, $line )[0];
        badprint "User '" . $line
            . "' does not specify hostname restrictions.";
        push( @generalrec,
            "Restrict Host for $luser\@% to $luser\@LimitedIPRangeOrLocalhost"
        );
        push( @generalrec,
            "RENAME USER $luser\@'" . $luser
            . "\@LimitedIPRangeOrLocalhost;" );
    }
}

unless ( -f $basic_password_files ) {
    badprint "There is no basic password file list!";
    return;
}

my @passwords = get_basic_passwords $basic_password_files;
infoprint "There are "
    . scalar(@passwords)
    . " basic passwords in the list.";
my $nbins = 0;
my $passreq;
if (@passwords) {
    my $nbInterPass = 0;
    foreach my $pass (@passwords) {
        $nbInterPass++;

        $pass =~ s/\s//g;
        $pass =~ s/\'/\\'/g;
        chomp($pass);

        # Looking for User with user/ uppercase /capitalise weak password
        @mysqlstatlist =
            select_array
            "SELECT CONCAT(user, '\@', host) FROM mysql.user WHERE $PASS_COLUMN_NAME =
            PASSWORD('"
                . $pass
                . "') OR $PASS_COLUMN_NAME = PASSWORD(UPPER('"
                . $pass
                . "')) OR $PASS_COLUMN_NAME = PASSWORD(CONCAT(UPPER(LEFT('"
                . $pass
                . "', 1)), SUBSTRING('"
                . $pass
                . "', 2, LENGTH('"

```

```

        . $pass . "'))))";
    debugprint "There is " . scalar(@mysqlstatlist) . " items.";
    if (@mysqlstatlist) {
        foreach my $line (@mysqlstatlist) {
            chomp($line);
            badprint "User '" . $line
                . "' is using weak password: $pass in a lower, upper or
capitalize derivative version.";

            push( @generalrec,
                "Set up a Secure Password for $line user: SET PASSWORD FOR '"
                    . ( split /\@/, $line )[0] . "'\@"
                    . ( split /\@/, $line )[1]
                    . "' = PASSWORD('secure_password');" );
            $nbins++;
        }
        debugprint "$nbInterPass / " . scalar(@passwords)
            if ( $nbInterPass % 1000 == 0 );
    }
}

if ( $nbins > 0 ) {
    push( @generalrec,
        $nbins
        . " user(s) used basic or weak password from basic dictionary." );
}
}

sub get_replication_status {
    subheaderprint "Replication Metrics";
    infoprint "Galera Synchronous replication: " . $myvar{'have_galera'};
    if ( scalar( keys %myslaves ) == 0 ) {
        infoprint "No replication slave(s) for this server.";
    }
    else {
        infoprint "This server is acting as master for "
            . scalar( keys %myslaves )
            . " server(s).";
    }
    infoprint "Binlog format: " . $myvar{'binlog_format'};
    infoprint "XA support enabled: " . $myvar{'innodb_support_xa'};

    infoprint "Semi synchronous replication Master: "
        . (
            defined( $myvar{'rpl_semi_sync_master_enabled'} )
            ? $myvar{'rpl_semi_sync_master_enabled'}
            : 'Not Activated'
        );
    infoprint "Semi synchronous replication Slave: "

```

```

. (
    defined( $myvar{'rpl_semi_sync_slave_enabled'} )
    ? $myvar{'rpl_semi_sync_slave_enabled'}
    : 'Not Activated'
);
if ( scalar( keys %myrepl ) == 0 and scalar( keys %myslaves ) == 0 ) {
    infoprint "This is a standalone server";
    return;
}
if ( scalar( keys %myrepl ) == 0 ) {
    infoprint
        "No replication setup for this server or replication not started.";
    return;
}

$result{'Replication'}{'status'} = \%myrepl;
my ($io_running) = $myrepl{'Slave_IO_Running'};
debugprint "IO RUNNING: $io_running ";
my ($sql_running) = $myrepl{'Slave_SQL_Running'};
debugprint "SQL RUNNING: $sql_running ";
my ($seconds_behind_master) = $myrepl{'Seconds_Behind_Master'};
debugprint "SECONDS : $seconds_behind_master ";

if ( defined($io_running)
    and ( $io_running !~ /yes/i or $sql_running !~ /yes/i ) )
{
    badprint
        "This replication slave is not running but seems to be configured.";
}
if ( defined($io_running)
    && $io_running =~ /yes/i
    && $sql_running =~ /yes/i )
{
    if ( $myvar{'read_only'} eq 'OFF' ) {
        badprint
            "This replication slave is running with the read_only option disabled.";
    }
    else {
        goodprint
            "This replication slave is running with the read_only option enabled.";
    }
    if ( $seconds_behind_master > 0 ) {
        badprint
            "This replication slave is lagging and slave has $seconds_behind_master
            second(s) behind master host.";
    }
    else {
        goodprint "This replication slave is up to date with master.";
    }
}

```

```

    }
}

sub validate_mysql_version {
    ( $mysqlvermajor, $mysqlverminor, $mysqlvermicro ) =
        $myvar{'version'} =~ /^(\\d+)(?:\\. (\\d+)|)(?:\\. (\\d+)|)/;
    $mysqlverminor ||= 0;
    $mysqlvermicro ||= 0;

    if ( mysql_version_eq(8)
        or mysql_version_eq( 5, 6 )
        or mysql_version_eq( 5, 7 )
        or mysql_version_eq( 10, 2 )
        or mysql_version_eq( 10, 3 )
        or mysql_version_eq( 10, 4 )
        or mysql_version_eq( 10, 5 ) )
    {
        goodprint "Currently running supported MySQL version "
            . $myvar{'version'} . "";
        return;
    }
    if ( mysql_version_ge(5)
        or mysql_version_ge(4)
        or mysql_version_eq( 10, 0 ) )
    {
        badprint "Your MySQL version "
            . $myvar{'version'}
            . " is EOL software! Upgrade soon!";
    }
}

# Checks if MySQL version is equal to (major, minor, micro)
sub mysql_version_eq {
    my ( $maj, $min, $mic ) = @_ ;
    my ( $mysqlvermajor, $mysqlverminor, $mysqlvermicro ) =
        $myvar{'version'} =~ /^(\\d+)(?:\\. (\\d+)|)(?:\\. (\\d+)|)/;

    return int($mysqlvermajor) == int($maj)
        if ( !defined($min) && !defined($mic) );
    return int($mysqlvermajor) == int($maj) && int($mysqlverminor) == int($min)
        if ( !defined($mic) );
    return ( int($mysqlvermajor) == int($maj)
        && int($mysqlverminor) == int($min)
        && int($mysqlvermicro) == int($mic) );
}

# Checks if MySQL version is greater than equal to (major, minor, micro)
sub mysql_version_ge {
    my ( $maj, $min, $mic ) = @_ ;

```

```

$min ||= 0;
$mic ||= 0;
my ( $mysqlvermajor, $mysqlverminor, $mysqlvermicro ) =
    $myvar{'version'} =~ /^(\\d+)(?:\\. (\\d+)|)(?:\\. (\\d+)|)/;

return
    int($mysqlvermajor) > int($maj)
    || ( int($mysqlvermajor) == int($maj) && int($mysqlverminor) > int($min) )
    || ( int($mysqlvermajor) == int($maj)
        && int($mysqlverminor) == int($min)
        && int($mysqlvermicro) >= int($mic) );
}

# Checks if MySQL version is lower than equal to (major, minor, micro)
sub mysql_version_le {
    my ( $maj, $min, $mic ) = @_;
    $min ||= 0;
    $mic ||= 0;
    my ( $mysqlvermajor, $mysqlverminor, $mysqlvermicro ) =
        $myvar{'version'} =~ /^(\\d+)(?:\\. (\\d+)|)(?:\\. (\\d+)|)/;
    return
        int($mysqlvermajor) < int($maj)
        || ( int($mysqlvermajor) == int($maj) && int($mysqlverminor) < int($min) )
        || ( int($mysqlvermajor) == int($maj)
            && int($mysqlverminor) == int($min)
            && int($mysqlvermicro) <= int($mic) );
}

# Checks if MySQL micro version is lower than equal to (major, minor, micro)
sub mysql_micro_version_le {
    my ( $maj, $min, $mic ) = @_;
    my ( $mysqlvermajor, $mysqlverminor, $mysqlvermicro ) =
        $myvar{'version'} =~ /^(\\d+)(?:\\. (\\d+)|)(?:\\. (\\d+)|)/;

    return $mysqlvermajor == $maj
        && ( $mysqlverminor == $min
            && $mysqlvermicro <= $mic );
}

# Checks for 32-bit boxes with more than 2GB of RAM
my ($arch);

sub check_architecture {
    if ( $doremove eq 1 ) { return; }
    if ( `uname` =~ /SunOS/ && `isainfo -b` =~ /64/ ) {
        $arch = 64;
        goodprint "Operating on 64-bit architecture";
    }
    elsif ( `uname` !~ /SunOS/ && `uname -m` =~ /(64|s390x)/ ) {

```

```

        $arch = 64;
        goodprint "Operating on 64-bit architecture";
    }
    elif ( `uname` =~ /AIX/ && `bootinfo -K` =~ /64/ ) {
        $arch = 64;
        goodprint "Operating on 64-bit architecture";
    }
    elif ( `uname` =~ /NetBSD|OpenBSD/ && `sysctl -b hw.machine` =~ /64/ ) {
        $arch = 64;
        goodprint "Operating on 64-bit architecture";
    }
    elif ( `uname` =~ /FreeBSD/ && `sysctl -b hw.machine_arch` =~ /64/ ) {
        $arch = 64;
        goodprint "Operating on 64-bit architecture";
    }
    elif ( `uname` =~ /Darwin/ && `uname -m` =~ /Power Macintosh/ ) {

# Darwin box.local 9.8.0 Darwin Kernel Version 9.8.0: Wed Jul 15 16:57:01 PDT
2009; root:xnu1228.15.4~1/RELEASE_PPC Power Macintosh
        $arch = 64;
        goodprint "Operating on 64-bit architecture";
    }
    elif ( `uname` =~ /Darwin/ && `uname -m` =~ /x86_64/ ) {

# Darwin gibas.local 12.3.0 Darwin Kernel Version 12.3.0: Sun Jan 6 22:37:10
PST 2013; root:xnu-2050.22.13~1/RELEASE_X86_64 x86_64
        $arch = 64;
        goodprint "Operating on 64-bit architecture";
    }
    else {
        $arch = 32;
        if ( $physical_memory > 2147483648 ) {
            badprint
"Switch to 64-bit OS - MySQL cannot currently use all of your RAM";
        }
        else {
            goodprint "Operating on 32-bit architecture with less than 2GB RAM";
        }
    }
    $result{'OS'}{'Architecture'} = "$arch bits";
}

# Start up a ton of storage engine counts/statistics
my ( %enginestats, %enginecount, $fragtables );

sub check_storage_engines {
    subheaderprint "Storage Engine Statistics";
    if ( $opt{skipsizes} eq 1 ) {

```

```

        infoprint "Skipped due to --skipsize option";
        return;
    }

    my $engines;
    if ( mysql_version_ge( 5, 5 ) ) {
        my @engineresults = select_array
"SELECT ENGINE,SUPPORT FROM information_schema.ENGINES ORDER BY ENGINE ASC";
        foreach my $line (@engineresults) {
            my ( $engine, $engineenabled );
            ( $engine, $engineenabled ) = $line =~ /([a-zA-Z_]*)\s+([a-zA-Z]+)/;
            $result{'Engine'}{$engine}{'Enabled'} = $engineenabled;
            $engines .=
                ( $engineenabled eq "YES" || $engineenabled eq "DEFAULT" )
                ? greenwrap "+" . $engine . " "
                : redwrap "-" . $engine . " ";
        }
    }
    elsif ( mysql_version_ge( 5, 1, 5 ) ) {
        my @engineresults = select_array
"SELECT ENGINE,SUPPORT FROM information_schema.ENGINES WHERE ENGINE NOT IN
('performance_schema','MyISAM','MERGE','MEMORY') ORDER BY ENGINE ASC";
        foreach my $line (@engineresults) {
            my ( $engine, $engineenabled );
            ( $engine, $engineenabled ) = $line =~ /([a-zA-Z_]*)\s+([a-zA-Z]+)/;
            $result{'Engine'}{$engine}{'Enabled'} = $engineenabled;
            $engines .=
                ( $engineenabled eq "YES" || $engineenabled eq "DEFAULT" )
                ? greenwrap "+" . $engine . " "
                : redwrap "-" . $engine . " ";
        }
    }
    else {
        $engines .=
            ( defined $myvar{'have_archive'} && $myvar{'have_archive'} eq "YES" )
            ? greenwrap "+Archive "
            : redwrap "-Archive ";
        $engines .=
            ( defined $myvar{'have_bdb'} && $myvar{'have_bdb'} eq "YES" )
            ? greenwrap "+BDB "
            : redwrap "-BDB ";
        $engines .=
            ( defined $myvar{'have_federated_engine'}
              && $myvar{'have_federated_engine'} eq "YES" )
            ? greenwrap "+Federated "
            : redwrap "-Federated ";
        $engines .=
            ( defined $myvar{'have_innodb'} && $myvar{'have_innodb'} eq "YES" )
            ? greenwrap "+InnoDB "

```

```

        : redwrap "-InnoDB ";
$engines .=
    ( defined $myvar{'have_isam'} && $myvar{'have_isam'} eq "YES" )
    ? greenwrap "+ISAM "
    : redwrap "-ISAM ";
$engines .=
    ( defined $myvar{'have_ndbcluster'}
      && $myvar{'have_ndbcluster'} eq "YES" )
    ? greenwrap "+NDBCluster "
    : redwrap "-NDBCluster ";
}

my @dblist = grep { $_ ne 'lost+found' } select_array "SHOW DATABASES";

$result{'Databases'}{'List'} = [@dblist];
infoprint "Status: $engines";
if ( mysql_version_ge( 5, 1, 5 ) ) {

# MySQL 5 servers can have table sizes calculated quickly from information
# schema
    my @templist = select_array
    "SELECT
ENGINE,SUM(DATA_LENGTH+INDEX_LENGTH),COUNT(ENGINE),SUM(DATA_LENGTH),SUM(INDEX_L
ENGTH) FROM information_schema.TABLES WHERE TABLE_SCHEMA NOT IN
('information_schema', 'performance_schema', 'mysql') AND ENGINE IS NOT NULL
GROUP BY ENGINE ORDER BY ENGINE ASC;";

    my ( $engine, $size, $count, $dsize, $isize );
    foreach my $line (@templist) {
        ( $engine, $size, $count, $dsize, $isize ) =
            $line =~ /([a-zA-Z_]+)\s+(\d+)\s+(\d+)\s+(\d+)\s+(\d+)/;
        debugprint "Engine Found: $engine";
        next unless ( defined($engine) or trim($engine) eq '' );
        $size = 0 unless ( defined($size) or trim($engine) eq '' );
        $isize = 0 unless ( defined($isize) or trim($engine) eq '' );
        $dsize = 0 unless ( defined($dsize) or trim($engine) eq '' );
        $count = 0 unless ( defined($count) or trim($engine) eq '' );
        $enginestats{$engine} = $size;
        $enginecount{$engine} = $count;
        $result{'Engine'}{$engine}{'Table Number'} = $count;
        $result{'Engine'}{$engine}{'Total Size'} = $size;
        $result{'Engine'}{$engine}{'Data Size'} = $dsize;
        $result{'Engine'}{$engine}{'Index Size'} = $isize;
    }
    my $not_innodb = '';
    if ( not defined $result{'Variables'}{'innodb_file_per_table'} ) {
        $not_innodb = "AND NOT ENGINE='InnoDB'";
    }
    elsif ( $result{'Variables'}{'innodb_file_per_table'} eq 'OFF' ) {

```

```

        $not_innodb = "AND NOT ENGINE='InnoDB'";
    }
    $result{'Tables'}{'Fragmented tables'} =
        [ select_array
"SELECT CONCAT(CONCAT(TABLE_SCHEMA, '.'), TABLE_NAME), DATA_FREE FROM
information_schema.TABLES WHERE TABLE_SCHEMA NOT IN
('information_schema', 'performance_schema', 'mysql') AND
DATA_LENGTH/1024/1024>100 AND
DATA_FREE*100/(DATA_LENGTH+INDEX_LENGTH+DATA_FREE) > 10 AND NOT ENGINE='MEMORY'
$not_innodb"
        ];
    $fragtables = scalar @{$result{'Tables'}{'Fragmented tables'}};

}
else {

    # MySQL < 5 servers take a lot of work to get table sizes
    my @tblist;

    # Now we build a database list, and loop through it to get storage engine
    stats for tables
    foreach my $db (@dblist) {
        chomp($db);
        if ( $db eq "information_schema"
            or $db eq "performance_schema"
            or $db eq "mysql"
            or $db eq "lost+found" )
        {
            next;
        }
        my @ixs = ( 1, 6, 9 );
        if ( !mysql_version_ge( 4, 1 ) ) {

            # MySQL 3.23/4.0 keeps Data_Length in the 5th (0-based) column
            @ixs = ( 1, 5, 8 );
        }
        push( @tblist,
            map { [ (split)[@ixs] ] }
                select_array "SHOW TABLE STATUS FROM \\`$db\\`" );
    }

    # Parse through the table list to generate storage engine
    counts/statistics
    $fragtables = 0;
    foreach my $tbl (@tblist) {
        debugprint "Data dump " . Dumper(@$tbl);
        my ( $engine, $size, $datafree ) = @$tbl;
        next if $engine eq 'NULL' or not defined($engine);
        $size      = 0 if $size eq 'NULL'      or not defined($size);
    }
}

```

```

        $datafree = 0 if $datafree eq 'NULL' or not defined($datafree);
        if ( defined $enginestats{$engine} ) {
            $enginestats{$engine} += $size;
            $enginecount{$engine} += 1;
        }
        else {
            $enginestats{$engine} = $size;
            $enginecount{$engine} = 1;
        }
        if ( $datafree > 0 ) {
            $fragtables++;
        }
    }
}

while ( my ( $engine, $size ) = each(%enginestats) ) {
    infoprint "Data in $engine tables: "
        . hr_bytes($size)
        . " (Tables: "
        . $enginecount{$engine} . ")" . " ";
}

# If the storage engine isn't being used, recommend it to be disabled
if ( !defined $enginestats{'InnoDB'}
    && defined $myvar{'have_innodb'}
    && $myvar{'have_innodb'} eq "YES" )
{
    badprint "InnoDB is enabled but isn't being used";
    push( @generalrec,
        "Add skip-innodb to MySQL configuration to disable InnoDB" );
}

if ( !defined $enginestats{'BerkeleyDB'}
    && defined $myvar{'have_bdb'}
    && $myvar{'have_bdb'} eq "YES" )
{
    badprint "BDB is enabled but isn't being used";
    push( @generalrec,
        "Add skip-bdb to MySQL configuration to disable BDB" );
}

if ( !defined $enginestats{'ISAM'}
    && defined $myvar{'have_isam'}
    && $myvar{'have_isam'} eq "YES" )
{
    badprint "MYISAM is enabled but isn't being used";
    push( @generalrec,
        "Add skip-isam to MySQL configuration to disable ISAM (MySQL > 4.1.0)"
    );
}

# Fragmented tables

```

```

if ( $fragtables > 0 ) {
    badprint "Total fragmented tables: $fragtables";
    push( @generalrec,
        "Run OPTIMIZE TABLE to defragment tables for better performance" );
    my $total_free = 0;
    foreach my $table_line ( @{ $result{'Tables'}{'Fragmented tables'} } ) {
        my ( $full_table_name, $data_free ) = split( /\s+/, $table_line );
        $data_free = 0 if ( !defined($data_free) or $data_free eq '' );
        $data_free = $data_free / 1024 / 1024;
        $total_free += $data_free;
        my ( $table_schema, $table_name ) = split( /\.\/, $full_table_name );
        push( @generalrec,
            " OPTIMIZE TABLE `"$table_schema`"."$table_name`; -- can free $data_free MB"
        );
    }
    push( @generalrec,
        "Total freed space after theses OPTIMIZE TABLE : $total_free Mb" );
}
else {
    goodprint "Total fragmented tables: $fragtables";
}

# Auto increments
my %tblist;

# Find the maximum integer
my $maxint = select_one "SELECT ~0";
$result{'MaxInt'} = $maxint;

# Now we use a database list, and loop through it to get storage engine stats
for tables
    foreach my $db (@dblist) {
        chomp($db);

        if ( !$tblist{$db} ) {
            $tblist{$db} = ();
        }

        if ( $db eq "information_schema" ) { next; }
        my @ia = ( 0, 10 );
        if ( !mysql_version_ge( 4, 1 ) ) {

            # MySQL 3.23/4.0 keeps Data_Length in the 5th (0-based) column
            @ia = ( 0, 9 );
        }
        push(
            @{ $tblist{$db} },
            map { [ (split)[@ia] ] }
                select_array "SHOW TABLE STATUS FROM \\\`$db\\`"

```

```

    );
}

my @dbnames = keys %tblist;

foreach my $db (@dbnames) {
    foreach my $tbl ( @{ $tblist{$db} } ) {
        my ( $name, $autoincrement ) = @$tbl;

        if ( $autoincrement =~ /\^d+?$/ ) {
            my $percent = percentage( $autoincrement, $maxint );
            $result{'PctAutoIncrement'}{"$db.$name"} = $percent;
            if ( $percent >= 75 ) {
                badprint
                "Table '$db.$name' has an autoincrement value near max capacity ($percent%)" ;
            }
        }
    }
}

}

my %mycalc;

sub calculations {
    if ( $mystat{'Questions'} < 1 ) {
        badprint
        "Your server has not answered any queries - cannot continue...";
        exit 2;
    }

    # Per-thread memory
    if ( mysql_version_ge(4) ) {
        $mycalc{'per_thread_buffers'} =
            $myvar{'read_buffer_size'} +
            $myvar{'read_rnd_buffer_size'} +
            $myvar{'sort_buffer_size'} +
            $myvar{'thread_stack'} +
            $myvar{'max_allowed_packet'} +
            $myvar{'join_buffer_size'};
    }
    else {
        $mycalc{'per_thread_buffers'} =
            $myvar{'record_buffer'} +
            $myvar{'record_rnd_buffer'} +
            $myvar{'sort_buffer'} +
            $myvar{'thread_stack'} +
            $myvar{'join_buffer_size'};
    }
}

```

```

$mycalc{'total_per_thread_buffers'} =
    $mycalc{'per_thread_buffers'} * $myvar{'max_connections'};
$mycalc{'max_total_per_thread_buffers'} =
    $mycalc{'per_thread_buffers'} * $mystat{'Max_used_connections'};

# Server-wide memory
$mycalc{'max_tmp_table_size'} =
    ( $myvar{'tmp_table_size'} > $myvar{'max_heap_table_size'} )
    ? $myvar{'max_heap_table_size'}
    : $myvar{'tmp_table_size'};
$mycalc{'server_buffers'} =
    $myvar{'key_buffer_size'} + $mycalc{'max_tmp_table_size'};
$mycalc{'server_buffers'} +=
    ( defined $myvar{'innodb_buffer_pool_size'} )
    ? $myvar{'innodb_buffer_pool_size'}
    : 0;
$mycalc{'server_buffers'} +=
    ( defined $myvar{'innodb_additional_mem_pool_size'} )
    ? $myvar{'innodb_additional_mem_pool_size'}
    : 0;
$mycalc{'server_buffers'} +=
    ( defined $myvar{'innodb_log_buffer_size'} )
    ? $myvar{'innodb_log_buffer_size'}
    : 0;
$mycalc{'server_buffers'} +=
    ( defined $myvar{'query_cache_size'} ) ? $myvar{'query_cache_size'} : 0;
$mycalc{'server_buffers'} +=
    ( defined $myvar{'aria_pagecache_buffer_size'} )
    ? $myvar{'aria_pagecache_buffer_size'}
    : 0;

# Global memory
# Max used memory is memory used by MySQL based on Max_used_connections
# This is the max memory used theoretically calculated with the max concurrent
connection number reached by mysql
$mycalc{'max_used_memory'} =
    $mycalc{'server_buffers'} +
    $mycalc{'max_total_per_thread_buffers'} +
    get_pf_memory();

# + get_gcache_memory();
$mycalc{'pct_max_used_memory'} =
    percentage( $mycalc{'max_used_memory'}, $physical_memory );

# Total possible memory is memory needed by MySQL based on max_connections
# This is the max memory MySQL can theoretically used if all connections
allowed has opened by mysql
$mycalc{'max_peak_memory'} =
    $mycalc{'server_buffers'} +

```

```

    $mycalc{'total_per_thread_buffers'} +
    get_pf_memory();

# + get_gcache_memory();
$mycalc{'pct_max_physical_memory'} =
    percentage( $mycalc{'max_peak_memory'}, $physical_memory );

debugprint "Max Used Memory: "
    . hr_bytes( $mycalc{'max_used_memory'} ) . "";
debugprint "Max Used Percentage RAM: "
    . $mycalc{'pct_max_used_memory'} . "%";

debugprint "Max Peak Memory: "
    . hr_bytes( $mycalc{'max_peak_memory'} ) . "";
debugprint "Max Peak Percentage RAM: "
    . $mycalc{'pct_max_physical_memory'} . "%";

# Slow queries
$mycalc{'pct_slow_queries'} =
    int( ( $mystat{'Slow_queries'} / $mystat{'Questions'} ) * 100 );

# Connections
$mycalc{'pct_connections_used'} = int(
    ( $mystat{'Max_used_connections'} / $myvar{'max_connections'} ) * 100 );
$mycalc{'pct_connections_used'} =
    ( $mycalc{'pct_connections_used'} > 100 )
    ? 100
    : $mycalc{'pct_connections_used'};

# Aborted Connections
$mycalc{'pct_connections_aborted'} =
    percentage( $mystat{'Aborted_connects'}, $mystat{'Connections'} );
debugprint "Aborted_connects: " . $mystat{'Aborted_connects'} . "";
debugprint "Connections: " . $mystat{'Connections'} . "";
debugprint "pct_connections_aborted: "
    . $mycalc{'pct_connections_aborted'} . "";

# Key buffers
if ( mysql_version_ge( 4, 1 ) && $myvar{'key_buffer_size'} > 0 ) {
    $mycalc{'pct_key_buffer_used'} = sprintf(
        "%.1f",
        (
            1 - (
                (
                    $mystat{'Key_blocks_unused'} *
                    $myvar{'key_cache_block_size'}
                ) / $myvar{'key_buffer_size'}
            )
        ) * 100
    );
}

```

```

    );
}
else {
    $mycalc{'pct_key_buffer_used'} = 0;
}

if ( $mystat{'Key_read_requests'} > 0 ) {
    $mycalc{'pct_keys_from_mem'} = sprintf(
        "%.1f",
        (
            100 - (
                ( $mystat{'Key_reads'} / $mystat{'Key_read_requests'} ) *
                100
            )
        )
    );
}
else {
    $mycalc{'pct_keys_from_mem'} = 0;
}
if ( defined $mystat{'Aria_pagecache_read_requests'}
    && $mystat{'Aria_pagecache_read_requests'} > 0 )
{
    $mycalc{'pct_aria_keys_from_mem'} = sprintf(
        "%.1f",
        (
            100 - (
                (
                    $mystat{'Aria_pagecache_reads'} /
                    $mystat{'Aria_pagecache_read_requests'}
                ) * 100
            )
        )
    );
}
else {
    $mycalc{'pct_aria_keys_from_mem'} = 0;
}

if ( $mystat{'Key_write_requests'} > 0 ) {
    $mycalc{'pct_wkeys_from_mem'} = sprintf( "%.1f",
        ( ( $mystat{'Key_writes'} / $mystat{'Key_write_requests'} ) * 100 )
    );
}
else {
    $mycalc{'pct_wkeys_from_mem'} = 0;
}

if ( $doremate eq 0 and !mysql_version_ge(5) ) {

```

```

    my $size = 0;
    $size += (split)[0]
    for
`find "$myvar{'datadir'}" -name "*.MYI" -print0 2>&1 | xargs $xargsflags -0 du
-L $duflags 2>&1`;
    $mycalc{'total_myisam_indexes'} = $size;
    $size = 0 + (split)[0]
    for
`find "$myvar{'datadir'}" -name "*.MAI" -print0 2>&1 | xargs $xargsflags -0 du
-L $duflags 2>&1`;
    $mycalc{'total_aria_indexes'} = $size;
}
elseif ( mysql_version_ge(5) ) {
    $mycalc{'total_myisam_indexes'} = select_one
"SELECT IFNULL(SUM(INDEX_LENGTH),0) FROM information_schema.TABLES WHERE
TABLE_SCHEMA NOT IN ('information_schema') AND ENGINE = 'MyISAM';";
    $mycalc{'total_aria_indexes'} = select_one
"SELECT IFNULL(SUM(INDEX_LENGTH),0) FROM information_schema.TABLES WHERE
TABLE_SCHEMA NOT IN ('information_schema') AND ENGINE = 'Aria';";
}
if ( defined $mycalc{'total_myisam_indexes'} ) {
    chomp( $mycalc{'total_myisam_indexes'} );
}
if ( defined $mycalc{'total_aria_indexes'} ) {
    chomp( $mycalc{'total_aria_indexes'} );
}

# Query cache
if ( mysql_version_ge(8) and mysql_version_le(10) ) {
    $mycalc{'query_cache_efficiency'} = 0;
}
elseif ( mysql_version_ge(4) ) {
    $mycalc{'query_cache_efficiency'} = sprintf(
        "%.1f",
        (
            $mystat{'Qcache_hits'} /
            ( $mystat{'Com_select'} + $mystat{'Qcache_hits'} )
        ) * 100
    );
    if ( $myvar{'query_cache_size'} ) {
        $mycalc{'pct_query_cache_used'} = sprintf(
            "%.1f",
            100 - (
                $mystat{'Qcache_free_memory'} / $myvar{'query_cache_size'}
            ) * 100
        );
    }
    if ( $mystat{'Qcache_lowmem_prunes'} == 0 ) {
        $mycalc{'query_cache_prunes_per_day'} = 0;
    }
}

```

```

    }
    else {
        $mycalc{'query_cache_prunes_per_day'} = int(
            $mystat{'Qcache_lowmem_prunes'} / ( $mystat{'Uptime'} / 86400 )
        );
    }
}

# Sorting
$mycalc{'total_sorts'} = $mystat{'Sort_scan'} + $mystat{'Sort_range'};
if ( $mycalc{'total_sorts'} > 0 ) {
    $mycalc{'pct_temp_sort_table'} = int(
        ( $mystat{'Sort_merge_passes'} / $mycalc{'total_sorts'} ) * 100 );
}

# Joins
$mycalc{'joins_without_indexes'} =
    $mystat{'Select_range_check'} + $mystat{'Select_full_join'};
$mycalc{'joins_without_indexes_per_day'} =
    int( $mycalc{'joins_without_indexes'} / ( $mystat{'Uptime'} / 86400 ) );

# Temporary tables
if ( $mystat{'Created_tmp_tables'} > 0 ) {
    if ( $mystat{'Created_tmp_disk_tables'} > 0 ) {
        $mycalc{'pct_temp_disk'} = int(
            (
                $mystat{'Created_tmp_disk_tables'} /
                $mystat{'Created_tmp_tables'}
            ) * 100
        );
    }
    else {
        $mycalc{'pct_temp_disk'} = 0;
    }
}

# Table cache
if ( $mystat{'Opened_tables'} > 0 ) {
    $mycalc{'table_cache_hit_rate'} =
        int( $mystat{'Open_tables'} * 100 / $mystat{'Opened_tables'} );
}
else {
    $mycalc{'table_cache_hit_rate'} = 100;
}

# Open files
if ( $myvar{'open_files_limit'} > 0 ) {
    $mycalc{'pct_files_open'} =
        int( $mystat{'Open_files'} * 100 / $myvar{'open_files_limit'} );
}

```

```

}

# Table locks
if ( $mystat{'Table_locks_immediate'} > 0 ) {
    if ( $mystat{'Table_locks_waited'} == 0 ) {
        $mycalc{'pct_table_locks_immediate'} = 100;
    }
    else {
        $mycalc{'pct_table_locks_immediate'} = int(
            $mystat{'Table_locks_immediate'} * 100 / (
                $mystat{'Table_locks_waited'} +
                $mystat{'Table_locks_immediate'}
            )
        );
    }
}

# Thread cache
$mycalc{'thread_cache_hit_rate'} =
    int( 100 -
        ( ( $mystat{'Threads_created'} / $mystat{'Connections'} ) * 100 ) );

# Other
if ( $mystat{'Connections'} > 0 ) {
    $mycalc{'pct_aborted_connections'} =
        int( ( $mystat{'Aborted_connects'} / $mystat{'Connections'} ) * 100 );
}
if ( $mystat{'Questions'} > 0 ) {
    $mycalc{'total_reads'} = $mystat{'Com_select'};
    $mycalc{'total_writes'} =
        $mystat{'Com_delete'} +
        $mystat{'Com_insert'} +
        $mystat{'Com_update'} +
        $mystat{'Com_replace'};
    if ( $mycalc{'total_reads'} == 0 ) {
        $mycalc{'pct_reads'} = 0;
        $mycalc{'pct_writes'} = 100;
    }
    else {
        $mycalc{'pct_reads'} = int(
            (
                $mycalc{'total_reads'} /
                ( $mycalc{'total_reads'} + $mycalc{'total_writes'} )
            ) * 100
        );
        $mycalc{'pct_writes'} = 100 - $mycalc{'pct_reads'};
    }
}
}

```

```

# InnoDB
if ( $myvar{'have_innodb'} eq "YES" ) {
    $mycalc{'innodb_log_size_pct'} =
        ( $myvar{'innodb_log_file_size'} *
          $myvar{'innodb_log_files_in_group'} * 100 /
          $myvar{'innodb_buffer_pool_size'} );
}
if ( !defined $myvar{'innodb_buffer_pool_size'} ) {
    $mycalc{'innodb_log_size_pct'} = 0;
    $myvar{'innodb_buffer_pool_size'} = 0;
}

# InnoDB Buffer pool read cache efficiency
(
    $mystat{'Innodb_buffer_pool_read_requests'},
    $mystat{'Innodb_buffer_pool_reads'}
)
= ( 1, 1 )
unless defined $mystat{'Innodb_buffer_pool_reads'};
$mycalc{'pct_read_efficiency'} = percentage(
    (
        $mystat{'Innodb_buffer_pool_read_requests'} -
        $mystat{'Innodb_buffer_pool_reads'}
    ),
    $mystat{'Innodb_buffer_pool_read_requests'}
) if defined $mystat{'Innodb_buffer_pool_read_requests'};
debugprint "pct_read_efficiency: " . $mycalc{'pct_read_efficiency'} . "";
debugprint "Innodb_buffer_pool_reads: "
    . $mystat{'Innodb_buffer_pool_reads'} . "";
debugprint "Innodb_buffer_pool_read_requests: "
    . $mystat{'Innodb_buffer_pool_read_requests'} . "";

# InnoDB log write cache efficiency
( $mystat{'Innodb_log_write_requests'}, $mystat{'Innodb_log_writes'} ) =
    ( 1, 1 )
unless defined $mystat{'Innodb_log_writes'};
$mycalc{'pct_write_efficiency'} = percentage(
    ( $mystat{'Innodb_log_write_requests'} - $mystat{'Innodb_log_writes'} ),
    $mystat{'Innodb_log_write_requests'}
) if defined $mystat{'Innodb_log_write_requests'};
debugprint "pct_write_efficiency: " . $mycalc{'pct_write_efficiency'} . "";
debugprint "Innodb_log_writes: " . $mystat{'Innodb_log_writes'} . "";
debugprint "Innodb_log_write_requests: "
    . $mystat{'Innodb_log_write_requests'} . "";
$mycalc{'pct_innodb_buffer_used'} = percentage(
    (
        $mystat{'Innodb_buffer_pool_pages_total'} -
        $mystat{'Innodb_buffer_pool_pages_free'}
    ),

```

```

    $mystat{'Innodb_buffer_pool_pages_total'}
) if defined $mystat{'Innodb_buffer_pool_pages_total'};

# Binlog Cache
if ( $myvar{'log_bin'} ne 'OFF' ) {
    $mycalc{'pct_binlog_cache'} = percentage(
        $mystat{'Binlog_cache_use'} - $mystat{'Binlog_cache_disk_use'},
        $mystat{'Binlog_cache_use'} );
}
}

sub mysql_stats {
    subheaderprint "Performance Metrics";

    # Show uptime, queries per second, connections, traffic stats
    my $qps;
    if ( $mystat{'Uptime'} > 0 ) {
        $qps = sprintf( "%.3f", $mystat{'Questions'} / $mystat{'Uptime'} );
    }
    push( @generalrec,
        "MySQL was started within the last 24 hours - recommendations may be
        inaccurate"
    ) if ( $mystat{'Uptime'} < 86400 );
    infoprint "Up for: "
        . pretty_uptime( $mystat{'Uptime'} ) . " ("
        . hr_num( $mystat{'Questions'} ) . " q ["
        . hr_num($qps)
        . " qps], "
        . hr_num( $mystat{'Connections'} )
        . " conn," . " TX: "
        . hr_bytes_rnd( $mystat{'Bytes_sent'} )
        . ", RX: "
        . hr_bytes_rnd( $mystat{'Bytes_received'} ) . " )";
    infoprint "Reads / Writes: "
        . $mycalc{'pct_reads'} . "% / "
        . $mycalc{'pct_writes'} . "%";

    # Binlog Cache
    if ( $myvar{'log_bin'} eq 'OFF' ) {
        infoprint "Binary logging is disabled";
    }
    else {
        infoprint "Binary logging is enabled (GTID MODE: "
            . ( defined( $myvar{'gtid_mode'} ) ? $myvar{'gtid_mode'} : "OFF" )
            . " )";
    }

    # Memory usage
    infoprint "Physical Memory      : " . hr_bytes($physical_memory);

```

```

infoprint "Max MySQL memory      : " . hr_bytes( $mycalc{'max_peak_memory'} );
infoprint "Other process memory: " . hr_bytes( get_other_process_memory() );

infoprint "Total buffers: "
    . hr_bytes( $mycalc{'server_buffers'} )
    . " global + "
    . hr_bytes( $mycalc{'per_thread_buffers'} )
    . " per thread ($myvar{'max_connections'} max threads)";
infoprint "P_S Max memory usage: " . hr_bytes_rnd( get_pf_memory() );
$result{'P_S'}{'memory'} = get_pf_memory();
$result{'P_S'}{'pretty_memory'} =
    hr_bytes_rnd( get_pf_memory() );
infoprint "Galera GCache Max memory usage: "
    . hr_bytes_rnd( get_gcache_memory() );
$result{'Galera'}{'GCache'}{'memory'} = get_gcache_memory();
$result{'Galera'}{'GCache'}{'pretty_memory'} =
    hr_bytes_rnd( get_gcache_memory() );

if ( $opt{buffers} ne 0 ) {
    infoprint "Global Buffers";
    infoprint " +-- Key Buffer: "
        . hr_bytes( $myvar{'key_buffer_size'} ) . "";
    infoprint " +-- Max Tmp Table: "
        . hr_bytes( $mycalc{'max_tmp_table_size'} ) . "";

    if ( defined $myvar{'query_cache_type'} ) {
        infoprint "Query Cache Buffers";
        infoprint " +-- Query Cache: "
            . $myvar{'query_cache_type'} . " - "
            . (
                $myvar{'query_cache_type'} eq 0 |
                $myvar{'query_cache_type'} eq 'OFF' ? "DISABLED"
                : (
                    $myvar{'query_cache_type'} eq 1 ? "ALL REQUESTS"
                    : "ON DEMAND"
                )
            ) . "";
        infoprint " +-- Query Cache Size: "
            . hr_bytes( $myvar{'query_cache_size'} ) . "";
    }

    infoprint "Per Thread Buffers";
    infoprint " +-- Read Buffer: "
        . hr_bytes( $myvar{'read_buffer_size'} ) . "";
    infoprint " +-- Read RND Buffer: "
        . hr_bytes( $myvar{'read_rnd_buffer_size'} ) . "";
    infoprint " +-- Sort Buffer: "
        . hr_bytes( $myvar{'sort_buffer_size'} ) . "";
    infoprint " +-- Thread stack: "

```

```

        . hr_bytes( $myvar{'thread_stack'} ) . "";
infoprint " +-- Join Buffer: "
        . hr_bytes( $myvar{'join_buffer_size'} ) . "";
if ( $myvar{'log_bin'} ne 'OFF' ) {
    infoprint "Binlog Cache Buffers";
    infoprint " +-- Binlog Cache: "
        . hr_bytes( $myvar{'binlog_cache_size'} ) . "";
}
}

if ( $sarch
    && $sarch == 32
    && $mycalc{'max_used_memory'} > 2 * 1024 * 1024 * 1024 )
{
    badprint
        "Allocating > 2GB RAM on 32-bit systems can cause system instability";
    badprint "Maximum reached memory usage: "
        . hr_bytes( $mycalc{'max_used_memory'} )
        . " ($mycalc{'pct_max_used_memory'}% of installed RAM)";
}
elseif ( $mycalc{'pct_max_used_memory'} > 85 ) {
    badprint "Maximum reached memory usage: "
        . hr_bytes( $mycalc{'max_used_memory'} )
        . " ($mycalc{'pct_max_used_memory'}% of installed RAM)";
}
else {
    goodprint "Maximum reached memory usage: "
        . hr_bytes( $mycalc{'max_used_memory'} )
        . " ($mycalc{'pct_max_used_memory'}% of installed RAM)";
}

if ( $mycalc{'pct_max_physical_memory'} > 85 ) {
    badprint "Maximum possible memory usage: "
        . hr_bytes( $mycalc{'max_peak_memory'} )
        . " ($mycalc{'pct_max_physical_memory'}% of installed RAM)";
    push( @generalrec,
        "Reduce your overall MySQL memory footprint for system stability" );
}
else {
    goodprint "Maximum possible memory usage: "
        . hr_bytes( $mycalc{'max_peak_memory'} )
        . " ($mycalc{'pct_max_physical_memory'}% of installed RAM)";
}

if ( $physical_memory <
    ( $mycalc{'max_peak_memory'} + get_other_process_memory() ) )
{
    badprint
        "Overall possible memory usage with other process exceeded memory";
}

```

```

        push( @generalrec,
            "Dedicate this server to your database for highest performance." );
    }
    else {
        goodprint
"Overall possible memory usage with other process is compatible with memory
available";
    }

# Slow queries
if ( $mycalc{'pct_slow_queries'} > 5 ) {
    badprint "Slow queries: $mycalc{'pct_slow_queries'}% ("
        . hr_num( $mystat{'Slow_queries'} ) . "/"
        . hr_num( $mystat{'Questions'} ) . ")";
}
else {
    goodprint "Slow queries: $mycalc{'pct_slow_queries'}% ("
        . hr_num( $mystat{'Slow_queries'} ) . "/"
        . hr_num( $mystat{'Questions'} ) . ")";
}
if ( $myvar{'long_query_time'} > 10 ) {
    push( @adjvars, "long_query_time (<= 10)" );
}
if ( defined( $myvar{'log_slow_queries'} ) ) {
    if ( $myvar{'log_slow_queries'} eq "OFF" ) {
        push( @generalrec,
            "Enable the slow query log to troubleshoot bad queries" );
    }
}

# Connections
if ( $mycalc{'pct_connections_used'} > 85 ) {
    badprint
"Highest connection usage: $mycalc{'pct_connections_used'}%
($mystat{'Max_used_connections'}/$myvar{'max_connections'})";
    push( @adjvars,
        "max_connections (> " . $myvar{'max_connections'} . ")" );
    push( @adjvars,
        "wait_timeout (< " . $myvar{'wait_timeout'} . ")",
        "interactive_timeout (< " . $myvar{'interactive_timeout'} . ")" );
    push( @generalrec,
"Reduce or eliminate persistent connections to reduce connection usage"
    );
}
else {
    goodprint
"Highest usage of available connections: $mycalc{'pct_connections_used'}%
($mystat{'Max_used_connections'}/$myvar{'max_connections'})";
}

```

```

# Aborted Connections
if ( $mycalc{'pct_connections_aborted'} > 3 ) {
    badprint
    "Aborted connections: $mycalc{'pct_connections_aborted'}%
($mystat{'Aborted_connects'}/$mystat{'Connections'})";
    push( @generalrec,
        "Reduce or eliminate unclosed connections and network issues" );
}
else {
    goodprint
    "Aborted connections: $mycalc{'pct_connections_aborted'}%
($mystat{'Aborted_connects'}/$mystat{'Connections'})";
}

# name resolution
if ( defined( $result{'Variables'}{'skip_networking'} )
    && $result{'Variables'}{'skip_networking'} eq 'ON' )
{
    infoprint
    "Skipped name resolution test due to skip_networking=ON in system variables.";
}
elseif ( not defined( $result{'Variables'}{'skip_name_resolve'} ) ) {
    infoprint
    "Skipped name resolution test due to missing skip_name_resolve in system
variables.";
}
elseif ( $result{'Variables'}{'skip_name_resolve'} eq 'OFF' ) {
    badprint
    "name resolution is active : a reverse name resolution is made for each new
connection and can reduce performance";
    push( @generalrec,
        "Configure your accounts with ip or subnets only, then update your
configuration with skip-name-resolve=1"
    );
}

# Query cache
if ( !mysql_version_ge(4) ) {

    # MySQL versions < 4.01 don't support query caching
    push( @generalrec,
        "Upgrade MySQL to version 4+ to utilize query caching" );
}
elseif ( mysql_version_eq(8) ) {
    infoprint "Query cache have been removed in MySQL 8";

    #return;
}

```

```

elseif ($myvar{'query_cache_size'} < 1
        or $myvar{'query_cache_type'} eq "OFF" )
{
    goodprint
    "Query cache is disabled by default due to mutex contention on multiprocessor
    machines.";
}
elseif ( $mystat{'Com_select'} == 0 ) {
    badprint
    "Query cache cannot be analyzed - no SELECT statements executed";
}
else {
    badprint
    "Query cache may be disabled by default due to mutex contention.";
    push( @adjvars, "query_cache_size (=0)" );
    push( @adjvars, "query_cache_type (=0)" );
    if ( $mycalc{'query_cache_efficiency'} < 20 ) {
        badprint
        "Query cache efficiency: $mycalc{'query_cache_efficiency'}% ( "
        . hr_num( $mystat{'Qcache_hits'} )
        . " cached / "
        . hr_num( $mystat{'Qcache_hits'} + $mystat{'Com_select'} )
        . " selects)";
        push( @adjvars,
            "query_cache_limit (> "
            . hr_bytes_rnd( $myvar{'query_cache_limit'} )
            . ", or use smaller result sets)" );
    }
    else {
        goodprint
        "Query cache efficiency: $mycalc{'query_cache_efficiency'}% ( "
        . hr_num( $mystat{'Qcache_hits'} )
        . " cached / "
        . hr_num( $mystat{'Qcache_hits'} + $mystat{'Com_select'} )
        . " selects)";
    }
    if ( $mycalc{'query_cache_prunes_per_day'} > 98 ) {
        badprint
        "Query cache prunes per day: $mycalc{'query_cache_prunes_per_day'}";
        if ( $myvar{'query_cache_size'} >= 128 * 1024 * 1024 ) {
            push( @generalrec,
                "Increasing the query_cache size over 128M may reduce performance"
            );
            push( @adjvars,
                "query_cache_size (> "
                . hr_bytes_rnd( $myvar{'query_cache_size'} )
                . ") [see warning above]" );
        }
    }
    else {

```

```

        push( @adjvars,
            "query_cache_size (> "
            . hr_bytes_rnd( $myvar{'query_cache_size'} )
            . ")" );
    }
}
else {
    goodprint
    "Query cache prunes per day: $mycalc{'query_cache_prunes_per_day'}";
}
}

# Sorting
if ( $mycalc{'total_sorts'} == 0 ) {
    goodprint "No Sort requiring temporary tables";
}
elseif ( $mycalc{'pct_temp_sort_table'} > 10 ) {
    badprint
    "Sorts requiring temporary tables: $mycalc{'pct_temp_sort_table'}% ("
    . hr_num( $mystat{'Sort_merge_passes'} )
    . " temp sorts / "
    . hr_num( $mycalc{'total_sorts'} )
    . " sorts)";
    push( @adjvars,
        "sort_buffer_size (> "
        . hr_bytes_rnd( $myvar{'sort_buffer_size'} )
        . ")" );
    push( @adjvars,
        "read_rnd_buffer_size (> "
        . hr_bytes_rnd( $myvar{'read_rnd_buffer_size'} )
        . ")" );
}
else {
    goodprint
    "Sorts requiring temporary tables: $mycalc{'pct_temp_sort_table'}% ("
    . hr_num( $mystat{'Sort_merge_passes'} )
    . " temp sorts / "
    . hr_num( $mycalc{'total_sorts'} )
    . " sorts)";
}

# Joins
if ( $mycalc{'joins_without_indexes_per_day'} > 250 ) {
    badprint
    "Joins performed without indexes: $mycalc{'joins_without_indexes'}";
    push( @adjvars,
        "join_buffer_size (> "
        . hr_bytes( $myvar{'join_buffer_size'} )
        . ", or always use indexes with JOINS)" );
}

```

```

        push(
            @generalrec,
            "We will suggest raising the 'join_buffer_size' until JOINS not using indexes
            are found.

            See https://dev.mysql.com/doc/internals/en/join-buffer-size.html
            (specially the conclusions at the bottom of the page)."
        );
    }
    else {
        goodprint "No joins without indexes";

        # No joins have run without indexes
    }

    # Temporary tables
    if ( $mystat{'Created_tmp_tables'} > 0 ) {
        if ( $mycalc{'pct_temp_disk'} > 25
            && $mycalc{'max_tmp_table_size'} < 256 * 1024 * 1024 )
        {
            badprint
                "Temporary tables created on disk: $mycalc{'pct_temp_disk'}% ("
                . hr_num( $mystat{'Created_tmp_disk_tables'} )
                . " on disk / "
                . hr_num( $mystat{'Created_tmp_tables'} )
                . " total)";
            push( @adjvars,
                "tmp_table_size (> "
                . hr_bytes_rnd( $myvar{'tmp_table_size'} )
                . ")" );
            push( @adjvars,
                "max_heap_table_size (> "
                . hr_bytes_rnd( $myvar{'max_heap_table_size'} )
                . ")" );
            push( @generalrec,
                "When making adjustments, make tmp_table_size/max_heap_table_size equal"
            );
            push( @generalrec,
                "Reduce your SELECT DISTINCT queries which have no LIMIT clause"
            );
        }
        elsif ( $mycalc{'pct_temp_disk'} > 25
            && $mycalc{'max_tmp_table_size'} >= 256 * 1024 * 1024 )
        {
            badprint
                "Temporary tables created on disk: $mycalc{'pct_temp_disk'}% ("
                . hr_num( $mystat{'Created_tmp_disk_tables'} )
                . " on disk / "
                . hr_num( $mystat{'Created_tmp_tables'} )
                . " total)";

```

```

        push( @generalrec,
            "Temporary table size is already large - reduce result set size"
        );
        push( @generalrec,
            "Reduce your SELECT DISTINCT queries without LIMIT clauses" );
    }
    else {
        goodprint
            "Temporary tables created on disk: $mycalc{'pct_temp_disk'}% ("
            . hr_num( $mystat{'Created_tmp_disk_tables'} )
            . " on disk / "
            . hr_num( $mystat{'Created_tmp_tables'} )
            . " total)";
    }
}
else {
    goodprint "No tmp tables created on disk";
}

# Thread cache
if ( defined( $myvar{'thread_handling'} )
    and $myvar{'thread_handling'} eq 'pool-of-threads' )
{
#
https://www.percona.com/doc/percona-server/LATEST/performance/threadpool.html
# When thread pool is enabled, the value of the thread_cache_size variable
# is ignored. The Threads_cached status variable contains 0 in this case.
    infoprint "Thread cache not used with thread_handling=pool-of-threads";
}
else {
    if ( $myvar{'thread_cache_size'} eq 0 ) {
        badprint "Thread cache is disabled";
        push( @generalrec,
            "Set thread_cache_size to 4 as a starting value" );
        push( @adjvars, "thread_cache_size (start at 4)" );
    }
    else {
        if ( $mycalc{'thread_cache_hit_rate'} <= 50 ) {
            badprint
                "Thread cache hit rate: $mycalc{'thread_cache_hit_rate'}% ("
                . hr_num( $mystat{'Threads_created'} )
                . " created / "
                . hr_num( $mystat{'Connections'} )
                . " connections)";
            push( @adjvars,
                "thread_cache_size (> $myvar{'thread_cache_size'})" );
        }
        else {
            goodprint

```

```

        "Thread cache hit rate: $mycalc{'thread_cache_hit_rate'}% ("
        . hr_num( $mystat{'Threads_created'} )
        . " created / "
        . hr_num( $mystat{'Connections'} )
        . " connections)";
    }
}

}

# Table cache
my $table_cache_var = "";
if ( $mystat{'Open_tables'} > 0 ) {
    if ( $mycalc{'table_cache_hit_rate'} < 20 ) {
        badprint "Table cache hit rate: $mycalc{'table_cache_hit_rate'}% ("
            . hr_num( $mystat{'Open_tables'} )
            . " open / "
            . hr_num( $mystat{'Opened_tables'} )
            . " opened)";
        if ( mysql_version_ge( 5, 1 ) ) {
            $table_cache_var = "table_open_cache";
        }
        else {
            $table_cache_var = "table_cache";
        }
    }

    push( @adjvars,
        $table_cache_var . " (> " . $myvar{$table_cache_var} . ")" );
    push( @generalrec,
        "Increase "
        . $table_cache_var
        . " gradually to avoid file descriptor limits" );
    push( @generalrec,
        "Read this before increasing "
        . $table_cache_var
        . " over 64: https://bit.ly/2Fulv7r" );
    push( @generalrec,
        "Read this before increasing for MariaDB"
        . "
https://mariadb.com/kb/en/library/optimizing-table\_open\_cache/
");
    push( @generalrec,
        "This is MyISAM only table_cache scalability problem, InnoDB not affected."
    );
    push( @generalrec,
        "See more details here: https://bugs.mysql.com/bug.php?id=49177"
    );
    push( @generalrec,
        "This bug already fixed in MySQL 5.7.9 and newer MySQL versions."
    );
}

```

```

        push( @generalrec,
            "Beware that open_files_limit ("
            . $myvar{'open_files_limit'}
            . ") variable " );
        push( @generalrec,
            "should be greater than $table_cache_var ("
            . $myvar{$table_cache_var}
            . ")" );
    }
    else {
        goodprint "Table cache hit rate: $mycalc{'table_cache_hit_rate'}% ("
            . hr_num( $mystat{'Open_tables'} )
            . " open / "
            . hr_num( $mystat{'Opened_tables'} )
            . " opened)";
    }
}

# Table definition cache
my $nbttables = select_one('SELECT COUNT(*) FROM information_schema.tables');
$mycalc{'total_tables'} = $nbttables;
if ( defined $myvar{'table_definition_cache'} ) {
    if ( $myvar{'table_definition_cache'} == -1 ) {
        infoprint( "table_definition_cache("
            . $myvar{'table_definition_cache'}
            . ") is in autosizing mode" );
    }
    elsif ( $myvar{'table_definition_cache'} < $nbttables ) {
        badprint "table_definition_cache("
            . $myvar{'table_definition_cache'}
            . ") is lower than number of tables($nbttables) ";
        push( @adjvars,
            "table_definition_cache("
            . $myvar{'table_definition_cache'} . ") > "
            . $nbttables
            . " or -1 (autosizing if supported)" );
    }
    else {
        goodprint "table_definition_cache("
            . $myvar{'table_definition_cache'}
            . ") is upper than number of tables($nbttables)";
    }
}
else {
    infoprint "No table_definition_cache variable found.";
}

# Open files
if ( defined $mycalc{'pct_files_open'} ) {

```

```

if ( $mycalc{'pct_files_open'} > 85 ) {
    badprint "Open file limit used: $mycalc{'pct_files_open'}% ("
        . hr_num( $mystat{'Open_files'} ) . "/"
        . hr_num( $myvar{'open_files_limit'} ) . ")";
    push( @adjvars,
        "open_files_limit (> " . $myvar{'open_files_limit'} . ")" );
}
else {
    goodprint "Open file limit used: $mycalc{'pct_files_open'}% ("
        . hr_num( $mystat{'Open_files'} ) . "/"
        . hr_num( $myvar{'open_files_limit'} ) . ")";
}
}

# Table locks
if ( defined $mycalc{'pct_table_locks_immediate'} ) {
    if ( $mycalc{'pct_table_locks_immediate'} < 95 ) {
        badprint
"Table locks acquired immediately: $mycalc{'pct_table_locks_immediate'}%";
        push( @generalrec,
            "Optimize queries and/or use InnoDB to reduce lock wait" );
    }
    else {
        goodprint
"Table locks acquired immediately: $mycalc{'pct_table_locks_immediate'}% ("
        . hr_num( $mystat{'Table_locks_immediate'} )
        . " immediate / "
        . hr_num( $mystat{'Table_locks_waited'} +
            $mystat{'Table_locks_immediate'} )
        . " locks)";
    }
}

# Binlog cache
if ( defined $mycalc{'pct_binlog_cache'} ) {
    if ( $mycalc{'pct_binlog_cache'} < 90
        && $mystat{'Binlog_cache_use'} > 0 )
    {
        badprint "Binlog cache memory access: "
        . $mycalc{'pct_binlog_cache'} . "% ("
        . (
            $mystat{'Binlog_cache_use'} - $mystat{'Binlog_cache_disk_use'} )
        . " Memory / "
        . $mystat{'Binlog_cache_use'}
        . " Total)";
        push( @generalrec,
            "Increase binlog_cache_size (Actual value: "
            . $myvar{'binlog_cache_size'}
            . ")" );
    }
}

```

```

        push( @adjvars,
            "binlog_cache_size ("
            . hr_bytes( $myvar{'binlog_cache_size'} + 16 * 1024 * 1024 )
            . ")" );
    }
    else {
        goodprint "Binlog cache memory access: "
            . $mycalc{'pct_binlog_cache'} . "% ("
            . (
                $mystat{'Binlog_cache_use'} - $mystat{'Binlog_cache_disk_use'} )
            . " Memory / "
            . $mystat{'Binlog_cache_use'}
            . " Total)";
        debugprint "Not enough data to validate binlog cache size\n"
            if $mystat{'Binlog_cache_use'} < 10;
    }
}

# Performance options
if ( !mysql_version_ge( 5, 1 ) ) {
    push( @generalrec, "Upgrade to MySQL 5.5+ to use asynchronous write" );
}
elseif ( $myvar{'concurrent_insert'} eq "OFF" ) {
    push( @generalrec, "Enable concurrent_insert by setting it to 'ON'" );
}
elseif ( $myvar{'concurrent_insert'} eq 0 ) {
    push( @generalrec, "Enable concurrent_insert by setting it to 1" );
}
}

# Recommendations for MyISAM
sub mysql_myisam {
    subheaderprint "MyISAM Metrics";
    if ( mysql_version_ge(8) and mysql_version_le(10) ) {
        infoprint "MyISAM Metrics are disabled on last MySQL versions.";
        return;
    }
}

# Key buffer usage
if ( defined( $mycalc{'pct_key_buffer_used'} ) ) {
    if ( $mycalc{'pct_key_buffer_used'} < 90 ) {
        badprint "Key buffer used: $mycalc{'pct_key_buffer_used'}% ("
            . hr_num( $myvar{'key_buffer_size'} *
                $mycalc{'pct_key_buffer_used'} /
                100 )
            . " used / "
            . hr_num( $myvar{'key_buffer_size'} )
            . " cache)";
    }
}

```

```

#push(@adjvars,"key_buffer_size (\~ ".hr_num( $myvar{'key_buffer_size'} *
$mycalc{'pct_key_buffer_used'} / 100).")");
}
else {
    goodprint "Key buffer used: $mycalc{'pct_key_buffer_used'}% ("
        . hr_num( $myvar{'key_buffer_size'} *
            $mycalc{'pct_key_buffer_used'} /
                100 )
        . " used / "
        . hr_num( $myvar{'key_buffer_size'} )
        . " cache)";
}
}
else {

    # No queries have run that would use keys
    debugprint "Key buffer used: $mycalc{'pct_key_buffer_used'}% ("
        . hr_num(
            $myvar{'key_buffer_size'} * $mycalc{'pct_key_buffer_used'} / 100 )
        . " used / "
        . hr_num( $myvar{'key_buffer_size'} )
        . " cache)";
}

# Key buffer
if ( !defined( $mycalc{'total_myisam_indexes'} ) ) {
    push( @generalrec,
        "Unable to calculate MyISAM index size on MySQL server < 5.0.0" );
}
else {
    if ( $myvar{'key_buffer_size'} < $mycalc{'total_myisam_indexes'}
        && $mycalc{'pct_keys_from_mem'} < 95 )
    {
        badprint "Key buffer size / total MyISAM indexes: "
            . hr_bytes( $myvar{'key_buffer_size'} ) . "/"
            . hr_bytes( $mycalc{'total_myisam_indexes'} ) . "";
        push( @adjvars,
            "key_buffer_size (> "
            . hr_bytes( $mycalc{'total_myisam_indexes'} )
            . ")" );
    }
    else {
        goodprint "Key buffer size / total MyISAM indexes: "
            . hr_bytes( $myvar{'key_buffer_size'} ) . "/"
            . hr_bytes( $mycalc{'total_myisam_indexes'} ) . "";
    }
    if ( $mystat{'Key_read_requests'} > 0 ) {
        if ( $mycalc{'pct_keys_from_mem'} < 95 ) {
            badprint

```

```

        "Read Key buffer hit rate: $mycalc{'pct_keys_from_mem'}% ("
        . hr_num( $mystat{'Key_read_requests'} )
        . " cached / "
        . hr_num( $mystat{'Key_reads'} )
        . " reads)";
    }
else {
    goodprint
        "Read Key buffer hit rate: $mycalc{'pct_keys_from_mem'}% ("
        . hr_num( $mystat{'Key_read_requests'} )
        . " cached / "
        . hr_num( $mystat{'Key_reads'} )
        . " reads)";
    }
}
else {

    # No queries have run that would use keys
    debugprint "Key buffer size / total MyISAM indexes: "
        . hr_bytes( $myvar{'key_buffer_size'} ) . "/"
        . hr_bytes( $mycalc{'total_myisam_indexes'} ) . "";
}
if ( $mystat{'Key_write_requests'} > 0 ) {
    if ( $mycalc{'pct_wkeys_from_mem'} < 95 ) {
        badprint
            "Write Key buffer hit rate: $mycalc{'pct_wkeys_from_mem'}% ("
            . hr_num( $mystat{'Key_write_requests'} )
            . " cached / "
            . hr_num( $mystat{'Key_writes'} )
            . " writes)";
    }
    else {
        goodprint
            "Write Key buffer hit rate: $mycalc{'pct_wkeys_from_mem'}% ("
            . hr_num( $mystat{'Key_write_requests'} )
            . " cached / "
            . hr_num( $mystat{'Key_writes'} )
            . " writes)";
    }
}
else {

    # No queries have run that would use keys
    debugprint
        "Write Key buffer hit rate: $mycalc{'pct_wkeys_from_mem'}% ("
        . hr_num( $mystat{'Key_write_requests'} )
        . " cached / "
        . hr_num( $mystat{'Key_writes'} )
        . " writes)";
}

```

```

    }
}

}

# Recommendations for ThreadPool
sub mariadb_threadpool {
    subheaderprint "ThreadPool Metrics";

    # AriaDB
    unless ( defined $myvar{'have_threadpool'}
        && $myvar{'have_threadpool'} eq "YES" )
    {
        infoprint "ThreadPool stat is disabled.";
        return;
    }
    infoprint "ThreadPool stat is enabled.";
    infoprint "Thread Pool Size: " . $myvar{'thread_pool_size'} . " thread(s).";

    if ( $myvar{'version'} =~ /mariadb|percona/i ) {
        infoprint "Using default value is good enough for your version ("
            . $myvar{'version'} . ")";
        return;
    }

    if ( $myvar{'have_innodb'} eq 'YES' ) {
        if ( $myvar{'thread_pool_size'} < 16
            or $myvar{'thread_pool_size'} > 36 )
        {
            badprint
"thread_pool_size between 16 and 36 when using InnoDB storage engine.";
            push( @generalrec,
                "Thread pool size for InnoDB usage ("
                    . $myvar{'thread_pool_size'}
                    . ")" );
            push( @adjvars,
                "thread_pool_size between 16 and 36 for InnoDB usage" );
        }
        else {
            goodprint
"thread_pool_size between 16 and 36 when using InnoDB storage engine.";
        }
        return;
    }

    if ( $myvar{'have_isam'} eq 'YES' ) {
        if ( $myvar{'thread_pool_size'} < 4 or $myvar{'thread_pool_size'} > 8 )
        {
            badprint
"thread_pool_size between 4 and 8 when using MyIsam storage engine.";
            push( @generalrec,

```

```

        "Thread pool size for MyIsam usage ("
        . $myvar{'thread_pool_size'}
        . ")" );
    push( @adjvars,
        "thread_pool_size between 4 and 8 for MyIsam usage" );
    }
    else {
        goodprint
        "thread_pool_size between 4 and 8 when using MyISAM storage engine.";
    }
}

}

sub get_pf_memory {

    # Performance Schema
    return 0 unless defined $myvar{'performance_schema'};
    return 0 if $myvar{'performance_schema'} eq 'OFF';

    my @infoPFSMemory = grep /performance_schema.memory/,
        select_array("SHOW ENGINE PERFORMANCE_SCHEMA STATUS");
    return 0 if scalar(@infoPFSMemory) == 0;
    $infoPFSMemory[0] =~ s/.*\s+(\d+)\$/$1/g;
    return $infoPFSMemory[0];
}

# Recommendations for Performance Schema
sub mysql_pfs {
    subheaderprint "Performance schema";

    # Performance Schema
    $myvar{'performance_schema'} = 'OFF'
    unless defined( $myvar{'performance_schema'} );
    unless ( $myvar{'performance_schema'} eq 'ON' ) {
        infoprint "Performance schema is disabled.";
        if ( mysql_version_ge( 5, 6 ) ) {
            push( @generalrec,
                "Performance schema should be activated for better diagnostics"
            );
            push( @adjvars, "performance_schema = ON enable PFS" );
        }
    }
    else {
        if ( mysql_version_le( 5, 5 ) ) {
            push( @generalrec,
                "Performance schema shouldn't be activated for MySQL and MariaDB 5.5 and lower
                version"
            );
            push( @adjvars, "performance_schema = OFF disable PFS" );
        }
    }
}

```

```

    }
}
debugprint "Performance schema is " . $myvar{'performance_schema'};
infoprint "Memory used by P_S: " . hr_bytes( get_pf_memory() );

if ( mysql_version_eq( 10, 0 ) ) {
    push( @generalrec,
"Performance schema shouldn't be activated for MariaDB 10.0 for performance
issue"
    );
    push( @adjvars, "performance_schema = OFF disable PFS" );
    return;
}
unless ( grep /^sys$/, select_array("SHOW DATABASES") ) {
    infoprint "Sys schema isn't installed.";
    push( @generalrec,
"Consider installing Sys schema from https://github.com/mysql/mysql-sys for
MySQL"
    ) unless ( mysql_version_le( 5, 6 ) );
    push( @generalrec,
"Consider installing Sys schema from https://github.com/FromDual/mariadb-sys
for MariaDB"
    ) unless ( mysql_version_eq( 10, 0 ) or mysql_version_eq( 5, 5 ) );

    return;
}
else {
    infoprint "Sys schema is installed.";
}
return if ( $opt{pfstat} == 0 or $myvar{'performance_schema'} ne 'ON' );

infoprint "Sys schema Version: "
    . select_one("select sys_version from sys.version");

# Top user per connection
subheaderprint "Performance schema: Top 5 user per connection";
my $nbL = 1;
for my $lQuery (
    select_array(
'select user, total_connections from sys.user_summary order by
total_connections desc LIMIT 5'
    )
)
{
    infoprint " +-- $nbL: $lQuery conn(s)";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

```

```

# Top user per statement
subheaderprint "Performance schema: Top 5 user per statement";
$nbL = 1;
for my $lQuery (
    select_array(
'select user, statements from sys.user_summary order by statements desc LIMIT
5'
    )
)
{
    infoprint " +-- $nbL: $lQuery stmt(s)";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
if ( $nbL == 1 );

# Top user per statement latency
subheaderprint "Performance schema: Top 5 user per statement latency";
$nbL = 1;
for my $lQuery (
    select_array(
'select user, statement_avg_latency from sys.x\\$user_summary order by
statement_avg_latency desc LIMIT 5'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
if ( $nbL == 1 );

# Top user per lock latency
subheaderprint "Performance schema: Top 5 user per lock latency";
$nbL = 1;
for my $lQuery (
    select_array(
'select user, lock_latency from sys.x\\$user_summary_by_statement_latency order
by lock_latency desc LIMIT 5'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
if ( $nbL == 1 );

```

```

# Top user per full scans
subheaderprint "Performance schema: Top 5 user per nb full scans";
$nbL = 1;
for my $lQuery (
    select_array(
        'select user, full_scans from sys.x\\$user_summary_by_statement_latency order
by full_scans desc LIMIT 5'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# Top user per row_sent
subheaderprint "Performance schema: Top 5 user per rows sent";
$nbL = 1;
for my $lQuery (
    select_array(
        'select user, rows_sent from sys.x\\$user_summary_by_statement_latency order by
rows_sent desc LIMIT 5'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# Top user per row modified
subheaderprint "Performance schema: Top 5 user per rows modified";
$nbL = 1;
for my $lQuery (
    select_array(
        'select user, rows_affected from sys.x\\$user_summary_by_statement_latency
order by rows_affected desc LIMIT 5'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# Top user per io

```

```

subheaderprint "Performance schema: Top 5 user per io";
$nbL = 1;
for my $lQuery (
    select_array(
'select user, file_ios from sys.x\\$user_summary order by file_ios desc LIMIT
5'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# Top user per io latency
subheaderprint "Performance schema: Top 5 user per io latency";
$nbL = 1;
for my $lQuery (
    select_array(
'select user, file_io_latency from sys.x\\$user_summary order by
file_io_latency desc LIMIT 5'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# Top host per connection
subheaderprint "Performance schema: Top 5 host per connection";
$nbL = 1;
for my $lQuery (
    select_array(
'select host, total_connections from sys.x\\$host_summary order by
total_connections desc LIMIT 5'
    )
)
{
    infoprint " +-- $nbL: $lQuery conn(s)";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# Top host per statement
subheaderprint "Performance schema: Top 5 host per statement";

```

```

$nbL = 1;
for my $lQuery (
    select_array(
'select host, statements from sys.x\\$host_summary order by statements desc
LIMIT 5'
    )
)
{
    infoprint " +-- $nbL: $lQuery stmt(s)";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# Top host per statement latency
subheaderprint "Performance schema: Top 5 host per statement latency";
$nbL = 1;
for my $lQuery (
    select_array(
'select host, statement_avg_latency from sys.x\\$host_summary order by
statement_avg_latency desc LIMIT 5'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# Top host per lock latency
subheaderprint "Performance schema: Top 5 host per lock latency";
$nbL = 1;
for my $lQuery (
    select_array(
'select host, lock_latency from sys.x\\$host_summary_by_statement_latency order
by lock_latency desc LIMIT 5'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# Top host per full scans
subheaderprint "Performance schema: Top 5 host per nb full scans";
$nbL = 1;

```

```

    for my $lQuery (
        select_array(
            'select host, full_scans from sys.x\\$host_summary_by_statement_latency order
            by full_scans desc LIMIT 5'
        )
    )
    {
        infoprint " +-- $nbL: $lQuery";
        $nbL++;
    }
    infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

    # Top host per rows sent
    subheaderprint "Performance schema: Top 5 host per rows sent";
    $nbL = 1;
    for my $lQuery (
        select_array(
            'select host, rows_sent from sys.x\\$host_summary_by_statement_latency order by
            rows_sent desc LIMIT 5'
        )
    )
    {
        infoprint " +-- $nbL: $lQuery";
        $nbL++;
    }
    infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

    # Top host per rows modified
    subheaderprint "Performance schema: Top 5 host per rows modified";
    $nbL = 1;
    for my $lQuery (
        select_array(
            'select host, rows_affected from sys.x\\$host_summary_by_statement_latency
            order by rows_affected desc LIMIT 5'
        )
    )
    {
        infoprint " +-- $nbL: $lQuery";
        $nbL++;
    }
    infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

    # Top host per io
    subheaderprint "Performance schema: Top 5 host per io";
    $nbL = 1;
    for my $lQuery (

```

```

        select_array(
'select host, file_ios from sys.x\\$host_summary order by file_ios desc LIMIT
5'
        )
    )
    {
        infoprint " +-- $nbL: $lQuery";
        $nbL++;
    }
    infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# Top 5 host per io latency
subheaderprint "Performance schema: Top 5 host per io latency";
$nbL = 1;
for my $lQuery (
    select_array(
'select host, file_io_latency from sys.x\\$host_summary order by
file_io_latency desc LIMIT 5'
    )
)
    {
        infoprint " +-- $nbL: $lQuery";
        $nbL++;
    }
    infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# Top IO type order by total io
subheaderprint "Performance schema: Top IO type order by total io";
$nbL = 1;
for my $lQuery (
    select_array(
'use sys;select substring(event_name,14), SUM(total)AS total from
sys.x\\$host_summary_by_file_io_type GROUP BY substring(event_name,14) ORDER BY
total DESC;'
    )
)
    {
        infoprint " +-- $nbL: $lQuery i/o";
        $nbL++;
    }
    infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# Top IO type order by total latency
subheaderprint "Performance schema: Top IO type order by total latency";
$nbL = 1;
for my $lQuery (

```

```

        select_array(
'select substring(event_name,14), ROUND(SUM(total_latency),1) AS total_latency
from sys.x\\$host_summary_by_file_io_type GROUP BY substring(event_name,14)
ORDER BY total_latency DESC;'
        )
    )
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# Top IO type order by max latency
subheaderprint "Performance schema: Top IO type order by max latency";
$nbL = 1;
for my $lQuery (
    select_array(
'use sys;select substring(event_name,14), MAX(max_latency) as max_latency from
sys.x\\$host_summary_by_file_io_type GROUP BY substring(event_name,14) ORDER BY
max_latency DESC;'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# Top Stages order by total io
subheaderprint "Performance schema: Top Stages order by total io";
$nbL = 1;
for my $lQuery (
    select_array(
'use sys;select substring(event_name,7), SUM(total)AS total from
sys.x\\$host_summary_by_stages GROUP BY substring(event_name,7) ORDER BY total
DESC;'
    )
)
{
    infoprint " +-- $nbL: $lQuery i/o";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# Top Stages order by total latency
subheaderprint "Performance schema: Top Stages order by total latency";

```

```

$nbL = 1;
for my $lQuery (
    select_array(
'use sys;select substring(event_name,7), ROUND(SUM(total_latency),1) AS
total_latency from sys.x\\$host_summary_by_stages GROUP BY
substring(event_name,7) ORDER BY total_latency DESC;'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
if ( $nbL == 1 );

# Top Stages order by avg latency
subheaderprint "Performance schema: Top Stages order by avg latency";
$nbL = 1;
for my $lQuery (
    select_array(
'use sys;select substring(event_name,7), MAX(avg_latency) as avg_latency from
sys.x\\$host_summary_by_stages GROUP BY substring(event_name,7) ORDER BY
avg_latency DESC;'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
if ( $nbL == 1 );

# Top host per table scans
subheaderprint "Performance schema: Top 5 host per table scans";
$nbL = 1;
for my $lQuery (
    select_array(
'select host, table_scans from sys.x\\$host_summary order by table_scans desc
LIMIT 5'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
if ( $nbL == 1 );

# InnoDB Buffer Pool by schema

```

```

subheaderprint "Performance schema: InnoDB Buffer Pool by schema";
$nbL = 1;
for my $lQuery (
    select_array(
'select object_schema, allocated, data, pages from
sys.x\\$innodb_buffer_stats_by_schema ORDER BY pages DESC'
    )
)
{
    infoprint " +-- $nbL: $lQuery page(s)";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# InnoDB Buffer Pool by table
subheaderprint "Performance schema: 40 InnoDB Buffer Pool by table";
$nbL = 1;
for my $lQuery (
    select_array(
'select object_schema, object_name, allocated,data, pages from
sys.x\\$innodb_buffer_stats_by_table ORDER BY pages DESC LIMIT 40'
    )
)
{
    infoprint " +-- $nbL: $lQuery page(s)";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# Process per allocated memory
subheaderprint "Performance schema: Process per time";
$nbL = 1;
for my $lQuery (
    select_array(
'select user, Command AS PROC, time from sys.x\\$processlist ORDER BY time
DESC;'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# InnoDB Lock Waits
subheaderprint "Performance schema: InnoDB Lock Waits";

```

```

$nbL = 1;
for my $lQuery (
    select_array(
'select wait_age_secs, locked_table, locked_type, waiting_query from
sys.x\\$innodb_lock_waits order by wait_age_secs DESC;'
    )
    )
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# Threads IO Latency
subheaderprint "Performance schema: Thread IO Latency";
$nbL = 1;
for my $lQuery (
    select_array(
'select user, total_latency, max_latency from sys.x\\$io_by_thread_by_latency
order by total_latency DESC;'
    )
    )
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# High Cost SQL statements
subheaderprint "Performance schema: Top 15 Most latency statements";
$nbL = 1;
for my $lQuery (
    select_array(
'select LEFT(query, 120), avg_latency from sys.x\\$statement_analysis order by
avg_latency desc LIMIT 15'
    )
    )
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# Top 5% slower queries
subheaderprint "Performance schema: Top 15 slower queries";
$nbL = 1;

```

```

    for my $lQuery (
        select_array(
            'select LEFT(query, 120), exec_count from
            sys.x\\$statements_with_runtimes_in_95th_percentile order by exec_count desc
            LIMIT 15'
        )
    )
    {
        infoprint " +-- $nbL: $lQuery s";
        $nbL++;
    }
    infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

    # Top 10 nb statement type
    subheaderprint "Performance schema: Top 15 nb statement type";
    $nbL = 1;
    for my $lQuery (
        select_array(
            'use sys;select statement, sum(total) as total from
            sys.x\\$host_summary_by_statement_type group by statement order by total desc
            LIMIT 15;'
        )
    )
    {
        infoprint " +-- $nbL: $lQuery";
        $nbL++;
    }
    infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

    # Top statement by total latency
    subheaderprint "Performance schema: Top 15 statement by total latency";
    $nbL = 1;
    for my $lQuery (
        select_array(
            'use sys;select statement, sum(total_latency) as total from
            sys.x\\$host_summary_by_statement_type group by statement order by total desc
            LIMIT 15;'
        )
    )
    {
        infoprint " +-- $nbL: $lQuery";
        $nbL++;
    }
    infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

    # Top statement by lock latency

```

```

    subheaderprint "Performance schema: Top 15 statement by lock latency";
    $nbL = 1;
    for my $lQuery (
        select_array(
            'use sys;select statement, sum(lock_latency) as total from
            sys.x\\$host_summary_by_statement_type group by statement order by total desc
            LIMIT 15;'
        )
    )
    {
        infoprint " +-- $nbL: $lQuery";
        $nbL++;
    }
    infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

    # Top statement by full scans
    subheaderprint "Performance schema: Top 15 statement by full scans";
    $nbL = 1;
    for my $lQuery (
        select_array(
            'use sys;select statement, sum(full_scans) as total from
            sys.x\\$host_summary_by_statement_type group by statement order by total desc
            LIMIT 15;'
        )
    )
    {
        infoprint " +-- $nbL: $lQuery";
        $nbL++;
    }
    infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

    # Top statement by rows sent
    subheaderprint "Performance schema: Top 15 statement by rows sent";
    $nbL = 1;
    for my $lQuery (
        select_array(
            'use sys;select statement, sum(rows_sent) as total from
            sys.x\\$host_summary_by_statement_type group by statement order by total desc
            LIMIT 15;'
        )
    )
    {
        infoprint " +-- $nbL: $lQuery";
        $nbL++;
    }
    infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

```

```

# Top statement by rows modified
subheaderprint "Performance schema: Top 15 statement by rows modified";
$nbL = 1;
for my $lQuery (
    select_array(
        'use sys;select statement, sum(rows_affected) as total from
        sys.x\\$host_summary_by_statement_type group by statement order by total desc
        LIMIT 15;'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
if ( $nbL == 1 );

# Use temporary tables
subheaderprint "Performance schema: 15 sample queries using temp table";
$nbL = 1;
for my $lQuery (
    select_array(
        'use sys;select left(query, 120) from sys.x\\$statements_with_temp_tables LIMIT
        15'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
if ( $nbL == 1 );

# Unused Indexes
subheaderprint "Performance schema: Unused indexes";
$nbL = 1;
for my $lQuery (
    select_array(
        "select \* from sys.schema_unused_indexes where object_schema not in
        ('performance_schema')"
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
if ( $nbL == 1 );

```

```

# Full table scans
subheaderprint "Performance schema: Tables with full table scans";
$nbL = 1;
for my $lQuery (
    select_array(
        'select * from sys.x\\$schema_tables_with_full_table_scans order by
rows_full_scanned DESC'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# Latest file IO by latency
subheaderprint "Performance schema: Latest FILE IO by latency";
$nbL = 1;
for my $lQuery (
    select_array(
        'use sys;select thread, file, latency, operation from sys.x\\$latest_file_io
ORDER BY latency LIMIT 10;'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# FILE by IO read bytes
subheaderprint "Performance schema: FILE by IO read bytes";
$nbL = 1;
for my $lQuery (
    select_array(
        'select file, total_read from sys.x\\$io_global_by_file_by_bytes order by
total_read DESC LIMIT 15;'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

```

```

# FILE by IO written bytes
subheaderprint "Performance schema: FILE by IO written bytes";
$nbL = 1;
for my $lQuery (
    select_array(
        'select file, total_written from sys.x\\$io_global_by_file_by_bytes order by
total_written DESC LIMIT 15'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# file per IO total latency
subheaderprint "Performance schema: file per IO total latency";
$nbL = 1;
for my $lQuery (
    select_array(
        'select file, total_latency from sys.x\\$io_global_by_file_by_latency ORDER BY
total_latency DESC LIMIT 20;'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# file per IO read latency
subheaderprint "Performance schema: file per IO read latency";
$nbL = 1;
for my $lQuery (
    select_array(
        'use sys;select file, read_latency from sys.x\\$io_global_by_file_by_latency
ORDER BY read_latency DESC LIMIT 20;'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# file per IO write latency

```

```

subheaderprint "Performance schema: file per IO write latency";
$nbL = 1;
for my $lQuery (
    select_array(
'use sys;select file, write_latency from sys.x\\$io_global_by_file_by_latency
ORDER BY write_latency DESC LIMIT 20;'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# Event Wait by read bytes
subheaderprint "Performance schema: Event Wait by read bytes";
$nbL = 1;
for my $lQuery (
    select_array(
'select event_name, total_read from sys.x\\$io_global_by_wait_by_bytes order by
total_read DESC LIMIT 15;'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# Event Wait by write bytes
subheaderprint "Performance schema: Event Wait written bytes";
$nbL = 1;
for my $lQuery (
    select_array(
'select event_name, total_written from sys.x\\$io_global_by_wait_by_bytes order
by total_written DESC LIMIT 15;'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# event per wait total latency
subheaderprint "Performance schema: event per wait total latency";

```

```

$nbL = 1;
for my $lQuery (
    select_array(
'use sys;select event_name, total_latency from
sys.x\\$io_global_by_wait_by_latency ORDER BY total_latency DESC LIMIT 20;'
    )
    )
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# event per wait read latency
subheaderprint "Performance schema: event per wait read latency";
$nbL = 1;
for my $lQuery (
    select_array(
'use sys;select event_name, read_latency from
sys.x\\$io_global_by_wait_by_latency ORDER BY read_latency DESC LIMIT 20;'
    )
    )
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# event per wait write latency
subheaderprint "Performance schema: event per wait write latency";
$nbL = 1;
for my $lQuery (
    select_array(
'use sys;select event_name, write_latency from
sys.x\\$io_global_by_wait_by_latency ORDER BY write_latency DESC LIMIT 20;'
    )
    )
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

#schema_index_statistics
# TOP 15 most read index
subheaderprint "Performance schema: TOP 15 most read indexes";

```

```

$nbL = 1;
for my $lQuery (
    select_array(
'use sys;select table_schema, table_name,index_name, rows_selected from
sys.x\\$schema_index_statistics ORDER BY ROWs_selected DESC LIMIT 15;'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# TOP 15 most used index
subheaderprint "Performance schema: TOP 15 most modified indexes";
$nbL = 1;
for my $lQuery (
    select_array(
'use sys;select table_schema, table_name,index_name,
rows_inserted+rows_updated+rows_deleted AS changes from
sys.x\\$schema_index_statistics ORDER BY
rows_inserted+rows_updated+rows_deleted DESC LIMIT 15;'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# TOP 15 high read latency index
subheaderprint "Performance schema: TOP 15 high read latency index";
$nbL = 1;
for my $lQuery (
    select_array(
'use sys;select table_schema, table_name,index_name, select_latency from
sys.x\\$schema_index_statistics ORDER BY select_latency DESC LIMIT 15;'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# TOP 15 high insert latency index

```

```

subheaderprint "Performance schema: TOP 15 most modified indexes";
$nbL = 1;
for my $lQuery (
    select_array(
'use sys;select table_schema, table_name,index_name, insert_latency from
sys.x\\$schema_index_statistics ORDER BY insert_latency DESC LIMIT 15;'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# TOP 15 high update latency index
subheaderprint "Performance schema: TOP 15 high update latency index";
$nbL = 1;
for my $lQuery (
    select_array(
'use sys;select table_schema, table_name,index_name, update_latency from
sys.x\\$schema_index_statistics ORDER BY update_latency DESC LIMIT 15;'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# TOP 15 high delete latency index
subheaderprint "Performance schema: TOP 15 high delete latency index";
$nbL = 1;
for my $lQuery (
    select_array(
'use sys;select table_schema, table_name,index_name, delete_latency from
sys.x\\$schema_index_statistics ORDER BY delete_latency DESC LIMIT 15;'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# TOP 15 most read tables
subheaderprint "Performance schema: TOP 15 most read tables";

```

```

$nbL = 1;
for my $lQuery (
    select_array(
'use sys;select table_schema, table_name, rows_fetched from
sys.x\\$schema_table_statistics ORDER BY ROWs_fetched DESC LIMIT 15;'
    )
    )
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# TOP 15 most used tables
subheaderprint "Performance schema: TOP 15 most modified tables";
$nbL = 1;
for my $lQuery (
    select_array(
'use sys;select table_schema, table_name,
rows_inserted+rows_updated+rows_deleted AS changes from
sys.x\\$schema_table_statistics ORDER BY
rows_inserted+rows_updated+rows_deleted DESC LIMIT 15;'
    )
    )
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# TOP 15 high read latency tables
subheaderprint "Performance schema: TOP 15 high read latency tables";
$nbL = 1;
for my $lQuery (
    select_array(
'use sys;select table_schema, table_name, fetch_latency from
sys.x\\$schema_table_statistics ORDER BY fetch_latency DESC LIMIT 15;'
    )
    )
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# TOP 15 high insert latency tables

```

```

subheaderprint "Performance schema: TOP 15 high insert latency tables";
$nbL = 1;
for my $lQuery (
    select_array(
'use sys;select table_schema, table_name, insert_latency from
sys.x\\$schema_table_statistics ORDER BY insert_latency DESC LIMIT 15;'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# TOP 15 high update latency tables
subheaderprint "Performance schema: TOP 15 high update latency tables";
$nbL = 1;
for my $lQuery (
    select_array(
'use sys;select table_schema, table_name, update_latency from
sys.x\\$schema_table_statistics ORDER BY update_latency DESC LIMIT 15;'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# TOP 15 high delete latency tables
subheaderprint "Performance schema: TOP 15 high delete latency tables";
$nbL = 1;
for my $lQuery (
    select_array(
'use sys;select table_schema, table_name, delete_latency from
sys.x\\$schema_table_statistics ORDER BY delete_latency DESC LIMIT 15;'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

# Redundant indexes
subheaderprint "Performance schema: Redundant indexes";

```

```

$nbL = 1;
for my $lQuery (
    select_array('use sys;select * from schema_redundant_indexes;') )
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

subheaderprint "Performance schema: Table not using InnoDB buffer";
$nbL = 1;
for my $lQuery (
    select_array(
' Select table_schema, table_name from
sys.x\\$schema_table_statistics_with_buffer where innodb_buffer_allocated IS
NULL;'
    )
    )
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

subheaderprint "Performance schema: Top 15 Tables using InnoDB buffer";
$nbL = 1;
for my $lQuery (
    select_array(
'select table_schema,table_name,innodb_buffer_allocated from
sys.x\\$schema_table_statistics_with_buffer where innodb_buffer_allocated IS
NOT NULL ORDER BY innodb_buffer_allocated DESC LIMIT 15;'
    )
    )
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

subheaderprint "Performance schema: Top 15 Tables with InnoDB buffer free";
$nbL = 1;
for my $lQuery (
    select_array(
'select table_schema,table_name,innodb_buffer_free from
sys.x\\$schema_table_statistics_with_buffer where innodb_buffer_allocated IS
NOT NULL ORDER BY innodb_buffer_free DESC LIMIT 15;'
    )
    )
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

```

```

    )
  )
  {
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
  }
  infoprint "No information found or indicators deactivated."
  if ( $nbL == 1 );

  subheaderprint "Performance schema: Top 15 Most executed queries";
  $nbL = 1;
  for my $lQuery (
    select_array(
      'select db, LEFT(query, 120), exec_count from sys.x\\$statement_analysis order
by exec_count DESC LIMIT 15;'
    )
  )
  {
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
  }
  infoprint "No information found or indicators deactivated."
  if ( $nbL == 1 );

  subheaderprint
    "Performance schema: Latest SQL queries in errors or warnings";
  $nbL = 1;
  for my $lQuery (
    select_array(
      'select LEFT(query, 120), last_seen from
sys.x\\$statements_with_errors_or_warnings ORDER BY last_seen LIMIT 40;'
    )
  )
  {
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
  }
  infoprint "No information found or indicators deactivated."
  if ( $nbL == 1 );

  subheaderprint "Performance schema: Top 20 queries with full table scans";
  $nbL = 1;
  for my $lQuery (
    select_array(
      'select db, LEFT(query, 120), exec_count from
sys.x\\$statements_with_full_table_scans order BY exec_count DESC LIMIT 20;'
    )
  )
  {

```

```

        infoprint " +-- $nbL: $lQuery";
        $nbL++;
    }
    infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

    subheaderprint "Performance schema: Last 50 queries with full table scans";
    $nbL = 1;
    for my $lQuery (
        select_array(
            'select db, LEFT(query, 120), last_seen from
            sys.x\\$statements_with_full_table_scans order BY last_seen DESC LIMIT 50;'
        )
    )
    {
        infoprint " +-- $nbL: $lQuery";
        $nbL++;
    }
    infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

    subheaderprint "Performance schema: TOP 15 reader queries (95% percentile)";
    $nbL = 1;
    for my $lQuery (
        select_array(
            'use sys;select db, LEFT(query, 120), rows_sent from
            sys.x\\$statements_with_runtimes_in_95th_percentile ORDER BY ROWs_sent DESC
            LIMIT 15;'
        )
    )
    {
        infoprint " +-- $nbL: $lQuery";
        $nbL++;
    }
    infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

    subheaderprint
        "Performance schema: TOP 15 most row look queries (95% percentile)";
    $nbL = 1;
    for my $lQuery (
        select_array(
            'use sys;select db, LEFT(query, 120), rows_examined AS search from
            sys.x\\$statements_with_runtimes_in_95th_percentile ORDER BY rows_examined DESC
            LIMIT 15;'
        )
    )
    {
        infoprint " +-- $nbL: $lQuery";
    }

```

```

        $nbL++;
    }
    infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

    subheaderprint
        "Performance schema: TOP 15 total latency queries (95% percentile)";
    $nbL = 1;
    for my $lQuery (
        select_array(
            'use sys;select db, LEFT(query, 120), total_latency AS search from
            sys.x\\$statements_with_runtimes_in_95th_percentile ORDER BY total_latency DESC
            LIMIT 15;'
        )
    )
    {
        infoprint " +-- $nbL: $lQuery";
        $nbL++;
    }
    infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

    subheaderprint
        "Performance schema: TOP 15 max latency queries (95% percentile)";
    $nbL = 1;
    for my $lQuery (
        select_array(
            'use sys;select db, LEFT(query, 120), max_latency AS search from
            sys.x\\$statements_with_runtimes_in_95th_percentile ORDER BY max_latency DESC
            LIMIT 15;'
        )
    )
    {
        infoprint " +-- $nbL: $lQuery";
        $nbL++;
    }
    infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

    subheaderprint
        "Performance schema: TOP 15 average latency queries (95% percentile)";
    $nbL = 1;
    for my $lQuery (
        select_array(
            'use sys;select db, LEFT(query, 120), avg_latency AS search from
            sys.x\\$statements_with_runtimes_in_95th_percentile ORDER BY avg_latency DESC
            LIMIT 15;'
        )
    )

```

```

{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

subheaderprint "Performance schema: Top 20 queries with sort";
$nbL = 1;
for my $lQuery (
    select_array(
        'select db, LEFT(query, 120), exec_count from sys.x\\$statements_with_sorting
order BY exec_count DESC LIMIT 20;'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

subheaderprint "Performance schema: Last 50 queries with sort";
$nbL = 1;
for my $lQuery (
    select_array(
        'select db, LEFT(query, 120), last_seen from sys.x\\$statements_with_sorting
order BY last_seen DESC LIMIT 50;'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

subheaderprint "Performance schema: TOP 15 row sorting queries with sort";
$nbL = 1;
for my $lQuery (
    select_array(
        'use sys;select db, LEFT(query, 120), rows_sorted from
sys.x\\$statements_with_sorting ORDER BY ROWs_sorted DESC LIMIT 15;'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}

```

```

infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

    subheaderprint "Performance schema: TOP 15 total latency queries with sort";
    $nbL = 1;
    for my $lQuery (
        select_array(
            'use sys;select db, LEFT(query, 120), total_latency AS search from
            sys.x\\$statements_with_sorting ORDER BY total_latency DESC LIMIT 15;'
        )
    )
    {
        infoprint " +-- $nbL: $lQuery";
        $nbL++;
    }
    infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

    subheaderprint "Performance schema: TOP 15 merge queries with sort";
    $nbL = 1;
    for my $lQuery (
        select_array(
            'use sys;select db, LEFT(query, 120), sort_merge_passes AS search from
            sys.x\\$statements_with_sorting ORDER BY sort_merge_passes DESC LIMIT 15;'
        )
    )
    {
        infoprint " +-- $nbL: $lQuery";
        $nbL++;
    }
    infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

    subheaderprint
        "Performance schema: TOP 15 average sort merges queries with sort";
    $nbL = 1;
    for my $lQuery (
        select_array(
            'select db, LEFT(query, 120), avg_sort_merges AS search from
            sys.x\\$statements_with_sorting ORDER BY avg_sort_merges DESC LIMIT 15;'
        )
    )
    {
        infoprint " +-- $nbL: $lQuery";
        $nbL++;
    }
    infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

```

```

subheaderprint "Performance schema: TOP 15 scans queries with sort";
$nbL = 1;
for my $lQuery (
    select_array(
'use sys;select db, LEFT(query, 120), sorts_using_scans AS search from
sys.x\\$statements_with_sorting ORDER BY sorts_using_scans DESC LIMIT 15;'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
if ( $nbL == 1 );

```

```

subheaderprint "Performance schema: TOP 15 range queries with sort";
$nbL = 1;
for my $lQuery (
    select_array(
'use sys;select db, LEFT(query, 120), sort_using_range AS search from
sys.x\\$statements_with_sorting ORDER BY sort_using_range DESC LIMIT 15;'
    )
)
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
if ( $nbL == 1 );

```

```

#####
####

```

```

#statements_with_temp_tables

```

```

#mysql> desc statements_with_temp_tables;

```

```

#-----+-----+-----+-----+-----+
-----+-----+
#| Field                | Type                | Null | Key | Default
| Extra |
#-----+-----+-----+-----+-----+
-----+-----+
#| query                | longtext            | YES  |     | NULL
|          |
#| db                   | varchar(64)         | YES  |     | NULL
|          |
#| exec_count           | bigint(20) unsigned | NO   |     | NULL
|          |

```

```

#| total_latency          | text          | YES | | NULL
| |
#| memory_tmp_tables      | bigint(20) unsigned | NO  | | NULL
| |
#| disk_tmp_tables        | bigint(20) unsigned | NO  | | NULL
| |
#| avg_tmp_tables_per_query | decimal(21,0)      | NO  | | 0
| |
#| tmp_tables_to_disk_pct  | decimal(24,0)      | NO  | | 0
| |
#| first_seen             | timestamp         | NO  | | 0000-00-00
00:00:00 | |
#| last_seen              | timestamp         | NO  | | 0000-00-00
00:00:00 | |
#| digest                 | varchar(32)       | YES | | NULL
| |
#+-----+-----+-----+-----+-----+
-----+-----+
#11 rows in set (0,01 sec)#
#
    subheaderprint "Performance schema: Top 20 queries with temp table";
    $nbL = 1;
    for my $lQuery (
        select_array(
'select db, LEFT(query, 120), exec_count from
sys.x\\$statements_with_temp_tables order BY exec_count DESC LIMIT 20;'
        )
    )
    {
        infoprint " +-- $nbL: $lQuery";
        $nbL++;
    }
    infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

    subheaderprint "Performance schema: Last 50 queries with temp table";
    $nbL = 1;
    for my $lQuery (
        select_array(
'select db, LEFT(query, 120), last_seen from
sys.x\\$statements_with_temp_tables order BY last_seen DESC LIMIT 50;'
        )
    )
    {
        infoprint " +-- $nbL: $lQuery";
        $nbL++;
    }
    infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

```

```

subheaderprint
    "Performance schema: TOP 15 total latency queries with temp table";
$nbL = 1;
for my $lQuery (
    select_array(
'select db, LEFT(query, 120), total_latency AS search from
sys.x\\$statements_with_temp_tables ORDER BY total_latency DESC LIMIT 15;'
    )
    )
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

subheaderprint "Performance schema: TOP 15 queries with temp table to disk";
$nbL = 1;
for my $lQuery (
    select_array(
'use sys;select db, LEFT(query, 120), disk_tmp_tables from
sys.x\\$statements_with_temp_tables ORDER BY disk_tmp_tables DESC LIMIT 15;'
    )
    )
{
    infoprint " +-- $nbL: $lQuery";
    $nbL++;
}
infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

```

```

#####
####

```

```

    #wait_classes_global_by_latency

```

```

#mysql> select * from wait_classes_global_by_latency;

```

```

#-----+-----+-----+-----+-----+-----+-----+
-----+
# event_class      | total | total_latency | min_latency | avg_latency |
max_latency |
#-----+-----+-----+-----+-----+-----+-----+
-----+
# wait/io/file     | 15381 | 1.23 s        | 0 ps        | 80.12 us    | 230.64
ms      |
# wait/io/table    | 59    | 7.57 ms       | 5.45 us     | 128.24 us   | 3.95
ms      |
# wait/lock/table  | 69    | 3.22 ms       | 658.84 ns   | 46.64 us    | 1.10
ms      |

```

```

#-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+
# rows in set (0,00 sec)

    subheaderprint "Performance schema: TOP 15 class events by number";
    $nbL = 1;
    for my $lQuery (
        select_array(
'use sys;select event_class, total from sys.x\\$wait_classes_global_by_latency
ORDER BY total DESC LIMIT 15;'
        )
    )
    {
        infoprint " +-- $nbL: $lQuery";
        $nbL++;
    }
    infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

    subheaderprint "Performance schema: TOP 30 events by number";
    $nbL = 1;
    for my $lQuery (
        select_array(
'use sys;select events, total from sys.x\\$waits_global_by_latency ORDER BY
total DESC LIMIT 30;'
        )
    )
    {
        infoprint " +-- $nbL: $lQuery";
        $nbL++;
    }
    infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

    subheaderprint "Performance schema: TOP 15 class events by total latency";
    $nbL = 1;
    for my $lQuery (
        select_array(
'use sys;select event_class, total_latency from
sys.x\\$wait_classes_global_by_latency ORDER BY total_latency DESC LIMIT 15;'
        )
    )
    {
        infoprint " +-- $nbL: $lQuery";
        $nbL++;
    }
    infoprint "No information found or indicators deactivated."
    if ( $nbL == 1 );

```

```

        subheaderprint "Performance schema: TOP 30 events by total latency";
        $nbL = 1;
        for my $lQuery (
            select_array(
                'use sys;select events, total_latency from sys.x\\$waits_global_by_latency
                ORDER BY total_latency DESC LIMIT 30;'
            )
        )
        {
            infoprint " +-- $nbL: $lQuery";
            $nbL++;
        }
        infoprint "No information found or indicators deactivated."
        if ( $nbL == 1 );

        subheaderprint "Performance schema: TOP 15 class events by max latency";
        $nbL = 1;
        for my $lQuery (
            select_array(
                'select event_class, max_latency from sys.x\\$wait_classes_global_by_latency
                ORDER BY max_latency DESC LIMIT 15;'
            )
        )
        {
            infoprint " +-- $nbL: $lQuery";
            $nbL++;
        }
        infoprint "No information found or indicators deactivated."
        if ( $nbL == 1 );

        subheaderprint "Performance schema: TOP 30 events by max latency";
        $nbL = 1;
        for my $lQuery (
            select_array(
                'select events, max_latency from sys.x\\$waits_global_by_latency ORDER BY
                max_latency DESC LIMIT 30;'
            )
        )
        {
            infoprint " +-- $nbL: $lQuery";
            $nbL++;
        }
        infoprint "No information found or indicators deactivated."
        if ( $nbL == 1 );
    }

    # Recommendations for Aria Engine
    sub mariadb_aria {

```

```

subheaderprint "Aria Metrics";

# Aria
if ( !defined $myvar{'have_aria'} ) {
    infoprint "Aria Storage Engine not available.";
    return;
}
if ( $myvar{'have_aria'} ne "YES" ) {
    infoprint "Aria Storage Engine is disabled.";
    return;
}
infoprint "Aria Storage Engine is enabled.";

# Aria pagecache
if ( !defined( $mycalc{'total_aria_indexes'} ) ) {
    push( @generalrec,
        "Unable to calculate Aria index size on MySQL server" );
}
else {
    if (
        $myvar{'aria_pagecache_buffer_size'} < $mycalc{'total_aria_indexes'}
        && $mycalc{'pct_aria_keys_from_mem'} < 95 )
    {
        badprint "Aria pagecache size / total Aria indexes: "
            . hr_bytes( $myvar{'aria_pagecache_buffer_size'} ) . "/"
            . hr_bytes( $mycalc{'total_aria_indexes'} ) . "";
        push( @adjvars,
            "aria_pagecache_buffer_size (> "
            . hr_bytes( $mycalc{'total_aria_indexes'} )
            . ")" );
    }
    else {
        goodprint "Aria pagecache size / total Aria indexes: "
            . hr_bytes( $myvar{'aria_pagecache_buffer_size'} ) . "/"
            . hr_bytes( $mycalc{'total_aria_indexes'} ) . "";
    }
    if ( $mystat{'Aria_pagecache_read_requests'} > 0 ) {
        if ( $mycalc{'pct_aria_keys_from_mem'} < 95 ) {
            badprint
"Aria pagecache hit rate: $mycalc{'pct_aria_keys_from_mem'}% ("
            . hr_num( $mystat{'Aria_pagecache_read_requests'} )
            . " cached / "
            . hr_num( $mystat{'Aria_pagecache_reads'} )
            . " reads)";
        }
        else {
            goodprint
"Aria pagecache hit rate: $mycalc{'pct_aria_keys_from_mem'}% ("
            . hr_num( $mystat{'Aria_pagecache_read_requests'} )

```

```

        . " cached / "
        . hr_num( $mystat{'Aria_pagecache_reads'} )
        . " reads)";
    }
}
else {

    # No queries have run that would use keys
}
}

# Recommendations for TokuDB
sub mariadb_tokudb {
    subheaderprint "TokuDB Metrics";

    # AriaDB
    unless ( defined $myvar{'have_tokudb'}
        && $myvar{'have_tokudb'} eq "YES" )
    {
        infoprint "TokuDB is disabled.";
        return;
    }
    infoprint "TokuDB is enabled.";

    # All is to done here
}

# Recommendations for XtraDB
sub mariadb_xtradb {
    subheaderprint "XtraDB Metrics";

    # XtraDB
    unless ( defined $myvar{'have_xtradb'}
        && $myvar{'have_xtradb'} eq "YES" )
    {
        infoprint "XtraDB is disabled.";
        return;
    }
    infoprint "XtraDB is enabled.";
    infoprint "Note that MariaDB 10.2 makes use of InnoDB, not XtraDB."

    # All is to done here
}

# Recommendations for RocksDB
sub mariadb_rocksdb {
    subheaderprint "RocksDB Metrics";

```

```

# RocksDB
unless ( defined $myvar{'have_rocksdb'}
    && $myvar{'have_rocksdb'} eq "YES" )
{
    infoprint "RocksDB is disabled.";
    return;
}
infoprint "RocksDB is enabled.";

# All is to do here
}

# Recommendations for Spider
sub mariadb_spider {
    subheaderprint "Spider Metrics";

    # Spider
    unless ( defined $myvar{'have_spider'}
        && $myvar{'have_spider'} eq "YES" )
    {
        infoprint "Spider is disabled.";
        return;
    }
    infoprint "Spider is enabled.";

    # All is to do here
}

# Recommendations for Connect
sub mariadb_connect {
    subheaderprint "Connect Metrics";

    # Connect
    unless ( defined $myvar{'have_connect'}
        && $myvar{'have_connect'} eq "YES" )
    {
        infoprint "Connect is disabled.";
        return;
    }
    infoprint "Connect is enabled.";

    # All is to do here
}

# Perl trim function to remove whitespace from the start and end of the string
sub trim {
    my $string = shift;
    return "" unless defined($string);
    $string =~ s/^\s+//;

```

```

$string =~ s/\s+$/;/;
return $string;
}

sub get_wsrep_options {
    return () unless defined $myvar{'wsrep_provider_options'};

    my @galera_options = split /\s/, $myvar{'wsrep_provider_options'};
    my $wsrep_slave_threads = $myvar{'wsrep_slave_threads'};
    push @galera_options, ' wsrep_slave_threads = ' . $wsrep_slave_threads;
    @galera_options = remove_cr @galera_options;
    @galera_options = remove_empty @galera_options;
    debugprint Dumper( \@galera_options );
    return @galera_options;
}

sub get_gcache_memory {
    my $gCacheMem = hr_raw( get_wsrep_option('gcache.size') );

    return 0 unless defined $gCacheMem and $gCacheMem ne '';
    return $gCacheMem;
}

sub get_wsrep_option {
    my $key = shift;
    return '' unless defined $myvar{'wsrep_provider_options'};
    my @galera_options = get_wsrep_options;
    return '' unless scalar(@galera_options) > 0;
    my @memValues = grep /\s*$key =/, @galera_options;
    my $memValue = $memValues[0];
    return 0 unless defined $memValue;
    $memValue =~ s/.*=\s*(.+)$/$1/g;
    return $memValue;
}

# Recommendations for Galera
sub mariadb_galera {
    subheaderprint "Galera Metrics";

    # Galera Cluster
    unless ( defined $myvar{'have_galera'}
        && $myvar{'have_galera'} eq "YES" )
    {
        infoprint "Galera is disabled.";
        return;
    }
    infoprint "Galera is enabled.";
    debugprint "Galera variables:";
    foreach my $gvar ( keys %myvar ) {

```

```

    next unless $gvar =~ /^wsrep.*;/;
    next if $gvar eq 'wsrep_provider_options';
    debugprint "\t" . trim($gvar) . " = " . $myvar{$gvar};
    $result{'Galera'}{'variables'}{$gvar} = $myvar{$gvar};
}
if ( not defined( $myvar{'wsrep_on'} ) or $myvar{'wsrep_on'} ne "ON" ) {
    infoprint "Galera is disabled.";
    return;
}
debugprint "Galera wsrep provider Options:";
my @galera_options = get_wsrep_options;
$result{'Galera'}{'wsrep_options'} = get_wsrep_options();
foreach my $gparam (@galera_options) {
    debugprint "\t" . trim($gparam);
}
debugprint "Galera status:";
foreach my $gstatus ( keys %mystat ) {
    next unless $gstatus =~ /^wsrep.*;/;
    debugprint "\t" . trim($gstatus) . " = " . $mystat{$gstatus};
    $result{'Galera'}{'status'}{$gstatus} = $myvar{$gstatus};
}
infoprint "GCache is using "
    . hr_bytes_rnd( get_wsrep_option('gcache.mem_size') );

#my @primaryKeysNbTables=();
my @primaryKeysNbTables = select_array(
    "Select CONCAT(c.table_schema,CONCAT('.', c.table_name))
from information_schema.columns c
join information_schema.tables t using (TABLE_SCHEMA, TABLE_NAME)
where c.table_schema not in ('mysql', 'information_schema',
'performance_schema')
and t.table_type != 'VIEW'
group by c.table_schema,c.table_name
having sum(if(c.column_key in ('PRI','UNI'), 1,0)) = 0"
);

infoprint "CPU core detected : " . (cpu_cores);
infoprint "wsrep_slave_threads: " . get_wsrep_option('wsrep_slave_threads');

if ( get_wsrep_option('wsrep_slave_threads') > ( (cpu_cores) * 4 )
    or get_wsrep_option('wsrep_slave_threads') < ( (cpu_cores) * 2 ) )
{
    badprint
"wsrep_slave_threads is not equal to 2, 3 or 4 times number of CPU(s)";
    push @adjvars, "wsrep_slave_threads = " . ( (cpu_cores) * 4 );
}
else {
    goodprint
    "wsrep_slave_threads is equal to 2, 3 or 4 times number of CPU(s)";
}

```

```

}

if ( get_wsrep_option('wsrep_slave_threads') > 1 ) {
    infoprint
        "wsrep parallel slave can cause frequent inconsistency crash.";
    push @adjvars,
        "Set wsrep_slave_threads to 1 in case of HA_ERR_FOUND_DUPP_KEY crash on slave";

    # check options for parallel slave
    if ( get_wsrep_option('wsrep_slave_FK_checks') eq "OFF" ) {
        badprint "wsrep_slave_FK_checks is off with parallel slave";
        push @adjvars,
            "wsrep_slave_FK_checks should be ON when using parallel slave";
    }

    # wsrep_slave_UK_checks seems useless in MySQL source code
    if ( $myvar{'innodb_autoinc_lock_mode'} != 2 ) {
        badprint
            "innodb_autoinc_lock_mode is incorrect with parallel slave";
        push @adjvars,
            "innodb_autoinc_lock_mode should be 2 when using parallel slave";
    }
}

if ( get_wsrep_option('gcs.fc_limit') != $myvar{'wsrep_slave_threads'} * 5 )
{
    badprint "gcs.fc_limit should be equal to 5 * wsrep_slave_threads
(=".($myvar{'wsrep_slave_threads'} * 5). " ");
    push @adjvars, "gcs.fc_limit= wsrep_slave_threads * 5
(=".($myvar{'wsrep_slave_threads'} * 5). " ");
}
else {
    goodprint "gcs.fc_limit is equal to 5 * wsrep_slave_threads (
="get_wsrep_option('gcs.fc_limit') .")";
}

if ( get_wsrep_option('gcs.fc_factor') != 0.8 ) {
    badprint "gcs.fc_factor should be equal to 0.8
(="get_wsrep_option('gcs.fc_factor') .")";
    push @adjvars, "gcs.fc_factor=0.8";
}
else {
    goodprint "gcs.fc_factor is equal to 0.8";
}

if ( get_wsrep_option('wsrep_flow_control_paused') > 0.02 ) {
    badprint "Fraction of time node pause flow control > 0.02";
}
else {
    goodprint

```

```

"Flow control fraction seems to be OK (wsrep_flow_control_paused<=0.02)";
}

if ( scalar(@primaryKeysNbTables) > 0 ) {
    badprint "Following table(s) don't have primary key:";
    foreach my $badtable (@primaryKeysNbTables) {
        badprint "\t$badtable";
        push @{$result{'Tables without PK'} }, $badtable;
    }
}
else {
    goodprint "All tables get a primary key";
}

my @nonInnoDBTables = select_array(
"select CONCAT(table_schema,CONCAT('.', table_name)) from
information_schema.tables where ENGINE <> 'InnoDB' and table_schema not in
('mysql', 'performance_schema', 'information_schema')"
);
if ( scalar(@nonInnoDBTables) > 0 ) {
    badprint "Following table(s) are not InnoDB table:";
    push @generalrec,
        "Ensure that all table(s) are InnoDB tables for Galera replication";
    foreach my $badtable (@nonInnoDBTables) {
        badprint "\t$badtable";
    }
}
else {
    goodprint "All tables are InnoDB tables";
}

if ( $myvar{'binlog_format'} ne 'ROW' ) {
    badprint "Binlog format should be in ROW mode.";
    push @adjvars, "binlog_format = ROW";
}
else {
    goodprint "Binlog format is in ROW mode.";
}

if ( $myvar{'innodb_flush_log_at_trx_commit'} != 0 ) {
    badprint "InnoDB flush log at each commit should be disabled.";
    push @adjvars, "innodb_flush_log_at_trx_commit = 0";
}
else {
    goodprint "InnoDB flush log at each commit is disabled for Galera.";
}

infoprint "Read consistency mode : " . $myvar{'wsrep_causal_reads'};

if ( defined( $myvar{'wsrep_cluster_name'} )
    and $myvar{'wsrep_on'} eq "ON" )
{

```

```

goodprint "Galera WsREP is enabled.";
if ( defined( $myvar{'wsrep_cluster_address'} )
    and trim("$myvar{'wsrep_cluster_address'}") ne "" )
{
    goodprint "Galera Cluster address is defined: "
        . $myvar{'wsrep_cluster_address'};
    my @NodesTmp = split /,/, $myvar{'wsrep_cluster_address'};
    my $nbNodes = @NodesTmp;
    infoprint "There are $nbNodes nodes in wsrep_cluster_address";
    my $nbNodesSize = trim( $mystat{'wsrep_cluster_size'} );
    if ( $nbNodesSize == 3 or $nbNodesSize == 5 ) {
        goodprint "There are $nbNodesSize nodes in wsrep_cluster_size.";
    }
    else {
        badprint
        "There are $nbNodesSize nodes in wsrep_cluster_size. Prefer 3 or 5 nodes
        architecture.";
        push @generalrec, "Prefer 3 or 5 nodes architecture.";
    }

    # wsrep_cluster_address doesn't include garbd nodes
    if ( $nbNodes > $nbNodesSize ) {
        badprint
        "All cluster nodes are not detected. wsrep_cluster_size less then node count in
        wsrep_cluster_address";
    }
    else {
        goodprint "All cluster nodes detected.";
    }
}
else {
    badprint "Galera Cluster address is undefined";
    push @adjvars,
        "set up wsrep_cluster_address variable for Galera replication";
}
if ( defined( $myvar{'wsrep_cluster_name'} )
    and trim( $myvar{'wsrep_cluster_name'} ) ne "" )
{
    goodprint "Galera Cluster name is defined: "
        . $myvar{'wsrep_cluster_name'};
}
else {
    badprint "Galera Cluster name is undefined";
    push @adjvars,
        "set up wsrep_cluster_name variable for Galera replication";
}
if ( defined( $myvar{'wsrep_node_name'} )
    and trim( $myvar{'wsrep_node_name'} ) ne "" )
{

```

```

        goodprint "Galera Node name is defined: "
            . $myvar{'wsrep_node_name'};
    }
else {
    badprint "Galera node name is undefined";
    push @adjvars,
        "set up wsrep_node_name variable for Galera replication";
}
if ( trim( $myvar{'wsrep_notify_cmd'} ) ne "" ) {
    goodprint "Galera Notify command is defined.";
}
else {
    badprint "Galera Notify command is not defined.";
    push( @adjvars, "set up parameter wsrep_notify_cmd to be notify" );
}
if (      trim( $myvar{'wsrep_sst_method'} ) !~ "^xtrabackup.*"
    and trim( $myvar{'wsrep_sst_method'} ) !~ "^mariabackup" )
{
    badprint "Galera SST method is not xtrabackup based.";
    push( @adjvars,
        "set up parameter wsrep_sst_method to xtrabackup based parameter"
    );
}
else {
    goodprint "SST Method is based on xtrabackup.";
}
if (
    (
        defined( $myvar{'wsrep_OSU_method'} )
        && trim( $myvar{'wsrep_OSU_method'} ) eq "TOI"
    )
    || ( defined( $myvar{'wsrep_osu_method'} )
        && trim( $myvar{'wsrep_osu_method'} ) eq "TOI" )
)
{
    goodprint "TOI is default mode for upgrade.";
}
else {
    badprint "Schema upgrade are not replicated automatically";
    push( @adjvars, "set up parameter wsrep_OSU_method to TOI" );
}
infoprint "Max WsRep message : "
    . hr_bytes( $myvar{'wsrep_max_ws_size'} );
}
else {
    badprint "Galera WsREP is disabled";
}

if ( defined( $mystat{'wsrep_connected'} ) )

```

```

        and $mystat{'wsrep_connected'} eq "ON" )
    {
        goodprint "Node is connected";
    }
else {
    badprint "Node is disconnected";
}
if ( defined( $mystat{'wsrep_ready'} ) and $mystat{'wsrep_ready'} eq "ON" )
{
    goodprint "Node is ready";
}
else {
    badprint "Node is not ready";
}
infoprint "Cluster status :" . $mystat{'wsrep_cluster_status'};
if ( defined( $mystat{'wsrep_cluster_status'} )
    and $mystat{'wsrep_cluster_status'} eq "Primary" )
{
    goodprint "Galera cluster is consistent and ready for operations";
}
else {
    badprint "Cluster is not consistent and ready";
}
if ( $mystat{'wsrep_local_state_uuid'} eq
    $mystat{'wsrep_cluster_state_uuid'} )
{
    goodprint "Node and whole cluster at the same level: "
        . $mystat{'wsrep_cluster_state_uuid'};
}
else {
    badprint "Node and whole cluster not the same level";
    infoprint "Node state uuid: " . $mystat{'wsrep_local_state_uuid'};
    infoprint "Cluster state uuid: " . $mystat{'wsrep_cluster_state_uuid'};
}
if ( $mystat{'wsrep_local_state_comment'} eq 'Synced' ) {
    goodprint "Node is synced with whole cluster.";
}
else {
    badprint "Node is not synced";
    infoprint "Node State : " . $mystat{'wsrep_local_state_comment'};
}
if ( $mystat{'wsrep_local_cert_failures'} == 0 ) {
    goodprint "There is no certification failures detected.";
}
else {
    badprint "There is "
        . $mystat{'wsrep_local_cert_failures'}
        . " certification failure(s)detected.";
}
}

```

```

for my $key ( keys %mystat ) {
    if ( $key =~ /wsrep_|galera/i ) {
        debugprint "WSREP: $key = $mystat{$key}";
    }
}
debugprint Dumper get_wsrep_options();
}

# Recommendations for InnoDB
sub mysql_innodb {
    subheaderprint "InnoDB Metrics";

    # InnoDB
    unless ( defined $myvar{'have_innodb'}
        && $myvar{'have_innodb'} eq "YES" )
    {
        infoprint "InnoDB is disabled.";
        if ( mysql_version_ge( 5, 5 ) ) {
            my $defengine = 'InnoDB';
            $defengine = $myvar{'default_storage_engine'}
                if defined( $myvar{'default_storage_engine'} );
            badprint
                "InnoDB Storage engine is disabled. $defengine is the default storage engine"
                if $defengine eq 'InnoDB';
            infoprint
                "InnoDB Storage engine is disabled. $defengine is the default storage engine"
                if $defengine ne 'InnoDB';
        }
        return;
    }
    infoprint "InnoDB is enabled.";
    if ( !defined $enginestats{'InnoDB'} ) {
        if ( $opt{skipsizes} eq 1 ) {
            infoprint "Skipped due to --skipsizes option";
            return;
        }
        badprint "No tables are Innodb";
        $enginestats{'InnoDB'} = 0;
    }

    if ( $opt{buffers} ne 0 ) {
        infoprint "InnoDB Buffers";
        if ( defined $myvar{'innodb_buffer_pool_size'} ) {
            infoprint " +-- InnoDB Buffer Pool: "
                . hr_bytes( $myvar{'innodb_buffer_pool_size'} ) . " ";
        }
        if ( defined $myvar{'innodb_buffer_pool_instances'} ) {
            infoprint " +-- InnoDB Buffer Pool Instances: "

```

```

        . $myvar{'innodb_buffer_pool_instances'} . "";
    }

    if ( defined $myvar{'innodb_buffer_pool_chunk_size'} ) {
        infoprint " +-- InnoDB Buffer Pool Chunk Size: "
            . hr_bytes( $myvar{'innodb_buffer_pool_chunk_size'} ) . "";
    }

    if ( defined $myvar{'innodb_additional_mem_pool_size'} ) {
        infoprint " +-- InnoDB Additional Mem Pool: "
            . hr_bytes( $myvar{'innodb_additional_mem_pool_size'} ) . "";
    }

    if ( defined $myvar{'innodb_log_file_size'} ) {
        infoprint " +-- InnoDB Log File Size: "
            . hr_bytes( $myvar{'innodb_log_file_size'} );
    }

    if ( defined $myvar{'innodb_log_files_in_group'} ) {
        infoprint " +-- InnoDB Log File In Group: "
            . $myvar{'innodb_log_files_in_group'};
    }

    if ( defined $myvar{'innodb_log_files_in_group'} ) {
        infoprint " +-- InnoDB Total Log File Size: "
            . hr_bytes( $myvar{'innodb_log_files_in_group'} *
                $myvar{'innodb_log_file_size'} )
            . "("
            . $mycalc{'innodb_log_size_pct'}
            . " % of buffer pool)";
    }

    if ( defined $myvar{'innodb_log_buffer_size'} ) {
        infoprint " +-- InnoDB Log Buffer: "
            . hr_bytes( $myvar{'innodb_log_buffer_size'} );
    }

    if ( defined $mystat{'Innodb_buffer_pool_pages_free'} ) {
        infoprint " +-- InnoDB Log Buffer Free: "
            . hr_bytes( $mystat{'Innodb_buffer_pool_pages_free'} ) . "";
    }

    if ( defined $mystat{'Innodb_buffer_pool_pages_total'} ) {
        infoprint " +-- InnoDB Log Buffer Used: "
            . hr_bytes( $mystat{'Innodb_buffer_pool_pages_total'} ) . "";
    }
}

if ( defined $myvar{'innodb_thread_concurrency'} ) {
    infoprint "InnoDB Thread Concurrency: "
        . $myvar{'innodb_thread_concurrency'};
}

# InnoDB Buffer Pool Size
if ( $myvar{'innodb_file_per_table'} eq "ON" ) {
    goodprint "InnoDB File per table is activated";
}

```

```

else {
    badprint "InnoDB File per table is not activated";
    push( @adjvars, "innodb_file_per_table=ON" );
}

# InnoDB Buffer Pool Size
if ( $myvar{'innodb_buffer_pool_size'} > $enginestats{'InnoDB'} ) {
    goodprint "InnoDB buffer pool / data size: "
        . hr_bytes( $myvar{'innodb_buffer_pool_size'} ) . "/"
        . hr_bytes( $enginestats{'InnoDB'} ) . "";
}
else {
    badprint "InnoDB buffer pool / data size: "
        . hr_bytes( $myvar{'innodb_buffer_pool_size'} ) . "/"
        . hr_bytes( $enginestats{'InnoDB'} ) . "";
    push( @adjvars,
        "innodb_buffer_pool_size (>= "
        . hr_bytes( $enginestats{'InnoDB'} )
        . ") if possible." );
}
if ( $mycalc{'innodb_log_size_pct'} < 20
    or $mycalc{'innodb_log_size_pct'} > 30 )
{
    badprint "Ratio InnoDB log file size / InnoDB Buffer pool size ("
        . $mycalc{'innodb_log_size_pct'} . " %): "
        . hr_bytes( $myvar{'innodb_log_file_size'} ) . " * "
        . $myvar{'innodb_log_files_in_group'} . "/"
        . hr_bytes( $myvar{'innodb_buffer_pool_size'} )
        . " should be equal to 25%";
    push(
        @adjvars,
        "innodb_log_file_size should be (= "
        . hr_bytes_rnd(
            $myvar{'innodb_buffer_pool_size'} /
            $myvar{'innodb_log_files_in_group'} / 4
        )
        . ") if possible, so InnoDB total log files size equals to 25% of
buffer pool size."
    );
    if ( mysql_version_le( 5, 6, 2 ) ) {
        push( @generalrec,
            "For MySQL 5.6.2 and lower, Max combined innodb_log_file_size should have a
ceiling of (4096MB / log files in group) - 1MB."
        );
    }
    push( @generalrec,
        "Before changing innodb_log_file_size and/or innodb_log_files_in_group read
this: https://bit.ly/2TcGgtU"
    );
}

```

```

}
else {
    goodprint "Ratio InnoDB log file size / InnoDB Buffer pool size: "
        . hr_bytes( $myvar{'innodb_log_file_size'} ) . " * "
        . $myvar{'innodb_log_files_in_group'} . "/"
        . hr_bytes( $myvar{'innodb_buffer_pool_size'} )
        . " should be equal to 25%";
}

# InnoDB Buffer Pool Instances (MySQL 5.6.6+)
if ( not mysql_version_ge( 10, 5 )
    and defined( $myvar{'innodb_buffer_pool_instances'} ) )
{

    # Bad Value if > 64
    if ( $myvar{'innodb_buffer_pool_instances'} > 64 ) {
        badprint "InnoDB buffer pool instances: "
            . $myvar{'innodb_buffer_pool_instances'} . "";
        push( @adjvars, "innodb_buffer_pool_instances (<= 64)" );
    }

    # InnoDB Buffer Pool Size > 1Go
    if ( $myvar{'innodb_buffer_pool_size'} > 1024 * 1024 * 1024 ) {

# InnoDB Buffer Pool Size / 1Go = InnoDB Buffer Pool Instances limited to 64
# max.

        # InnoDB Buffer Pool Size > 64Go
        my $max_innodb_buffer_pool_instances =
            int( $myvar{'innodb_buffer_pool_size'} / ( 1024 * 1024 * 1024 ) );
        $max_innodb_buffer_pool_instances = 64
        if ( $max_innodb_buffer_pool_instances > 64 );

        if ( $myvar{'innodb_buffer_pool_instances'} !=
            $max_innodb_buffer_pool_instances )
        {
            badprint "InnoDB buffer pool instances: "
                . $myvar{'innodb_buffer_pool_instances'} . "";
            push( @adjvars,
                "innodb_buffer_pool_instances(="
                . $max_innodb_buffer_pool_instances
                . ")" );
        }
        else {
            goodprint "InnoDB buffer pool instances: "
                . $myvar{'innodb_buffer_pool_instances'} . "";
        }

        # InnoDB Buffer Pool Size < 1Go

```

```

    }
    else {
        if ( $myvar{'innodb_buffer_pool_instances'} != 1 ) {
            badprint
            "InnoDB buffer pool <= 1G and Innodb_buffer_pool_instances(!=1).";
            push( @adjvars, "innodb_buffer_pool_instances (=1)" );
        }
        else {
            goodprint "InnoDB buffer pool instances: "
                . $myvar{'innodb_buffer_pool_instances'} . "";
        }
    }
}

# InnoDB Used Buffer Pool Size vs CHUNK size
if ( !defined( $myvar{'innodb_buffer_pool_chunk_size'} ) ) {
    infoprint
        "InnoDB Buffer Pool Chunk Size not used or defined in your version";
}
else {
    infoprint "Number of InnoDB Buffer Pool Chunk : "
        . int( $myvar{'innodb_buffer_pool_size'} ) /
        int( $myvar{'innodb_buffer_pool_chunk_size'} ) . " for "
        . $myvar{'innodb_buffer_pool_instances'}
        . " Buffer Pool Instance(s)";

    if (
        int( $myvar{'innodb_buffer_pool_size'} ) % (
            int( $myvar{'innodb_buffer_pool_chunk_size'} ) *
            int( $myvar{'innodb_buffer_pool_instances'} )
        ) eq 0
    )
    {
        goodprint
            "Innodb_buffer_pool_size aligned with Innodb_buffer_pool_chunk_size &
            Innodb_buffer_pool_instances";
    }
    else {
        badprint
            "Innodb_buffer_pool_size aligned with Innodb_buffer_pool_chunk_size &
            Innodb_buffer_pool_instances";

        #push( @adjvars, "Adjust innodb_buffer_pool_instances,
        innodb_buffer_pool_chunk_size with innodb_buffer_pool_size" );
        push( @adjvars,
            "innodb_buffer_pool_size must always be equal to or a multiple of
            innodb_buffer_pool_chunk_size * innodb_buffer_pool_instances"
        );
    }
}

```

```

}

# InnoDB Read efficiency
if ( defined $mycalc{'pct_read_efficiency'}
    && $mycalc{'pct_read_efficiency'} < 90 )
{
    badprint "InnoDB Read buffer efficiency: "
        . $mycalc{'pct_read_efficiency'} . "% ("
        . ( $mystat{'Innodb_buffer_pool_read_requests'} -
            $mystat{'Innodb_buffer_pool_reads'} )
        . " hits/ "
        . $mystat{'Innodb_buffer_pool_read_requests'}
        . " total)";
}
else {
    goodprint "InnoDB Read buffer efficiency: "
        . $mycalc{'pct_read_efficiency'} . "% ("
        . ( $mystat{'Innodb_buffer_pool_read_requests'} -
            $mystat{'Innodb_buffer_pool_reads'} )
        . " hits/ "
        . $mystat{'Innodb_buffer_pool_read_requests'}
        . " total)";
}

# InnoDB Write efficiency
if ( defined $mycalc{'pct_write_efficiency'}
    && $mycalc{'pct_write_efficiency'} < 90 )
{
    badprint "InnoDB Write Log efficiency: "
        . abs( $mycalc{'pct_write_efficiency'} ) . "% ("
        . abs( $mystat{'Innodb_log_write_requests'} -
            $mystat{'Innodb_log_writes'} )
        . " hits/ "
        . $mystat{'Innodb_log_write_requests'}
        . " total)";
}
else {
    goodprint "InnoDB Write log efficiency: "
        . $mycalc{'pct_write_efficiency'} . "% ("
        . ( $mystat{'Innodb_log_write_requests'} -
            $mystat{'Innodb_log_writes'} )
        . " hits/ "
        . $mystat{'Innodb_log_write_requests'}
        . " total)";
}

# InnoDB Log Waits
if ( defined $mystat{'Innodb_log_waits'}
    && $mystat{'Innodb_log_waits'} > 0 )

```

```

{
    badprint "InnoDB log waits: "
        . percentage( $mystat{'Innodb_log_waits'},
            $mystat{'Innodb_log_writes'} )
        . "% ("
        . $mystat{'Innodb_log_waits'}
        . " waits / "
        . $mystat{'Innodb_log_writes'}
        . " writes)";
    push( @adjvars,
        "innodb_log_buffer_size (>= "
        . hr_bytes_rnd( $myvar{'innodb_log_buffer_size'} )
        . ") " );
}
else {
    goodprint "InnoDB log waits: "
        . percentage( $mystat{'Innodb_log_waits'},
            $mystat{'Innodb_log_writes'} )
        . "% ("
        . $mystat{'Innodb_log_waits'}
        . " waits / "
        . $mystat{'Innodb_log_writes'}
        . " writes)";
}
$result{'Calculations'} = {%mycalc};
}

sub check_metadata_perf {
    subheaderprint "Analysis Performance Metrics";
    if ( defined $myvar{'innodb_stats_on_metadata'} ) {
        infoprint "innodb_stats_on_metadata: "
            . $myvar{'innodb_stats_on_metadata'};
        if ( $myvar{'innodb_stats_on_metadata'} eq 'ON' ) {
            badprint "Stat are updated during querying INFORMATION_SCHEMA.";
            push @adjvars, "SET innodb_stats_on_metadata = OFF";

            #Disabling innodb_stats_on_metadata
            select_one("SET GLOBAL innodb_stats_on_metadata = OFF;");
            return 1;
        }
    }
    goodprint "No stat updates during querying INFORMATION_SCHEMA.";
    return 0;
}

# Recommendations for Database metrics
sub mysql_databases {
    return if ( $opt{dbstat} == 0 );
}

```

```

subheaderprint "Database Metrics";
unless ( mysql_version_ge( 5, 5 ) ) {
    infoprint
"Skip Database metrics from information schema missing in this version";
    return;
}

@dblist = select_array(
"SELECT SCHEMA_NAME FROM information_schema.SCHEMATA WHERE SCHEMA_NAME NOT IN (
'mysql', 'performance_schema', 'information_schema', 'sys' );"
);
infoprint "There is " . scalar(@dblist) . " Database(s).";
my @totaldbinfo = split /\s/,
    select_one(
"SELECT SUM(TABLE_ROWS), SUM(DATA_LENGTH), SUM(INDEX_LENGTH) ,
SUM(DATA_LENGTH+INDEX_LENGTH),
COUNT(TABLE_NAME),COUNT(DISTINCT(TABLE_COLLATION)),COUNT(DISTINCT(ENGINE)) FROM
information_schema.TABLES WHERE TABLE_SCHEMA NOT IN ( 'mysql',
'performance_schema', 'information_schema', 'sys' );"
);
infoprint "All User Databases:";
infoprint " +-- TABLE : "
    . ( $totaldbinfo[4] eq 'NULL' ? 0 : $totaldbinfo[4] ) . "";
infoprint " +-- ROWS : "
    . ( $totaldbinfo[0] eq 'NULL' ? 0 : $totaldbinfo[0] ) . "";
infoprint " +-- DATA : "
    . hr_bytes( $totaldbinfo[1] ) . "("
    . percentage( $totaldbinfo[1], $totaldbinfo[3] ) . "%)";
infoprint " +-- INDEX : "
    . hr_bytes( $totaldbinfo[2] ) . "("
    . percentage( $totaldbinfo[2], $totaldbinfo[3] ) . "%)";
infoprint " +-- SIZE : " . hr_bytes( $totaldbinfo[3] ) . "";
infoprint " +-- COLLA : "
    . ( $totaldbinfo[5] eq 'NULL' ? 0 : $totaldbinfo[5] ) . " ("
    . (
        join ", ",
        select_array(
            "SELECT DISTINCT(TABLE_COLLATION) FROM information_schema.TABLES;"
        ) . ")";
infoprint " +-- ENGIN : "
    . ( $totaldbinfo[6] eq 'NULL' ? 0 : $totaldbinfo[6] ) . " ("
    . (
        join ", ",
        select_array("SELECT DISTINCT(ENGINE) FROM information_schema.TABLES;"
        ) . ")";

$result{'Databases'}{'All databases'}{'Rows'} =
    ( $totaldbinfo[0] eq 'NULL' ? 0 : $totaldbinfo[0] );
$result{'Databases'}{'All databases'}{'Data Size'} = $totaldbinfo[1];

```

```

$result{'Databases'}{'All databases'}{'Data Pct'} =
    percentage( $totaldbinfo[1], $totaldbinfo[3] ) . "%";
$result{'Databases'}{'All databases'}{'Index Size'} = $totaldbinfo[2];
$result{'Databases'}{'All databases'}{'Index Pct'} =
    percentage( $totaldbinfo[2], $totaldbinfo[3] ) . "%";
$result{'Databases'}{'All databases'}{'Total Size'} = $totaldbinfo[3];
print "\n" unless ( $opt{'silent'} or $opt{'json'} );

foreach (@dblist) {
    my @dbinfo = split /\s/,
        select_one(
            "SELECT TABLE_SCHEMA, SUM(TABLE_ROWS), SUM(DATA_LENGTH), SUM(INDEX_LENGTH) ,
            SUM(DATA_LENGTH+INDEX_LENGTH), COUNT(DISTINCT
            ENGINE),COUNT(TABLE_NAME),COUNT(DISTINCT(TABLE_COLLATION)),COUNT(DISTINCT(ENGINE))
            FROM information_schema.TABLES WHERE TABLE_SCHEMA='$_' GROUP BY
            TABLE_SCHEMA ORDER BY TABLE_SCHEMA"
        );
    next unless defined $dbinfo[0];
    infoprint "Database: " . $dbinfo[0] . " ";
    infoprint " +-- TABLE: "
        . ( !defined( $dbinfo[6] ) or $dbinfo[6] eq 'NULL' ? 0 : $dbinfo[6] )
        . " ";
    infoprint " +-- COLL : "
        . ( $dbinfo[7] eq 'NULL' ? 0 : $dbinfo[7] ) . " ("
        . (
            join ", ",
            select_array(
                "SELECT DISTINCT(TABLE_COLLATION) FROM information_schema.TABLES WHERE
                TABLE_SCHEMA='$_';"
            )
        ) . ")";
    infoprint " +-- ROWS : "
        . ( !defined( $dbinfo[1] ) or $dbinfo[1] eq 'NULL' ? 0 : $dbinfo[1] )
        . " ";
    infoprint " +-- DATA : "
        . hr_bytes( $dbinfo[2] ) . "("
        . percentage( $dbinfo[2], $dbinfo[4] ) . "%)";
    infoprint " +-- INDEX: "
        . hr_bytes( $dbinfo[3] ) . "("
        . percentage( $dbinfo[3], $dbinfo[4] ) . "%)";
    infoprint " +-- TOTAL: " . hr_bytes( $dbinfo[4] ) . " ";
    infoprint " +-- ENGIN : "
        . ( $dbinfo[8] eq 'NULL' ? 0 : $dbinfo[8] ) . " ("
        . (
            join ", ",
            select_array(
                "SELECT DISTINCT(ENGINE) FROM information_schema.TABLES WHERE
                TABLE_SCHEMA='$_'"
            )
        )
    );
}

```

```

) . "));
badprint "Index size is larger than data size for $dbinfo[0] \n"
if ( $dbinfo[2] ne 'NULL' )
and ( $dbinfo[3] ne 'NULL' )
and ( $dbinfo[2] < $dbinfo[3] );
badprint "There are " . $dbinfo[5] . " storage engines. Be careful. \n"
if $dbinfo[5] > 1;
$result{'Databases'}{ $dbinfo[0] }{'Rows'} = $dbinfo[1];
$result{'Databases'}{ $dbinfo[0] }{'Tables'} = $dbinfo[6];
$result{'Databases'}{ $dbinfo[0] }{'Collations'} = $dbinfo[7];
$result{'Databases'}{ $dbinfo[0] }{'Data Size'} = $dbinfo[2];
$result{'Databases'}{ $dbinfo[0] }{'Data Pct'} =
percentage( $dbinfo[2], $dbinfo[4] ) . "%";
$result{'Databases'}{ $dbinfo[0] }{'Index Size'} = $dbinfo[3];
$result{'Databases'}{ $dbinfo[0] }{'Index Pct'} =
percentage( $dbinfo[3], $dbinfo[4] ) . "%";
$result{'Databases'}{ $dbinfo[0] }{'Total Size'} = $dbinfo[4];

if ( $dbinfo[7] > 1 ) {
badprint $dbinfo[7]
. " different collations for database "
. $dbinfo[0];
push( @generalrec,
"Check all table collations are identical for all tables in "
. $dbinfo[0]
. " database." );
}
else {
goodprint $dbinfo[7]
. " collation for "
. $dbinfo[0]
. " database.";
}
if ( $dbinfo[8] > 1 ) {
badprint $dbinfo[8]
. " different engines for database "
. $dbinfo[0];
push( @generalrec,
"Check all table engines are identical for all tables in "
. $dbinfo[0]
. " database." );
}
else {
goodprint $dbinfo[8] . " engine for " . $dbinfo[0] . " database.";
}

my @distinct_column_charset = select_array(
"select DISTINCT(CHARACTER_SET_NAME) from information_schema.COLUMNS where
CHARACTER_SET_NAME IS NOT NULL AND TABLE_SCHEMA ='$_'"

```

```

);
infoprint "Charsets for $dbinfo[0] database table column: "
    . join( ' ', ' ', @distinct_column_charset );
if ( scalar(@distinct_column_charset) > 1 ) {
    badprint $dbinfo[0]
        . " table column(s) has several charsets defined for all text like
column(s).";
    push( @generalrec,
        "Limit charset for column to one charset if possible for "
        . $dbinfo[0]
        . " database." );
}
else {
    goodprint $dbinfo[0]
        . " table column(s) has same charset defined for all text like
column(s).";
}

my @distinct_column_collation = select_array(
    "select DISTINCT(COLLATION_NAME) from information_schema.COLUMNS where
    COLLATION_NAME IS NOT NULL AND TABLE_SCHEMA = '$_'"
);
infoprint "Collations for $dbinfo[0] database table column: "
    . join( ' ', ' ', @distinct_column_collation );
if ( scalar(@distinct_column_collation) > 1 ) {
    badprint $dbinfo[0]
        . " table column(s) has several collations defined for all text
like column(s).";
    push( @generalrec,
        "Limit collations for column to one collation if possible for "
        . $dbinfo[0]
        . " database." );
}
else {
    goodprint $dbinfo[0]
        . " table column(s) has same collation defined for all text like
column(s).";
}
}

}

# Recommendations for database columns
sub mysql_tables {
    return if ( $opt{tbstat} == 0 );

    subheaderprint "Table Column Metrics";
    unless ( mysql_version_ge( 5, 5 ) ) {
        infoprint

```

```

"Skip Database metrics from information schema missing in this version";
    return;
}
if ( mysql_version_ge(8) and not mysql_version_eq(10) ) {
    infoprint
"MySQL and Percona version 8 and greater have remove PROCEDURE ANALYSE
feature";
}
foreach (@dblist) {
    my $dbname = $_;
    next unless defined $_;
    infoprint "Database: " . $_ . " ";
    my @dbtable = select_array(
"SELECT TABLE_NAME FROM information_schema.TABLES WHERE TABLE_SCHEMA='$dbname'
AND TABLE_TYPE='BASE TABLE' ORDER BY TABLE_NAME"
    );
    foreach (@dbtable) {
        my $tname = $_;
        infoprint " +-- TABLE: $tname";
        my @tblcol = select_array(
"SELECT COLUMN_NAME FROM information_schema.COLUMNS WHERE
TABLE_SCHEMA='$dbname' AND TABLE_NAME='$tname'"
        );
        foreach (@tblcol) {
            my $ctype = select_one(
"SELECT COLUMN_TYPE FROM information_schema.COLUMNS WHERE
TABLE_SCHEMA='$dbname' AND TABLE_NAME='$tname' AND COLUMN_NAME='$_' "
            );
            my $isnull = select_one(
"SELECT IS_NULLABLE FROM information_schema.COLUMNS WHERE
TABLE_SCHEMA='$dbname' AND TABLE_NAME='$tname' AND COLUMN_NAME='$_' "
            );
            infoprint "      +-- Column $tname.$_: ";
            my $current_type =
                uc($ctype) . ( $isnull eq 'NO' ? " NOT NULL" : "" );
            my $optimal_type = '';

            if ($opt{colstat} == 1) {
                $optimal_type = select_str_g( "Optimal_fieldtype",
"SELECT \\\$_\\ FROM \\\$_dbname\\ \\\$_tname\\ PROCEDURE ANALYSE(100000)"
                ) unless ( mysql_version_ge(8) and not mysql_version_eq(10) );
            }
            if ( $optimal_type eq '' ) {
                infoprint "          Current Fieldtype: $current_type";

                #infoprint "          Optimal Fieldtype: Not available";
            }
            elsif ( $current_type ne $optimal_type

```

```

        and $current_type !~ /. *DATETIME.*/
        and $current_type !~ /. *TIMESTAMP.*/ )
    {
        infoprint "        Current Fieldtype: $current_type";
        if ( $optimal_type =~ /. *ENUM\(. */ ) {
            $optimal_type = "ENUM( ... )";
        }
        infoprint "        Optimal Fieldtype: $optimal_type ";
        if ( $optimal_type !~ /. *ENUM\(. */ ) {
            badprint
"Consider changing type for column $_ in table $dbname.$tbname";
            push( @generalrec,
"ALTER TABLE \"$dbname\".\"$tbname\" MODIFY \"$_\" $optimal_type;"
            );
        }
    }

    }
else {
    goodprint "$dbname.$tbname ($_) type: $current_type";
}
}

}

# Recommendations for Indexes metrics
sub mysql_indexes {
    return if ( $opt{idxstat} == 0 );

    subheaderprint "Indexes Metrics";
    unless ( mysql_version_ge( 5, 5 ) ) {
        infoprint
            "Skip Index metrics from information schema missing in this version";
        return;
    }

    # unless ( mysql_version_ge( 5, 6 ) ) {
    #     infoprint
    # "Skip Index metrics from information schema due to erroneous information
    # provided in this version";
    #     return;
    # }

    my $selIdxReq = <<'ENDSQL';
SELECT
    CONCAT(CONCAT(t.TABLE_SCHEMA, '.'),t.TABLE_NAME) AS 'table'
, CONCAT(CONCAT(CONCAT(s.INDEX_NAME, '('),s.COLUMN_NAME), ')') AS 'index'
, s.SEQ_IN_INDEX AS 'seq'
, s2.max_columns AS 'maxcol'

```

```

, s.CARDINALITY AS 'card'
, t.TABLE_ROWS AS 'est_rows'
, INDEX_TYPE as type
, ROUND((s.CARDINALITY / IFNULL(t.TABLE_ROWS, 0.01)) * 100), 2) AS 'sel'
FROM INFORMATION_SCHEMA.STATISTICS s
INNER JOIN INFORMATION_SCHEMA.TABLES t
  ON s.TABLE_SCHEMA = t.TABLE_SCHEMA
  AND s.TABLE_NAME = t.TABLE_NAME
INNER JOIN (
  SELECT
    TABLE_SCHEMA
  , TABLE_NAME
  , INDEX_NAME
  , MAX(SEQ_IN_INDEX) AS max_columns
  FROM INFORMATION_SCHEMA.STATISTICS
  WHERE TABLE_SCHEMA NOT IN ('mysql', 'information_schema',
'performance_schema')
  AND INDEX_TYPE <> 'FULLTEXT'
  GROUP BY TABLE_SCHEMA, TABLE_NAME, INDEX_NAME
) AS s2
ON s.TABLE_SCHEMA = s2.TABLE_SCHEMA
AND s.TABLE_NAME = s2.TABLE_NAME
AND s.INDEX_NAME = s2.INDEX_NAME
WHERE t.TABLE_SCHEMA NOT IN ('mysql', 'information_schema',
'performance_schema')
AND t.TABLE_ROWS > 10
AND s.CARDINALITY IS NOT NULL
AND (s.CARDINALITY / IFNULL(t.TABLE_ROWS, 0.01)) < 8.00
ORDER BY sel
LIMIT 10;
ENDSQL

my @idxinfo = select_array($selIdxReq);
infoprint "Worst selectivity indexes:";
foreach (@idxinfo) {
  debugprint "$_";
  my @info = split /\s/;
  infoprint "Index: " . $info[1] . " ";

  infoprint " +-- COLUMN      : " . $info[0] . " ";
  infoprint " +-- NB SEQS     : " . $info[2] . " sequence(s)";
  infoprint " +-- NB COLS     : " . $info[3] . " column(s)";
  infoprint " +-- CARDINALITY : " . $info[4] . " distinct values";
  infoprint " +-- NB ROWS     : " . $info[5] . " rows";
  infoprint " +-- TYPE         : " . $info[6];
  infoprint " +-- SELECTIVITY : " . $info[7] . "%";

  $result{'Indexes'}{ $info[1] }{'Column'} = $info[0];
  $result{'Indexes'}{ $info[1] }{'Sequence number'} = $info[2];
  $result{'Indexes'}{ $info[1] }{'Number of column'} = $info[3];

```

```

        $result{'Indexes'}{ $info[1] }{'Cardinality'}      = $info[4];
        $result{'Indexes'}{ $info[1] }{'Row number'}      = $info[5];
        $result{'Indexes'}{ $info[1] }{'Index Type'}      = $info[6];
        $result{'Indexes'}{ $info[1] }{'Selectivity'}     = $info[7];
        if ( $info[7] < 25 ) {
            badprint "$info[1] has a low selectivity";
        }
    }

    return

    unless ( defined( $myvar{'performance_schema'} )
        and $myvar{'performance_schema'} eq 'ON' );

    $selIdxReq = <<'ENDSQL';
    SELECT CONCAT(CONCAT(object_schema, '.'), object_name) AS 'table', index_name
    FROM performance_schema.table_io_waits_summary_by_index_usage
    WHERE index_name IS NOT NULL
    AND count_star = 0
    AND index_name <> 'PRIMARY'
    AND object_schema NOT IN ( 'mysql', 'performance_schema', 'information_schema'
    )
    ORDER BY count_star, object_schema, object_name;
    ENDSQL

    @idxinfo = select_array($selIdxReq);
    infoprint "Unused indexes:";
    push( @generalrec, "Remove unused indexes." ) if ( scalar(@idxinfo) > 0 );
    foreach (@idxinfo) {
        debugprint "$_";
        my @info = split /\s/;
        badprint "Index: $info[1] on $info[0] is not used.";
        push @{ $result{'Indexes'}{'Unused Indexes'} },
            $info[0] . "." . $info[1];
    }
}

# Take the two recommendation arrays and display them at the end of the output
sub make_recommendations {
    $result{'Recommendations'} = \@generalrec;
    $result{'Adjust variables'} = \@adjvars;
    subheaderprint "Recommendations";
    if ( @generalrec > 0 ) {
        prettyprint "General recommendations:";
        foreach (@generalrec) { prettyprint "    " . $_ . " "; }
    }
    if ( @adjvars > 0 ) {
        prettyprint "Variables to adjust:";
        if ( $mycalc{'pct_max_physical_memory'} > 90 ) {
            prettyprint
                "    *** MySQL's maximum memory usage is dangerously high ***\n"
        }
    }
}

```

```

        . "    *** Add RAM before increasing MySQL buffer variables ***";
    }
    foreach (@adjvars) { prettyprint "    " . $_ . " "; }
}
if ( @generalrec == 0 && @adjvars == 0 ) {
    prettyprint "No additional performance recommendations are available.";
}
}

sub close_outputfile {
    close($fh) if defined($fh);
}

sub headerprint {
    prettyprint
        " >> MySQLTuner $tunerverversion - Major Hayden <major@mhtx.net>\n"
        . " >> Bug reports, feature requests, and downloads at
http://mysqлтuner.pl/\n"
        . " >> Run with '--help' for additional options and output filtering";
}

sub string2file {
    my $filename = shift;
    my $content = shift;
    open my $fh, q(>), $filename
        or die
        "Unable to open $filename in write mode. Please check permissions for this file
or directory";
    print $fh $content if defined($content);
    close $fh;
    debugprint $content if ( $opt{'debug'} );
}

sub file2array {
    my $filename = shift;
    debugprint "* reading $filename" if ( $opt{'debug'} );
    my $fh;
    open( $fh, q(<), "$filename" )
        or die "Couldn't open $filename for reading: $!\n";
    my @lines = <$fh>;
    close($fh);
    return @lines;
}

sub file2string {
    return join( ' ', file2array(@_) );
}

my $templateModel;

```

```

if ( $opt{'template'} ne 0 ) {
    $templateModel = file2string( $opt{'template'} );
}
else {
    # DEFAULT REPORT TEMPLATE
    $templateModel = <<'END_TEMPLATE';
<!DOCTYPE html>
<html>
<head>
    <title>MySQLTuner Report</title>
    <meta charset="UTF-8">
</head>
<body>

<h1>Result output</h1>
<pre>
{$data}
</pre>

</body>
</html>
END_TEMPLATE
}

sub dump_result {
    debugprint Dumper( \%result ) if ( $opt{'debug'} );
    debugprint "HTML REPORT: $opt{'reportfile'}";

    if ( $opt{'reportfile'} ne 0 ) {
        eval { require Text::Template };
        eval { require JSON };
        if ($?) {
            badprint "Text::Template Module is needed.";
            die "Text::Template Module is needed.";
        }

        my $json      = JSON->new->allow_nonref;
        my $json_text = $json->pretty->encode( \%result );
        my %vars      = (
            'data' => \%result,
            'debug' => $json_text,
        );
        my $template;
        {
            no warnings 'once';
            $template = Text::Template->new(
                TYPE      => 'STRING',
                PREPEND    => q{;},
                SOURCE     => $templateModel,
            );
        }
    }
}

```

```

        DELIMITERS => [ '[%', '%]' ]
    ) or die "Couldn't construct template: $Text::Template::ERROR";
}

open my $fh, q(>), $opt{'reportfile'}
or die
"Unable to open $opt{'reportfile'} in write mode. please check permissions for
this file or directory";
$template->fill_in( HASH => \%vars, OUTPUT => $fh );
close $fh;
}

if ( $opt{'json'} ne 0 ) {
    eval { require JSON };
    if ($?) {
        print "$bad JSON Module is needed.\n";
        return 1;
    }

    my $json = JSON->new->allow_nonref;
    print $json->utf8(1)->pretty( ( $opt{'prettyjson'} ? 1 : 0 ) )
        ->encode( \%result );

    if ( $opt{'outputfile'} ne 0 ) {
        unlink $opt{'outputfile'} if ( -e $opt{'outputfile'} );
        open my $fh, q(>), $opt{'outputfile'}
        or die
"Unable to open $opt{'outputfile'} in write mode. please check permissions for
this file or directory";
        print $fh $json->utf8(1)->pretty( ( $opt{'prettyjson'} ? 1 : 0 ) )
            ->encode( \%result );
        close $fh;
    }
}

}

sub which {
    my $prog_name = shift;
    my $path_string = shift;
    my @path_array = split /:/, $ENV{'PATH'};

    for my $path (@path_array) {
        return "$path/$prog_name" if ( -x "$path/$prog_name" );
    }

    return 0;
}

# -----

```

```

# BEGIN 'MAIN'
# -----
headerprint;      # Header Print

validate_tuner_version;    # Check last version
mysql_setup;             # Gotta login first
debugprint "MySQL FINAL Client : $mysqlcmd $mysqllogin";
debugprint "MySQL Admin FINAL Client : $mysqladmincmd $mysqllogin";

#exit(0);

os_setup;              # Set up some OS variables
get_all_vars;          # Toss variables/status into hashes
get_tuning_info;       # Get information about the tuning connexion
validate_mysql_version; # Check current MySQL version

check_architecture;     # Suggest 64-bit upgrade
system_recommendations; # avoid to many service on the same host
log_file_recommendations; # check log file content
check_storage_engines;  # Show enabled storage engines

check_metadata_perf;    # Show parameter impacting performance during analysis
mysql_databases;        # Show informations about databases
mysql_tables;           # Show informations about table column

mysql_indexes;          # Show informations about indexes
security_recommendations; # Display some security recommendations
cve_recommendations;    # Display related CVE
calculations;           # Calculate everything we need
mysql_stats;            # Print the server stats
mysqlsl_pfs;            # Print Performance schema info
mariadb_threadpool;     # Print MariaDB ThreadPool stats
mysql_myisam;           # Print MyISAM stats
mysql_innodb;           # Print InnoDB stats
mariadb_aria;           # Print MariaDB Aria stats
mariadb_tokudb;         # Print MariaDB Tokudb stats
mariadb_xtradb;         # Print MariaDB XtraDB stats

#mariadb_rockdb;        # Print MariaDB RockDB stats
#mariadb_spider;        # Print MariaDB Spider stats
#mariadb_connect;       # Print MariaDB Connect stats
mariadb_galera;         # Print MariaDB Galera Cluster stats
get_replication_status; # Print replication info
make_recommendations;    # Make recommendations based on stats
dump_result;            # Dump result if debug is on
close_outputfile;       # Close reportfile if needed

# -----
# END 'MAIN'
# -----

```

```
1;
```

```
__END__
```

```
=pod
```

```
=encoding UTF-8
```

```
=head1 NAME
```

MySQLTuner 1.7.24 - MySQL High Performance Tuning Script

```
=head1 IMPORTANT USAGE GUIDELINES
```

To run the script with the default options, run the script without arguments
Allow MySQL server to run for at least 24-48 hours before trusting suggestions
Some routines may require root level privileges (script will provide warnings)
You must provide the remote server's total memory when connecting to other servers

```
=head1 CONNECTION AND AUTHENTICATION
```

--host <hostname>	Connect to a remote host to perform tests (default: localhost)
--socket <socket>	Use a different socket for a local connection
--port <port>	Port to use for connection (default: 3306)
--user <username>	Username to use for authentication
--userenv <envvar>	Name of env variable which contains username to use for authentication
--pass <password>	Password to use for authentication
--passenv <envvar>	Name of env variable which contains password to use for authentication
--ssl-ca <path>	Path to public key
--mysqladmin <path>	Path to a custom mysqladmin executable
--mysqlcmd <path>	Path to a custom mysql executable
--defaults-file <path>	Path to a custom .my.cnf
--server-log <path>	Path to explicit log file

```
=head1 PERFORMANCE AND REPORTING OPTIONS
```

--skipsize (default: on)	Don't enumerate tables and their types/sizes (Recommended for servers with many tables)
--skippassword off)	Don't perform checks on user passwords (default: off)
--checkversion check)	Check for updates to MySQLTuner (default: don't check)
--updateversion	Check for updates to MySQLTuner and update when newer version is available (default: don't check)

--forcemem <size>	Amount of RAM installed in megabytes
--forceswap <size>	Amount of swap memory configured in megabytes
--passwordfile <path>	Path to a password file list(one password by line)

=head1 OUTPUT OPTIONS

--silent	Don't output anything on screen
--nogood	Remove OK responses
--nobad	Remove negative/suggestion responses
--noinfo	Remove informational responses
--debug	Print debug information
--noprocess	Consider no other process is running
--dbstat	Print database information
--nodbstat	Don't Print database information
--tbstat	Print table information
--notbstat	Don't Print table information
--colstat	Print column information
--nocolstat	Don't Print column information
--idxstat	Print index information
--noidxstat	Don't Print index information
--sysstat	Print system information
--nosysstat	Don't Print system information
--pfstat	Print Performance schema
--nopfstat	Don't Print Performance schema
--verbose	Prints out all options (default: no verbose,
dbstat, idxstat, sysstat, tbstat, pfstat)	
--bannedports	Ports banned separated by comma(,)
--maxportallowed	Number of ports opened allowed on this hosts
--cvefile <path>	CVE File for vulnerability checks
--nocolor	Don't print output in color
--json	Print result as JSON string
--buffers	Print global and per-thread buffer values
--outputfile <path>	Path to a output txt file
--reportfile <path>	Path to a report txt file
--template <path>	Path to a template file

=head1 PERLDOC

You can find documentation for this module with the perldoc command.

```
perldoc mysqltuner
```

=head2 INTERNALS

L<<https://github.com/major/MySQLTuner-perl/blob/master/INTERNALS.md>>

Internal documentation

=head1 AUTHORS

Major Hayden - major@mhtx.net

=head1 CONTRIBUTORS

=over 4

=item *

Matthew Montgomery

=item *

Paul Kehrer

=item *

Dave Burgess

=item *

Jonathan Hinds

=item *

Mike Jackson

=item *

Nils Breunese

=item *

Shawn Ashlee

=item *

Luuk Vosslander

=item *

Ville Skytta

=item *

Trent Hornibrook

=item *

Jason Gill

=item *

Mark Imbriaco

=item *

Greg Eden

=item *

Aubin Galinotti

=item *

Giovanni Bechis

=item *

Bill Bradford

=item *

Ryan Novosielski

=item *

Michael Scheidell

=item *

Blair Christensen

=item *

Hans du Plooy

=item *

Victor Trac

=item *

Everett Barnes

=item *

Tom Krouper

=item *

Gary Barrueto

=item *

Simon Greenaway

=item *

Adam Stein

=item *

Isart Montane

=item *

Baptiste M.

=item *

Cole Turner

=item *

Major Hayden

=item *

Joe Ashcraft

=item *

Jean-Marie Renouard

=item *

Stephan GroBberndt

=item *

Christian Loos

=back

=head1 SUPPORT

Bug reports, feature requests, [and](http://mysqltuner.pl/) downloads at <http://mysqltuner.pl/>

Bug tracker can be found at <https://github.com/major/MySQLTuner-perl/issues>

Maintained by Major Hayden (major@mhtx.net) - Licensed under GPL

=head1 SOURCE CODE

L<<https://github.com/major/MySQLTuner-perl>>

git clone <https://github.com/major/MySQLTuner-perl.git>

=head1 COPYRIGHT AND LICENSE

Copyright (C) [2006-2020](#) Major Hayden - major@mhtx.net

For the latest updates, please visit <http://mysqltuner.pl/>

Git repository available at <https://github.com/major/MySQLTuner-perl>

This program is free software: you can redistribute it [and/or](#) modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version [3](#) of the License, [or](#) (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY [or](#) FITNESS FOR A PARTICULAR PURPOSE.

See the GNU General Public License [for](#) more details.

You should have received a copy of the GNU General Public License along with this program. If not, see [<https://www.gnu.org/licenses/>](https://www.gnu.org/licenses/).

=cut

```
# Local variables:
# indent-tabs-mode: t
# cperl-indent-level: 8
# perl-indent-level: 8
# End:
```